

DOCUMENT DE RÉVISION

# SmartAutoTrack

Questions probables du jury sur la base de données, réponses détaillées et commandes SQL expliquées

---

Base MySQL 8 « charles » · 19 tables · requêtes testées sur la base réelle

Version du dépôt : 8e01fd1 · 2 octobre 2026

# Sommaire

1. Comprendre ma base : le modèle de données
2. Clés, contraintes et intégrité
3. Les commandes SQL essentielles
4. Requêtes avancées : jointures, regroupements, sous-requêtes
5. Transactions, sécurité et performances
6. Aide-mémoire et entraînement
7. Questions pièges sur les mots de passe

# 1. Comprendre ma base : le modèle de données

Ce chapitre prépare les questions « de fond » du jury : à quoi sert une base de données, comment la tienne est organisée, et pourquoi tu as fait ces choix. Toutes les requêtes montrées ici ont été exécutées sur ta vraie base `charles` (MySQL 8.0.41), en lecture seule. Les résultats affichés sont ceux obtenus réellement au moment de la rédaction ; ils changeront si tes données changent.

## Avant de commencer : lire une requête SELECT

Presque toutes les démonstrations de ce chapitre utilisent la commande `SELECT`, qui sert à **lire** des données sans rien modifier. Voici le sens des mots-clés que tu vas croiser :

| Mot-clé                                  | Ce qu'il fait                                                                                                                                                         |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SELECT</code>                      | Indique les colonnes (ou les calculs) que l'on veut voir dans le résultat.                                                                                            |
| <code>AS</code>                          | Donne un nom (un « alias ») à une colonne ou une table du résultat, pour le rendre lisible.                                                                           |
| <code>FROM</code>                        | Indique la table dans laquelle on lit.                                                                                                                                |
| <code>WHERE</code>                       | Filtre : ne garde que les lignes qui respectent la condition.                                                                                                         |
| <code>JOIN ...</code><br><code>ON</code> | Relie deux tables : on associe les lignes dont les colonnes indiquées après <code>ON</code> sont égales.                                                              |
| <code>LEFT JOIN</code>                   | Comme <code>JOIN</code> , mais garde aussi les lignes de la table de gauche qui n'ont pas de correspondance (les colonnes de droite valent alors <code>NULL</code> ). |
| <code>GROUP BY</code>                    | Regroupe les lignes qui ont la même valeur, pour calculer un total par groupe.                                                                                        |
| <code>COUNT(*)</code>                    | Compte le nombre de lignes (dans tout le résultat, ou dans chaque groupe).                                                                                            |
| <code>ORDER BY</code>                    | Trie le résultat ( <code>ASC</code> croissant par défaut, <code>DESC</code> décroissant).                                                                             |
| <code>LIMIT n</code>                     | Ne garde que les <code>n</code> premières lignes.                                                                                                                     |

### Q1. Qu'est-ce qu'une base de données relationnelle, et qu'est-ce qu'un SGBD ?

Une **base de données relationnelle** range les informations dans des **tables**. Une table ressemble à un tableau : chaque **ligne** (on dit aussi « enregistrement » ou « tuple ») représente un objet du monde réel, par exemple un véhicule, et chaque **colonne** (ou « attribut ») représente une caractéristique de cet objet, par exemple sa marque ou son kilométrage.

Le mot « relationnelle » vient du fait que les tables sont **reliées entre elles** par des valeurs communes. Dans ma base, la table `vehicule` possède une colonne `idClient` qui contient le numéro du client propriétaire : c'est ce lien qui permet de savoir à qui appartient chaque véhicule, sans recopier les informations du client dans chaque ligne de véhicule.

Deux notions sont centrales :

- La **clé primaire** ( `PRIMARY KEY` ) est la colonne qui identifie chaque ligne de façon unique. Exemple : `idVehicule` dans `vehicule`. Deux véhicules ne peuvent pas avoir le même `idVehicule`, et cette valeur ne peut pas être vide.
- La **clé étrangère** ( `FOREIGN KEY` ) est une colonne qui contient la clé primaire d'une autre table. Exemple : `vehicule.idClient` pointe vers `client.idClient`. Le SGBD refuse alors d'enregistrer un

véhicule pour un client qui n'existe pas : c'est ce qu'on appelle l'**intégrité référentielle**.

Un **SGBD** (Système de Gestion de Base de Données) est le logiciel qui stocke ces tables, exécute les requêtes SQL, vérifie les contraintes (clés, types, unicité), gère les accès simultanés de plusieurs utilisateurs et protège les données en cas de panne. Dans mon projet, le SGBD est **MySQL 8.0**, fourni par XAMPP. **SQL** (Structured Query Language) est le langage qu'on utilise pour parler au SGBD.

On peut demander au SGBD sa version et le nom de la base utilisée :

```
SELECT VERSION() AS version, DATABASE() AS base;
```

VERSION() et DATABASE() sont des fonctions intégrées de MySQL ; il n'y a pas de FROM car on ne lit dans aucune table.

```
version | base
-----+-----
8.0.41  | charles
```

## Q2. Pourquoi avoir choisi MySQL, et qu'est-ce que InnoDB ?

J'ai choisi MySQL pour plusieurs raisons concrètes :

- Il est **gratuit et open source**, et il est livré avec **XAMPP** (Apache + MySQL + PHP), l'environnement que j'utilise pour développer l'application.
- Il s'intègre très bien avec **PHP** via l'extension **PDO**, que j'utilise avec des requêtes préparées (le fichier `config/database.php` ouvre la connexion avec `charset=utf8mb4`).
- C'est le SGBD le plus répandu chez les hébergeurs web, ce qui facilite un futur déploiement.
- Mes besoins (une vingtaine de tables, des volumes modestes, des transactions) correspondent exactement à ce que MySQL fait bien.

**InnoDB** est le **moteur de stockage** de MySQL, c'est-à-dire la partie qui écrit physiquement les données sur le disque. MySQL en propose plusieurs (InnoDB, MyISAM, MEMORY...). InnoDB est celui par défaut depuis MySQL 5.5, et c'est le seul des moteurs courants qui offre ce dont j'ai besoin :

- **Les clés étrangères** : MyISAM les accepte dans la syntaxe mais ne les vérifie pas. Avec InnoDB, mes 34 contraintes `FOREIGN KEY` sont réellement contrôlées.
- **Les transactions** (`BEGIN`, `COMMIT`, `ROLLBACK`) : à l'inscription (`auth/register.php`), je crée l'utilisateur, puis sa fiche `client`, puis `particulier` ou `entreprise`, dans une seule transaction. Si une étape échoue, tout est annulé.
- **Le verrouillage par ligne** : `SELECT ... FOR UPDATE`, utilisé dans `includes/payments.php`, bloque une seule ligne pendant le paiement au lieu de toute la table.
- **La reprise après panne** : InnoDB tient un journal qui lui permet de revenir à un état cohérent après un arrêt brutal.

On peut vérifier que les 19 tables utilisent bien InnoDB en interrogeant `information_schema`, une base spéciale de MySQL qui décrit toutes les autres bases (ses tables, ses colonnes, ses contraintes) :

```
SELECT ENGINE, COUNT(*) AS nb_tables
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
GROUP BY ENGINE;
```

- `information_schema.TABLES` : la table qui liste toutes les tables du serveur.

- `WHERE TABLE_SCHEMA = DATABASE()` : ne garde que les tables de ma base `charles`.
- `GROUP BY ENGINE + COUNT(*)` : compte les tables par moteur.

```
ENGINE | nb_tables
-----+-----
InnoDB | 19
```

### Piège fréquent

Le jury peut demander « MySQL et SQL, c'est la même chose ? ». Non : **SQL** est un langage normalisé ; **MySQL** est un logiciel (un SGBD) qui comprend ce langage. PostgreSQL, Oracle ou SQL Server sont d'autres SGBD qui parlent aussi SQL, avec de petites différences.

### Q3. Combien de tables contient ta base, et comment sont-elles organisées ?

Ma base contient **19 tables**. Je les regroupe en sept domaines fonctionnels :

| Domaine                   | Tables                                                                                                                                                                            | Rôle                                                                                                                                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Comptes                   | <code>utilisateur</code> , <code>administrateur</code> , <code>client</code> , <code>particulier</code> , <code>entreprise</code> , <code>technicien</code> , <code>garage</code> | Qui se connecte et avec quel rôle. <code>utilisateur</code> contient ce qui est commun (nom, email, mot de passe haché) ; les autres tables précisent le rôle. |
| Parc automobile           | <code>vehicule</code>                                                                                                                                                             | Les véhicules des clients (marque, modèle, immatriculation, numéro de châssis, kilométrage).                                                                   |
| Métier                    | <code>intervention</code> , <code>anomalie</code> , <code>reparation</code> , <code>paiement</code>                                                                               | Le cœur du suivi : une intervention est demandée sur un véhicule, des anomalies sont détectées, des réparations sont réalisées puis payées.                    |
| Communication             | <code>messages</code> , <code>notifications</code>                                                                                                                                | Messagerie interne entre utilisateurs et alertes envoyées à un utilisateur.                                                                                    |
| Traçabilité               | <code>journalactivites</code>                                                                                                                                                     | Historique des actions (qui a fait quoi, quand, sur quelle intervention, réparation ou anomalie).                                                              |
| Documents                 | <code>technician_documents</code> , <code>documentgarage</code>                                                                                                                   | Pièces justificatives envoyées par les techniciens (diplôme, certificat...) et par les garages, vérifiées par l'administrateur.                                |
| Intelligence artificielle | <code>assistantia</code> , <code>analyseia</code>                                                                                                                                 | <code>assistantia</code> stocke une analyse et sa recommandation ; <code>analyseia</code> la rattache à une anomalie, une intervention ou une réparation.      |

On peut compter les tables directement :

```
SELECT COUNT(*) AS nb_tables
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE();
```

```
nb_tables
-----
19
```

Et voir quelles tables sont les plus « référencées », c'est-à-dire celles vers lesquelles pointent le plus de clés étrangères. Cela montre les tables centrales du modèle :

```
SELECT REFERENCED_TABLE_NAME AS table_parente,  
       COUNT(*) AS nb_cles_etrangeres  
FROM information_schema.REFERENTIAL_CONSTRAINTS  
WHERE CONSTRAINT_SCHEMA = DATABASE()  
GROUP BY REFERENCED_TABLE_NAME  
ORDER BY nb_cles_etrangeres DESC;
```

REFERENTIAL\_CONSTRAINTS liste toutes les clés étrangères ; REFERENCED\_TABLE\_NAME est la table vers laquelle chaque clé pointe.

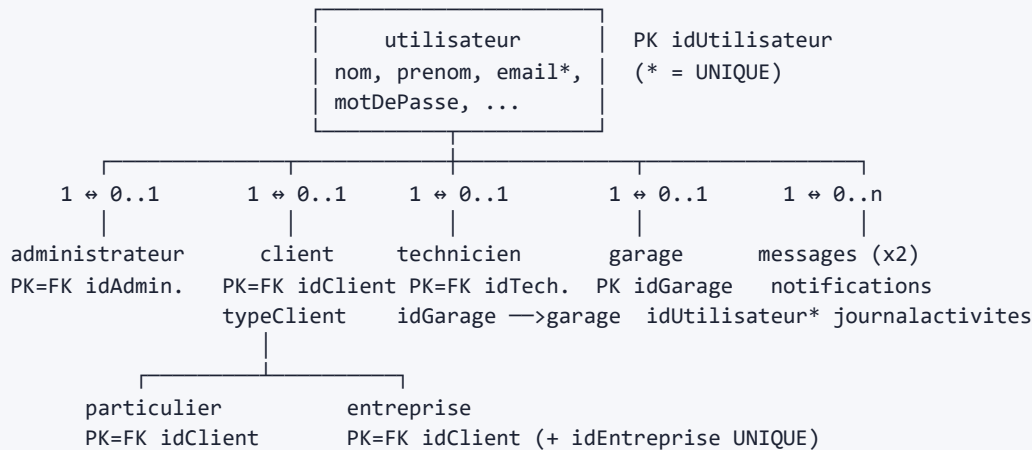
| table_parente | nb_cles_etrangeres |
|---------------|--------------------|
| utilisateur   | 8                  |
| intervention  | 5                  |
| garage        | 5                  |
| client        | 5                  |
| technicien    | 4                  |
| anomalie      | 2                  |
| reparation    | 2                  |
| vehicule      | 2                  |
| assistantia   | 1                  |

(total : 34 clés étrangères)

On voit que utilisateur et intervention sont les deux pivots de la base : l'un pour les comptes, l'autre pour le métier.

#### Q4. Peux-tu nous dessiner le schéma des relations entre tes tables ?

Voici le schéma relationnel simplifié. Une flèche  $A.x \rightarrow B$  signifie que la colonne  $x$  de la table  $A$  est une clé étrangère vers la clé primaire de  $B$ . Les indications  $1$ ,  $0..1$ ,  $0..n$  sont les cardinalités (expliquées à la question Q7).



#### PARC ET MÉTIER

|                                  |                                             |
|----------------------------------|---------------------------------------------|
| client 1 — 0..n vehicule         | (vehicule.idClient, vehicule.idGarage)      |
| vehicule 1 — 0..n intervention   | (intervention.idVehicule)                   |
| client 1 — 0..n intervention     | (intervention.idClient)                     |
| technicien 1 — 0..n intervention | (intervention.idTechnicien, peut être NULL) |
| garage 1 — 0..n intervention     | (intervention.idGarage, peut être NULL)     |
| vehicule 1 — 0..n anomalie       | (anomalie.idVehicule)                       |
| intervention 1 — 0..n anomalie   | (anomalie.idIntervention, peut être NULL)   |
| intervention 1 — 0..n reparation | (reparation.idIntervention, pas de UNIQUE)  |
| intervention 1 — 0..n paiement   | (paiement.idIntervention)                   |
| client 1 — 0..n paiement         | (paiement.idClient)                         |

#### DOCUMENTS

|                                          |                 |
|------------------------------------------|-----------------|
| technicien 1 — 0..n technician_documents | (technicien_id) |
| garage 1 — 0..n documentgarage           | (idGarage)      |

#### IA

|                                                   |                                  |
|---------------------------------------------------|----------------------------------|
| assistantia 1 — 0..1 analyseia                    | (analyseia.idAnalyse = PK et FK) |
| analyseia —> anomalie / intervention / reparation | (chacune facultative)            |

### Q5. Pourquoi une table « utilisateur » et des tables séparées pour chaque rôle ? Comment fonctionne cet héritage ?

Tous les acteurs de l'application (administrateur, client, technicien, gérant de garage) ont des informations **communes** : nom, prénom, email, mot de passe, téléphone, photo, thème. Ils ont aussi des informations **propres à leur rôle** : un technicien a une spécialité et un statut de validation, un client a un type (particulier ou entreprise), une entreprise a une raison sociale.

J'ai donc appliqué la **spécialisation** (on dit aussi « héritage » ou « généralisation/spécialisation » en Merise et UML) : la table mère **utilisateur** contient les colonnes communes, et chaque table fille contient les colonnes spécifiques. La technique utilisée est la **clé primaire partagée** : la clé primaire de la table fille est **aussi** une clé étrangère vers la table mère. Exemple dans **technicien** :

```

CREATE TABLE technicien (
    idTechnicien INT NOT NULL,
    ...
    PRIMARY KEY (idTechnicien),
    CONSTRAINT fk_technicien_utilisateur
        FOREIGN KEY (idTechnicien) REFERENCES utilisateur (idUtilisateur)
        ON DELETE CASCADE ON UPDATE CASCADE
);
  
```

- **PRIMARY KEY** (`idTechnicien`) : un technicien est identifié par ce numéro, et il ne peut exister qu'une seule fiche technicien par numéro.
- **FOREIGN KEY** (`idTechnicien`) **REFERENCES** `utilisateur` (`idUtilisateur`) : ce numéro doit exister dans `utilisateur`. Le technicien n°8 **est** l'utilisateur n°8.
- **ON DELETE CASCADE** : si on supprime l'utilisateur, sa fiche technicien est supprimée automatiquement.
- **ON UPDATE CASCADE** : si le numéro de l'utilisateur changeait, il serait mis à jour dans la fiche technicien.
- Remarque : `idTechnicien` n'a **pas** d'**AUTO\_INCREMENT**, car le numéro n'est pas inventé par la table fille : il est recopié depuis `utilisateur` (dans le code, avec `$conn->lastInsertId()`).

Le même schéma s'applique à `administrateur` et `client` (vers `utilisateur`), puis à `particulier` et `entreprise` (vers `client`) : c'est un héritage à deux niveaux.

### Pourquoi ce choix plutôt qu'une seule grande table ?

- Une table unique aurait beaucoup de colonnes vides (un client n'a pas de `specialite`, un technicien n'a pas de `raisonSociale`).
- Chaque rôle peut avoir ses propres contraintes (`raisonSociale` est **NOT NULL** dans `entreprise`, ce qui serait impossible dans une table commune).
- Les autres tables pointent exactement vers le bon rôle : `intervention.idTechnicien` pointe vers `technicien`, donc on ne peut pas y mettre un client par erreur.
- Le rôle n'est plus une simple colonne texte modifiable : il est matérialisé par l'existence d'une ligne dans la table fille.

On retrouve le rôle de chaque utilisateur par des jointures externes :

```
SELECT CASE
    WHEN a.idAdministrateur IS NOT NULL THEN 'ADMIN'
    WHEN c.idClient         IS NOT NULL THEN 'CLIENT'
    WHEN t.idTechnicien     IS NOT NULL THEN 'TECHNICIEN'
    WHEN g.idGarage         IS NOT NULL THEN 'GARAGE'
    ELSE 'AUCUN'
END AS role,
COUNT(*) AS nb
FROM utilisateur u
LEFT JOIN administrateur a ON a.idAdministrateur = u.idUtilisateur
LEFT JOIN client c        ON c.idClient         = u.idUtilisateur
LEFT JOIN technicien t    ON t.idTechnicien     = u.idUtilisateur
LEFT JOIN garage g        ON g.idUtilisateur    = u.idUtilisateur
GROUP BY role;
```

- **LEFT JOIN** : on garde chaque utilisateur, même s'il n'a pas de ligne dans la table du rôle (la colonne vaut alors **NULL**).
- **CASE WHEN ... THEN ... ELSE ... END** : c'est un « si... alors... sinon » ; le premier test vrai donne la valeur.
- **IS NOT NULL** : vrai si la jointure a trouvé une ligne. (On n'écrit jamais `= NULL`, qui ne fonctionne pas en SQL.)

| role       | nb |
|------------|----|
| CLIENT     | 5  |
| ADMIN      | 1  |
| TECHNICIEN | 8  |
| GARAGE     | 1  |

(15 utilisateurs au total, aucun sans rôle)

### Piège fréquent

Le jury peut remarquer que `garage` ne suit pas exactement le même modèle : sa clé primaire `idGarage` est indépendante (avec `AUTO_INCREMENT`), et le lien vers le compte gérant passe par une colonne `idUtilisateur` déclarée `UNIQUE` et facultative (`ON DELETE SET NULL`). Une justification possible (à confirmer selon ta conception) : un garage est d'abord une **entité métier** (un établissement, avec ses techniciens, ses véhicules, ses documents) qui doit survivre si le compte du gérant est supprimé. Le `UNIQUE` garantit qu'un compte gère au plus un garage. Autre limite à connaître : rien dans le schéma n'empêche techniquement un même utilisateur d'avoir à la fois une ligne `client` et une ligne `technicien` ; c'est le code PHP (inscription en transaction) qui garantit qu'un seul rôle est créé.

### Q6. Comment distingues-tu un client particulier d'une entreprise ?

La table `client` a une colonne `typeClient` (`VARCHAR(30)`) qui vaut `'PARTICULIER'` ou `'ENTREPRISE'`, et chaque client possède une ligne dans **une seule** des deux tables filles : `particulier` (adresse) ou `entreprise` (raison sociale obligatoire, adresse). Les deux tables filles ont pour clé primaire `idClient`, qui est aussi une clé étrangère vers `client`.

```
SELECT c.typeClient,
       (p.idClient IS NOT NULL) AS est_particulier,
       (e.idClient IS NOT NULL) AS est_entreprise,
       COUNT(*) AS nb
FROM client c
LEFT JOIN particulier p ON p.idClient = c.idClient
LEFT JOIN entreprise e ON e.idClient = c.idClient
GROUP BY c.typeClient, est_particulier, est_entreprise;
```

L'expression `(p.idClient IS NOT NULL)` vaut `1` (vrai) ou `0` (faux) : MySQL représente les booléens par des entiers.

| typeClient  | est_particulier | est_entreprise | nb |
|-------------|-----------------|----------------|----|
| PARTICULIER | 1               | 0              | 4  |
| ENTREPRISE  | 0               | 1              | 1  |

Le résultat montre que les données sont cohérentes : les 4 particuliers sont uniquement dans `particulier`, l'entreprise uniquement dans `entreprise`.

### Piège fréquent

« La colonne `typeClient` n'est-elle pas redondante ? » Oui, en partie : l'information peut se déduire de la table fille où se trouve le client. Je la garde parce qu'elle rend les requêtes simples (un `WHERE typeClient = 'ENTREPRISE'` au lieu d'une jointure) et qu'elle est écrite dans la même transaction que la ligne fille, donc elle ne peut pas diverger à l'inscription. Une amélioration possible serait d'en faire un `ENUM('PARTICULIER', 'ENTREPRISE')` pour interdire toute autre valeur. À noter aussi : `entreprise` possède une colonne `idEntreprise` en `AUTO_INCREMENT UNIQUE` en plus de sa clé primaire `idClient` ; c'est un numéro interne d'entreprise, pas la clé utilisée pour les liens.

## Q7. Quelles sont les cardinalités principales de ton modèle ? Une intervention peut-elle avoir plusieurs réparations ?

Une **cardinalité** indique combien de lignes d'une table peuvent être reliées à une ligne d'une autre table. On l'écrit avec un minimum et un maximum : `0..1` (zéro ou un), `1` (exactement un), `0..n` (zéro ou plusieurs). Dans le schéma physique, on les lit dans les contraintes :

- une clé étrangère `NOT NULL` signifie « exactement 1 parent obligatoire » ;
- une clé étrangère qui accepte `NULL` signifie « 0 ou 1 parent » ;
- une clé étrangère **sans** `UNIQUE` signifie qu'un parent peut avoir **plusieurs** enfants ;
- une clé étrangère **avec** `UNIQUE` (ou qui est la clé primaire) limite à **un seul** enfant par parent.

| Relation                                           | Cardinalité                                                                                        | Ce qui le prouve dans le schéma                                                                                                   |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| client → vehicule                                  | Un client a 0..n véhicules ; un véhicule a exactement 1 client                                     | <code>vehicule.idClient INT NOT NULL</code> , sans <code>UNIQUE</code>                                                            |
| vehicule → intervention                            | Un véhicule a 0..n interventions ; une intervention concerne exactement 1 véhicule                 | <code>intervention.idVehicule NOT NULL</code>                                                                                     |
| technicien → intervention                          | Une intervention a 0..1 technicien (affecté plus tard)                                             | <code>intervention.idTechnicien DEFAULT NULL</code>                                                                               |
| intervention → réparation                          | Une intervention a <b>0..n</b> réparations ; une réparation appartient à exactement 1 intervention | <code>reparation.idIntervention NOT NULL</code> , <b>pas de</b> <code>UNIQUE</code> (seulement un index simple <code>KEY</code> ) |
| intervention → anomalie                            | Une anomalie est liée à 0..1 intervention ; elle concerne toujours 1 véhicule                      | <code>anomalie.idIntervention DEFAULT NULL</code> , <code>anomalie.idVehicule NOT NULL</code>                                     |
| intervention → paiement                            | Une intervention a 0..n paiements (par ex. un paiement échoué puis un payé)                        | <code>paiement.idIntervention NOT NULL</code> , sans <code>UNIQUE</code>                                                          |
| utilisateur → client / technicien / administrateur | <code>1 ↔ 0..1</code>                                                                              | clé primaire partagée (la FK est aussi la PK, donc unique)                                                                        |
| utilisateur → garage                               | Un compte gère 0..1 garage                                                                         | <code>garage.idUtilisateur UNIQUE</code> et <code>NULL</code> autorisé                                                            |
| garage → technicien                                | Un garage emploie 0..n techniciens ; un technicien appartient à 0..1 garage                        | <code>technicien.idGarage DEFAULT NULL</code>                                                                                     |

Pour répondre précisément à la question : **oui, le schéma autorise plusieurs réparations par intervention**, puisqu'il n'y a pas de contrainte `UNIQUE` sur `reparation.idIntervention`. Dans les données actuelles, chaque intervention réparée n'a qu'une réparation. On le vérifie ainsi :

```
SELECT COUNT(*) AS nb_reparations,
       COUNT(DISTINCT idIntervention) AS nb_interventions_distinctes
FROM reparation;
```

COUNT(DISTINCT colonne) compte les valeurs **différentes** : si les deux nombres sont égaux, aucune intervention n'a deux réparations.

| nb_reparations | nb_interventions_distinctes |
|----------------|-----------------------------|
| 2              | 2                           |

Et pour la relation client → véhicule, on compte les véhicules de chaque client :

```
SELECT c.idClient, COUNT(v.idVehicule) AS nb_vehicules
FROM client c
LEFT JOIN vehicule v ON v.idClient = c.idClient
GROUP BY c.idClient
ORDER BY nb_vehicules DESC;
```

Ici on écrit COUNT(v.idVehicule) et non COUNT(\*) : COUNT(colonne) ignore les NULL, donc un client sans véhicule obtiendrait bien 0 au lieu de 1.

| idClient | nb_vehicules |
|----------|--------------|
| 999012   | 5            |
| 6        | 1            |
| 11       | 1            |
| 14       | 1            |
| 999021   | 1            |

Le client 999012 (l'entreprise) possède 5 véhicules : c'est l'illustration du « 0..n ».

### Q8. Comment passe-t-on du MCD au MLD ? Montre-le sur trois relations de ta base.

Le **MCD** (Modèle Conceptuel de Données, méthode Merise) décrit les **entités** (CLIENT, VEHICULE...) et les **associations** entre elles (« possède », « concerne »...) avec leurs cardinalités, sans parler de tables ni de clés étrangères. Le **MLD** (Modèle Logique de Données) traduit ce schéma en **tables** avec clés primaires et étrangères. Le **MPD** (physique) est ensuite le code SQL CREATE TABLE pour MySQL.

Les règles de passage principales sont :

1. Chaque entité devient une table ; son identifiant devient la clé primaire.
2. Une association **un-à-plusieurs**, de cardinalités (0,1 ou 1,1) d'un côté et (0,n ou 1,n) de l'autre, disparaît : l'identifiant de l'entité qui porte la cardinalité (0,n) ou (1,n) devient une clé étrangère dans la table de l'entité qui porte la cardinalité (1,1) ou (0,1). Exemple : CLIENT (0,n) — possède — (1,1) VEHICULE donne `vehicule.idClient`.
3. Une association **n – n** (cardinalité maximale n des deux côtés) (plusieurs-à-plusieurs) devient une table à part, dont la clé primaire combine les deux clés étrangères.
4. Une spécialisation (héritage) devient des tables filles dont la clé primaire est aussi clé étrangère vers la table mère.

#### Exemple 1 : CLIENT — possède — VEHICULE

MCD : CLIENT (0,n) — possède — (1,1) VEHICULE. Un client possède zéro ou plusieurs véhicules ; un véhicule est possédé par un et un seul client.

MLD (règle 2) : l'association disparaît, la clé du client part dans le véhicule.

```
client (idClient, typeClient)
vehicule (idVehicule, #idClient, #idGarage, marque, modele, immatriculation, ...)
(# = clé étrangère ; 1re colonne = clé primaire, soulignée sur un MLD dessiné)
```

Le (1,1) devient `idClient INT NOT NULL` : un véhicule ne peut pas exister sans client.

### Exemple 2 : INTERVENTION — donne lieu à — REPARATION

MCD : INTERVENTION (0,n) — donne lieu à — (1,1) REPARATION.

```
reparation (idReparation, #idIntervention, #idTechnicien, description, cout, statut, ...)
```

`idIntervention NOT NULL` traduit le (1,1). Le `ON DELETE CASCADE` de `fk_reparation_intervention` exprime en plus que la réparation n'a pas de sens sans son intervention : si on supprime l'intervention, ses réparations disparaissent.

### Exemple 3 : UTILISATEUR — spécialisation — TECHNICIEN

MCD : TECHNICIEN est un sous-type de UTILISATEUR (il hérite de nom, email...).

```
utilisateur (idUtilisateur, nom, prenom, email, motDePasse, ...)
technicien (#idTechnicien, typeTechnicien, #idGarage, specialite, statutValidation, ...)
idTechnicien = clé primaire ET clé étrangère vers utilisateur
```

Règle 4 : la clé est partagée. Le technicien emploie aussi la règle 2 pour son garage (`#idGarage`, facultatif).

#### Piège fréquent

« As-tu une association plusieurs-à-plusieurs ? » Dans le schéma actuel, il n'y a pas de table d'association pure avec clé primaire composée. Les relations « plusieurs-à-plusieurs » du métier passent par une table qui a sa propre existence : par exemple, technicien et véhicule sont reliés en n-n **à travers** `intervention`, qui a son propre identifiant et ses propres attributs (date, statut, priorité). C'est une association porteuse de données transformée en entité.

## Q9. Ta base est-elle normalisée ? Explique les formes normales 1FN, 2FN, 3FN avec tes tables.

La **normalisation** est un ensemble de règles qui évitent de stocker deux fois la même information. La redondance pose problème : si l'information est écrite à deux endroits, on risque de la modifier à un seul et d'avoir des données contradictoires (on parle d'« anomalies de mise à jour »).

### 1FN (première forme normale)

Une table est en 1FN si chaque case contient **une seule valeur** (pas de liste) et si chaque ligne a une clé primaire. Dans ma base, un client ayant plusieurs véhicules n'a pas une colonne « `vehicules = 'Peugeot 308, Toyota Corolla'` » : chaque véhicule est une ligne de `vehicule`. De même, les documents d'un technicien sont des lignes de `technician_documents` et non une liste dans `technicien`.

Point honnête : les colonnes `competences` (table `technicien`) et `piecesUtilisees` (table `reparation`) sont des champs `TEXT` libres. Elles sont traitées comme du texte descriptif, jamais filtrées ni comptées élément par élément ; si on voulait gérer un stock de pièces, il faudrait une table `piece` et une table de liaison.

## 2FN (deuxième forme normale)

Une table est en 2FN si elle est en 1FN et si toutes les colonnes non-clés dépendent de la **totalité** de la clé primaire. Cette règle ne pose problème qu'avec des clés composées de plusieurs colonnes. Toutes mes tables ont une clé primaire à **une seule colonne** (`idVehicule`, `idIntervention`...), donc elles sont automatiquement en 2FN.

## 3FN (troisième forme normale)

Une table est en 3FN si elle est en 2FN et si aucune colonne non-clé ne dépend d'une **autre colonne non-clé** (pas de « dépendance transitive »). Exemple : je ne stocke pas le nom du client dans `vehicule`. Si je le faisais, le nom dépendrait de `idClient`, et non de `idVehicule` ; il faudrait le corriger dans chaque véhicule à chaque changement. À la place, on fait une jointure :

```
SELECT v.marque, v.modele, i.type, i.statut
FROM intervention i
JOIN vehicule v ON v.idVehicule = i.idVehicule
ORDER BY i.dateIntervention DESC
LIMIT 3;
```

| marque         | modele  | type               | statut    |
|----------------|---------|--------------------|-----------|
| Citroen        | Jumper  | Entretien courant  | PLANIFIEE |
| Peugeot        | 308     | Contrôle général   | PLANIFIEE |
| Toyota Corolla | berline | Diagnostic général | PLANIFIEE |

La marque et le modèle ne sont stockés qu'une fois, dans `vehicule`, et sont retrouvés au besoin.

### Piège fréquent

Un jury attentif verra que `intervention` contient à la fois `idVehicule` et `idClient`, alors que le client se déduit du véhicule (`vehicule.idClient`). C'est une **dénormalisation volontaire** (une entorse à la 3FN) : elle simplifie et accélère les requêtes très fréquentes « interventions de ce client » et conserve le demandeur historique même si le véhicule changeait de propriétaire. Le risque est l'incohérence ; on peut la contrôler par une requête :

```
SELECT COUNT(*) AS incoherences
FROM intervention i
JOIN vehicule v ON v.idVehicule = i.idVehicule
WHERE i.idClient <> v.idClient;
```

```
incoherences
-----
0
```

(`<>` signifie « différent de ».) La même remarque vaut pour `paiement.idClient`, que l'on pourrait déduire de `paiement.idIntervention`.

## Q10. Pourquoi utiliser le type ENUM pour les statuts ? Quels sont les statuts possibles ?

Un **ENUM** est un type de colonne qui n'accepte qu'une **liste fermée de valeurs** déclarée à la création de la table. Exemple dans `intervention` :

```
statut ENUM('PLANIFIEE','EN_COURS','TERMINEE','ANNULEE')
NOT NULL DEFAULT 'PLANIFIEE'
```

- `ENUM(...)` : seules ces quatre valeurs sont autorisées.
- `NOT NULL` : une intervention a toujours un statut.
- `DEFAULT 'PLANIFIEE'` : si on n'indique pas de statut à l'insertion, MySQL met `PLANIFIEE`.

On peut lister tous les statuts de la base depuis `information_schema.COLUMNS` :

```
SELECT TABLE_NAME, COLUMN_NAME, COLUMN_TYPE
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE()
  AND DATA_TYPE = 'enum'
  AND COLUMN_NAME IN ('statut','statutGarage','statutValidation','statutVerification')
ORDER BY TABLE_NAME;
```

`IN (...)` signifie « égal à l'une de ces valeurs ».

| TABLE_NAME     | COLUMN_NAME        | COLUMN_TYPE                                       |
|----------------|--------------------|---------------------------------------------------|
| anomalie       | statut             | enum('NOUVELLE','EN_COURS','TRAITEE','IGNOREE')   |
| documentgarage | statutVerification | enum('EN_ATTENTE','VALIDE','REJETE')              |
| garage         | statutGarage       | enum('EN_ATTENTE','VALIDE','REJETE','SUSPENDU')   |
| intervention   | statut             | enum('PLANIFIEE','EN_COURS','TERMINEE','ANNULEE') |
| paiement       | statut             | enum('EN_ATTENTE','PAYE','ECHOUE','ANNULE')       |
| reparation     | statut             | enum('EN_ATTENTE','EN_COURS','TERMINEE')          |
| technicien     | statutValidation   | enum('EN_ATTENTE','VALIDE','REJETE','SUSPENDU')   |

D'autres ENUM existent pour des niveaux : `anomalie.niveau` (`FAIBLE`, `MOYEN`, `CRITIQUE`), `intervention.priorite` (`BASSE`, `MOYENNE`, `HAUTE`), `technicien.typeTechnicien` (`INTERNE`, `GARAGE`), ainsi que `lu` (oui/non) dans `messages` et `notifications`.

#### Avantages :

- **Intégrité** : impossible d'enregistrer une faute de frappe comme `'TERMINE'` ou `'en cours'`. Ma base tourne en mode strict (`sql_mode` contient `STRICT_TRANS_TABLES`), donc une valeur hors liste provoque une **erreur** au lieu d'être remplacée silencieusement par une chaîne vide.
- **Lisibilité** : la liste des états possibles est documentée dans le schéma lui-même.
- **Compacité** : MySQL stocke en interne un petit numéro (1 ou 2 octets) et non le texte.

Conséquence amusante de ce numéro interne : un tri sur un ENUM suit l'**ordre de déclaration**, pas l'ordre alphabétique. C'est pratique, car cela suit le cycle de vie :

```
SELECT statut, COUNT(*) AS nb
FROM intervention
GROUP BY statut
ORDER BY statut;
```

| statut    | nb |
|-----------|----|
| PLANIFIEE | 5  |
| EN_COURS  | 1  |
| TERMINEE  | 4  |

Par ordre alphabétique, `EN_COURS` serait venu avant `PLANIFIEE`.

#### Limites :

- Ajouter un statut (par exemple `EN_ATTENTE_PIECES`) demande de modifier la structure de la table avec `ALTER TABLE ... MODIFY statut ENUM(...)`, ce qui peut verrouiller ou reconstruire la table.
- Le type `ENUM` est propre à MySQL ; il n'est pas portable tel quel vers d'autres SGBD.
- On ne peut pas stocker d'informations supplémentaires sur un statut (libellé affiché, couleur, ordre personnalisé). L'alternative est une **table de référence** (par ex. `statut_intervention(code, libelle)`) reliée par clé étrangère, ou une colonne `VARCHAR` avec une contrainte `CHECK` (supportée depuis MySQL 8.0.16).
- L'`ENUM` contrôle la **valeur** mais pas les **transitions** : rien n'empêche de passer de `TERMINEE` à `PLANIFIEE`. Ce contrôle est fait dans le code PHP.

### Piège fréquent

Les valeurs d'un `ENUM` sont sensibles à la façon dont on les écrit dans le code : le code doit utiliser exactement `'TERMINEE'`. MySQL compare la valeur reçue à la liste avec la collation de la colonne (`utf8mb4_unicode_ci`, insensible à la casse et aux accents) : `'terminee'` devrait donc être reconnu et enregistré sous la forme `'TERMINEE'`, mais `'TERMINÉ'` (il manque un E) ou `'FINIE'` serait refusé en mode strict. Ce comportement n'a pas été testé par un vrai `INSERT` ; le plus sûr est de toujours écrire la valeur exacte.

## Q11. Que signifient `utf8mb4` et `utf8mb4_unicode_ci` ? Pourquoi ce choix ?

Deux réglages déterminent la façon dont MySQL gère le texte :

- Le **jeu de caractères** (`CHARSET`) indique comment chaque caractère est codé en octets. `utf8mb4` est le vrai UTF-8 complet : il code chaque caractère sur 1 à 4 octets et couvre tout Unicode (lettres accentuées, caractères de toutes les langues, émojis).
- La **collation** (`COLLATE`) indique comment comparer et trier les textes. `utf8mb4_unicode_ci` applique les règles de tri Unicode ; `ci` veut dire **case insensitive** : majuscules et minuscules sont considérées comme égales, et dans cette collation les accents sont aussi ignorés lors des comparaisons.

Pourquoi ce choix : l'application est en français, utilisée au Cameroun, et manipule des noms, adresses, descriptions de pannes avec des accents (Contrôle général). L'ancien jeu `utf8` de MySQL (appelé aussi `utf8mb3`) se limite à 3 octets et refuse les caractères de 4 octets comme les émojis ; `utf8mb4` évite ce problème. La connexion PHP utilise aussi `charset=utf8mb4`, pour que les caractères ne soient pas abîmés entre PHP et MySQL.

```
SELECT @@character_set_database AS charset_base,
       @@collation_database      AS collation_base;
```

Les noms commençant par `@@` sont des **variables système** du serveur.

```
charset_base | collation_base
-----+-----
utf8mb4      | utf8mb4_unicode_ci
```

Démonstration du nombre d'octets et de la comparaison insensible :

```
SELECT CHAR_LENGTH('é') AS nb_caracteres,
       LENGTH('é')      AS nb_octets,
       LENGTH(CONVERT(0xF09F9A97 USING utf8mb4)) AS octets_emoji,
       'élève' = 'ELEVE' COLLATE utf8mb4_unicode_ci AS egalite_unicode_ci;
```

| nb_caracteres | nb_octets | octets_emoji | egalite_unicode_ci |
|---------------|-----------|--------------|--------------------|
| 1             | 2         | 4            | 1                  |

`CHAR_LENGTH` compte les caractères, `LENGTH` compte les octets. `0xF09F9A97` est le code hexadécimal de l'emoji « voiture » ; `CONVERT(... USING utf8mb4)` le transforme en texte `utf8mb4`. Cet emoji occupe 4 octets, ce que seul `utf8mb4` sait stocker. Le mot-clé `COLLATE utf8mb4_unicode_ci` impose la collation de mes tables pour la comparaison (sans lui, MySQL utiliserait celle de la connexion PHP, `utf8mb4_general_ci`). Le résultat `1` de la dernière colonne montre que `'élève'` et `'ELEVE'` sont considérés égaux. Avec la collation binaire `utf8mb4_bin`, la même comparaison donne `0`.

### Piège fréquent

Conséquence pratique : la contrainte `UNIQUE` sur `utilisateur.email` traite `Jean@exemple.cm` et `jean@exemple.cm` comme **le même email**. C'est un avantage (on évite les doublons de comptes), mais il faut le savoir : une recherche `WHERE nom = 'Eloundou'` trouvera aussi `'ÉLOUNDOU'`.

## Q12. Pourquoi DECIMAL pour les montants et pas FLOAT ?

Les montants sont stockés en `DECIMAL(10,2)` : `paiement.montant` et `reparation.cout`. La durée `reparation.dureeIntervention` est en `DECIMAL(5,2)`.

- `DECIMAL(10,2)` signifie : **10 chiffres au total**, dont **2 après la virgule**. Le maximum est donc 99 999 999,99, largement suffisant pour des montants en francs CFA.
- `DECIMAL` est un nombre **exact** : MySQL le stocke chiffre par chiffre. `FLOAT` et `DOUBLE` sont des nombres **approchés** en binaire : certaines valeurs décimales simples, comme 0,1, ne peuvent pas être représentées exactement.

Démonstration exécutée sur le serveur :

```
SELECT 0.1 + 0.2 = 0.3 AS avec_decimal,
       CAST(0.1 AS DOUBLE) + CAST(0.2 AS DOUBLE) = CAST(0.3 AS DOUBLE) AS avec_double;
```

En MySQL, un nombre écrit avec un point (`0.1`) est traité comme un `DECIMAL` exact. `CAST(x AS DOUBLE)` le convertit en nombre à virgule flottante.

| avec_decimal | avec_double |
|--------------|-------------|
| 1            | 0           |

Avec `DECIMAL`, `0,1 + 0,2` vaut exactement `0,3` (résultat `1` = vrai). Avec `DOUBLE`, ce n'est pas le cas (`0` = faux) : le résultat est `0,30000000000000004`. Sur une facture, une somme de paiements ou un rapprochement comptable, ces petites erreurs s'accumulent et faussent les totaux. C'est pourquoi on utilise toujours `DECIMAL` pour l'argent.

### Piège fréquent

« Le franc CFA n'a pas de centimes, pourquoi 2 décimales ? » C'est exact, le XAF s'utilise en pratique sans décimales. Les deux décimales laissent de la marge (calcul de taxes, pourcentages, autre devise) sans coût notable. Un `INT` ou `DECIMAL(12,0)` serait aussi défendable ; l'important est de ne pas utiliser `FLOAT`.

### Q13. Pourquoi VARCHAR, TEXT, DATETIME, DATE ? Comment as-tu choisi les types des colonnes ?

Chaque type est choisi selon la nature de la donnée :

| Type                      | Signification                                                 | Exemples dans ma base                                                                                                      | Pourquoi                                                                                                                                                |
|---------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>INT</code>          | Entier (environ $\pm 2,1$ milliards)                          | toutes les clés <code>id...</code> ,<br><code>vehicule.kilometrage</code> ,<br><code>vehicule.annee</code>                 | Comparaisons et jointures rapides ; <code>AUTO_INCREMENT</code> génère les numéros.                                                                     |
| <code>VARCHAR(n)</code>   | Texte de longueur variable, au plus <code>n</code> caractères | <code>email VARCHAR(150)</code> , <code>immatriculation VARCHAR(50)</code> , <code>telephone VARCHAR(20)</code>            | Ne prend que la place utilisée ; la limite <code>n</code> protège contre des saisies aberrantes. Indexable (donc utilisable avec <code>UNIQUE</code> ). |
| <code>VARCHAR(255)</code> | Idem                                                          | <code>motDePasse</code>                                                                                                    | Le mot de passe est stocké <b>haché</b> (jamais en clair) ; 255 laisse de la place si l'algorithme de hachage change.                                   |
| <code>TEXT</code>         | Texte long (jusqu'à 64 Ko)                                    | <code>description</code> , <code>diagnostic</code> ,<br><code>travauxEffectues</code> , <code>messages.contenu</code>      | Texte libre de longueur imprévisible.                                                                                                                   |
| <code>DATETIME</code>     | Date + heure, ex. <code>2026-10-02 14:30:00</code>            | <code>intervention.dateIntervention</code> ,<br><code>anomalie.dateDetection</code> ,<br><code>reparation.dateDebut</code> | Un événement a un moment précis ; <code>DEFAULT CURRENT_TIMESTAMP</code> remplit la date automatiquement.                                               |
| <code>DATE</code>         | Date seule                                                    | <code>vehicule.dateAcquisition</code>                                                                                      | L'heure d'achat n'a pas d'intérêt.                                                                                                                      |
| <code>TIMESTAMP</code>    | Date + heure, convertie selon le fuseau                       | <code>messages.date_envoi</code> ,<br><code>notifications.date_creation</code>                                             | Adapté aux horodatages techniques.                                                                                                                      |
| <code>DECIMAL</code>      | Nombre exact                                                  | <code>montant</code> , <code>cout</code>                                                                                   | Voir Q12.                                                                                                                                               |
| <code>ENUM</code>         | Liste fermée                                                  | tous les statuts                                                                                                           | Voir Q10.                                                                                                                                               |

Le téléphone est en `VARCHAR` et non en `INT` : on ne fait pas de calcul dessus, et un numéro peut commencer par `+237` ou par un zéro, que le type entier perdrait. Même raisonnement pour l'immatriculation et le numéro de châssis.

**DATETIME ou TIMESTAMP ?** `DATETIME` enregistre la date telle quelle, de l'an 1000 à l'an 9999.

`TIMESTAMP` est converti en temps universel (UTC) à l'écriture puis reconverti selon le fuseau de la session à la lecture, et il est limité à l'intervalle 1970–2038. Pour des dates métier (rendez-vous, détection d'une panne), `DATETIME` est plus sûr.

## Piège fréquent

Le jury peut relever un manque d'homogénéité : les tables `messages`, `notifications` et `technician_documents` utilisent des noms en anglais et en `snake_case` (`expediteur_id`, `date_creation`) et le type `TIMESTAMP`, alors que le reste de la base est en `camelCase` français avec `DATETIME`. Ces tables ont probablement été ajoutées à un autre moment du projet (à adapter selon ton historique réel). C'est sans conséquence sur le fonctionnement, mais une harmonisation des conventions serait une amélioration à proposer.

## Q14. À quoi servent les contraintes UNIQUE, NOT NULL et DEFAULT dans ta base ?

Ce sont des **règles d'intégrité** que MySQL vérifie à chaque insertion ou modification. Elles protègent les données même si le code PHP contenait une erreur.

- **NOT NULL** : la colonne doit toujours avoir une valeur. Ex. `vehicule.marque`, `utilisateur.email`, `paiement.montant`.
- **UNIQUE** : deux lignes ne peuvent pas avoir la même valeur. Ma base en a sur `utilisateur.email` (un compte par email), `vehicule.immatriculation` et `vehicule.numeroChassis` (un véhicule ne peut pas être enregistré deux fois), et `garage.idUtilisateur` (un gérant pour au plus un garage). Particularité : **UNIQUE** accepte plusieurs `NULL`, donc plusieurs véhicules peuvent ne pas encore avoir de numéro de châssis renseigné.
- **DEFAULT** : valeur mise automatiquement si on n'en donne pas. Ex. `vehicule.kilometrage` `DEFAULT 0`, `vehicule.etat` `DEFAULT 'BON'`, `utilisateur.themePreference` `DEFAULT 'light'`, `dateCreation` `DEFAULT CURRENT_TIMESTAMP`.

Une contrainte **UNIQUE** crée automatiquement un **index**, une structure qui permet à MySQL de retrouver une ligne sans lire toute la table. De même, chaque clé étrangère possède un index ( `KEY fk_vehicule_client (idClient)` ). La commande `EXPLAIN` montre que MySQL s'en sert :

```
EXPLAIN SELECT * FROM vehicule WHERE idClient = 6;
```

| table    | type | possible_keys      | key                | rows |
|----------|------|--------------------|--------------------|------|
| vehicule | ref  | fk_vehicule_client | fk_vehicule_client | 1    |

`key = fk_vehicule_client` indique que l'index de la clé étrangère est utilisé ; `rows = 1` est le nombre de lignes que MySQL estime devoir lire.

## Q15. Que se passe-t-il quand on supprime une ligne référencée ? Explique CASCADE, SET NULL et RESTRICT.

Chaque clé étrangère précise ce que fait MySQL quand la ligne **parente** est supprimée (`ON DELETE`) ou que sa clé change (`ON UPDATE`). Ma base utilise les trois comportements, chacun pour une raison :

| Règle    | Effet                                                                | Exemple dans ma base                                                                                                               | Pourquoi                                                                                                                          |
|----------|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| CASCADE  | Les lignes enfants sont supprimées aussi                             | fk_client_utilisateur ,<br>fk_reparation_intervention ,<br>fk_anomalie_vehicule ,<br>fk_notifications_user                         | L'enfant n'a aucun sens sans son parent (une fiche client sans compte, une notification sans destinataire).                       |
| SET NULL | La clé étrangère de l'enfant devient NULL                            | fk_intervention_technicien ,<br>fk_vehicule_garage ,<br>fk_journal_utilisateur                                                     | L'enfant doit survivre : si un technicien est supprimé, l'intervention reste dans l'historique, simplement sans technicien.       |
| RESTRICT | La suppression du parent est <b>refusée</b> tant qu'il a des enfants | fk_vehicule_client ,<br>fk_intervention_vehicule ,<br>fk_intervention_client ,<br>fk_paiement_client ,<br>fk_paiement_intervention | Protéger les données importantes : on ne peut pas effacer un client qui a des véhicules, ni une intervention qui a des paiements. |

Ces choix sont cohérents entre eux : supprimer un `utilisateur` client déclenche la cascade vers `client`, mais cette cascade est **bloquée** si le client possède encore des véhicules ( `RESTRICT` sur `fk_vehicule_client` ). On ne perd donc jamais l'historique d'un véhicule par accident. Toutes les règles de la table `utilisateur` et de ses rôles peuvent être affichées ainsi :

```
SELECT TABLE_NAME, CONSTRAINT_NAME, REFERENCED_TABLE_NAME, DELETE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = DATABASE()
  AND TABLE_NAME IN ('client','particulier','entreprise','technicien','administrateur')
ORDER BY TABLE_NAME;
```

| TABLE_NAME     | CONSTRAINT_NAME           | REFERENCED_TABLE_NAME | DELETE_RULE |
|----------------|---------------------------|-----------------------|-------------|
| administrateur | fk_admin_utilisateur      | utilisateur           | CASCADE     |
| client         | fk_client_utilisateur     | utilisateur           | CASCADE     |
| entreprise     | fk_entreprise_client      | client                | CASCADE     |
| particulier    | fk_particulier_client     | client                | CASCADE     |
| technicien     | fk_technicien_garage      | garage                | SET NULL    |
| technicien     | fk_technicien_utilisateur | utilisateur           | CASCADE     |

### Piège fréquent

Dans `messages`, `notifications` et `technician_documents`, les clés étrangères n'indiquent pas de clause `ON UPDATE`. Dans ce cas MySQL applique la valeur par défaut, `NO ACTION`, qui en InnoDB se comporte comme `RESTRICT` : modifier le numéro d'un utilisateur qui a des messages serait refusé. Ce n'est pas gênant en pratique, car on ne modifie jamais une clé primaire générée par `AUTO_INCREMENT`.

## À retenir

- Ma base charles : MySQL 8.0.41, **19 tables InnoDB**, **34 clés étrangères**, jeu de caractères `utf8mb4`, collation `utf8mb4_unicode_ci`.
- InnoDB est choisi pour les **clés étrangères vérifiées**, les **transactions** et le **verrouillage par ligne**.
- `utilisateur` est la table mère ; `administrateur`, `client`, `technicien` en héritent par **clé primaire partagée** (PK = FK), puis `particulier` et `entreprise` héritent de `client`. `garage` est relié au compte par `idUtilisateur` `UNIQUE`.
- Cardinalités : se lisent dans le schéma ( `NOT NULL` = obligatoire, `NULL` = facultatif, `UNIQUE` = au plus un). Une intervention peut avoir **plusieurs** réparations (pas de `UNIQUE`).
- MCD → MLD : une association 1–n devient une clé étrangère côté « n » ; une association n–n devient une table.
- Normalisation : 1FN (une valeur par case), 2FN (automatique avec des clés à une colonne), 3FN (pas de dépendance entre colonnes non-clés). Exception assumée : `intervention.idClient` et `paiement.idClient`.
- `ENUM` : liste fermée, contrôlée en mode strict, triée selon l'ordre de déclaration ; limite : ajouter une valeur impose un `ALTER TABLE`.
- Argent en `DECIMAL(10,2)` (exact), jamais en `FLOAT` (approché) ; téléphone et immatriculation en `VARCHAR` ; dates métier en `DATETIME`.
- `CASCADE` = supprimer les enfants, `SET NULL` = détacher les enfants, `RESTRICT` = interdire la suppression.

## 2. Clés, contraintes et intégrité

Une **contrainte** est une règle que MySQL fait respecter lui-même, quel que soit le programme qui écrit dans la base. Si une requête viole la règle, MySQL la refuse et renvoie une erreur. C'est la différence avec une vérification faite en PHP : le PHP peut être contourné (autre script, phpMyAdmin, bug), la contrainte non. Ce chapitre passe en revue les contraintes réellement présentes dans la base `charles` : 19 clés primaires, 34 clés étrangères, 5 contraintes d'unicité, et de nombreuses colonnes `NOT NULL` et `DEFAULT`.

Tous les résultats montrés ci-dessous viennent d'une exécution réelle sur la base, en lecture seule. Les commandes qui modifient la base (`DELETE`, `UPDATE`, `INSERT`, `ALTER`) sont seulement expliquées : leur syntaxe a été vérifiée avec `EXPLAIN`, qui analyse la requête sans l'exécuter.

### Les clés

#### Q16. Qu'est-ce qu'une clé primaire, et quelle est celle de vos tables ?

La **clé primaire** (`PRIMARY KEY`) est la colonne (ou le groupe de colonnes) qui identifie chaque ligne de façon unique. Elle a deux propriétés imposées par MySQL : elle est **unique** (deux lignes ne peuvent pas avoir la même valeur) et elle est **obligatoire** (elle ne peut jamais être `NULL`). Une table n'a qu'une seule clé primaire.

Dans ma base, chacune des 19 tables a une clé primaire sur une seule colonne, le plus souvent nommée sur le modèle `id` + nom de la table : `utilisateur.idUtilisateur`, `vehicule.idVehicule`, `intervention.idIntervention`, `reparation.idReparation`, `paiement.idPaiement`, etc. Il y a des exceptions : `documentgarage.idDocument`, `journalactivites.idActivite`, `assistentia.idAnalyse` et `analyseia.idAnalyse`, `particulier.idClient` et `entreprise.idClient` (clé recopiée de la table mère, voir plus bas). Les trois tables ajoutées plus tard (`messages`, `notifications`, `technician_documents`) utilisent simplement `id`.

Voici comment elle est déclarée dans la table `vehicule` :

```
CREATE TABLE vehicule (  
  idVehicule INT NOT NULL AUTO_INCREMENT,  
  ...  
  PRIMARY KEY (idVehicule)  
);
```

- `INT` : la colonne contient un nombre entier.
- `NOT NULL` : la valeur est obligatoire.
- `AUTO_INCREMENT` : MySQL choisit lui-même le numéro (voir Q17).
- `PRIMARY KEY (idVehicule)` : désigne cette colonne comme identifiant de la ligne. MySQL crée automatiquement un index nommé `PRIMARY` dessus.

**Cas particulier : la clé primaire partagée (héritage).** Un utilisateur peut être client, technicien ou administrateur. Je n'ai pas mis de colonne « rôle » : le rôle est matérialisé par une ligne dans une table fille dont la clé primaire est en même temps une clé étrangère vers `utilisateur`. Par exemple :

```
CREATE TABLE client (
  idClient INT NOT NULL,
  typeClient VARCHAR(30) NOT NULL,
  PRIMARY KEY (idClient),
  CONSTRAINT fk_client_utilisateur FOREIGN KEY (idClient)
    REFERENCES utilisateur (idUtilisateur)
    ON DELETE CASCADE ON UPDATE CASCADE
);
```

Le client n°6 est donc exactement l'utilisateur n°6. Le même schéma est utilisé pour `technicien`, `administrateur`, et un niveau plus bas pour `particulier` et `entreprise` (clé `idClient` vers `client`), ainsi que pour `analyseia` (clé `idAnalyse` vers `assistantia`). On peut vérifier la répartition :

```
SELECT (SELECT COUNT(*) FROM utilisateur) AS utilisateurs,
       (SELECT COUNT(*) FROM client) AS clients,
       (SELECT COUNT(*) FROM technicien) AS techniciens,
       (SELECT COUNT(*) FROM administrateur) AS administrateurs,
       (SELECT COUNT(*) FROM garage WHERE idUtilisateur IS NOT NULL) AS comptes_garage;
```

| utilisateurs | clients | techniciens | administrateurs | comptes_garage |
|--------------|---------|-------------|-----------------|----------------|
| 15           | 5       | 8           | 1               | 1              |

$5 + 8 + 1 + 1 = 15$ . L'addition seule ne suffit pas à le prouver (un même utilisateur pourrait être compté deux fois), mais une vérification complémentaire confirme qu'aucun identifiant n'apparaît dans deux rôles et qu'aucun utilisateur n'est sans rôle : aujourd'hui, chaque utilisateur a exactement un rôle (voir aussi Q30, point 7).

### Piège fréquent

« Clé primaire » ne veut pas dire « colonne qui s'appelle id ». C'est la contrainte `PRIMARY KEY` qui compte. Et une clé primaire peut aussi être une clé étrangère : c'est le cas de `client.idClient`, `technicien.idTechnicien` et `administrateur.idAdministrateur`.

## Q17. À quoi sert `AUTO_INCREMENT` ? Pourquoi vos numéros ne se suivent-ils pas ?

`AUTO_INCREMENT` demande à MySQL de générer automatiquement la valeur de la colonne lors d'un `INSERT` : il prend le compteur de la table, l'attribue à la nouvelle ligne, puis l'augmente de 1. On n'écrit donc pas la colonne dans l'`INSERT`. Exemple réel tiré de `auth/register.php` :

```
INSERT INTO utilisateur (nom, prenom, email, telephone, motDePasse)
VALUES (?, ?, ?, ?, ?);
```

La colonne `idUtilisateur` n'est pas citée : MySQL la remplit. En PHP, `$conn->lastInsertId()` récupère ensuite le numéro attribué, pour l'utiliser dans l'insertion suivante (`INSERT INTO client (idClient, typeClient) VALUES (?, ?)`).

On peut lire la valeur actuelle du compteur de chaque table :

```
SELECT TABLE_NAME, AUTO_INCREMENT
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE() AND AUTO_INCREMENT IS NOT NULL
ORDER BY TABLE_NAME;
```

| TABLE_NAME           | AUTO_INCREMENT |
|----------------------|----------------|
| anomalie             | 921            |
| documentgarage       | 3              |
| entreprise           | 6              |
| garage               | 10             |
| intervention         | 933            |
| journalactivites     | 51             |
| messages             | 16             |
| notifications        | 84             |
| paiement             | 1              |
| reparation           | 917            |
| technician_documents | 5              |
| utilisateur          | 999034         |
| vehicule             | 928            |

13 lignes. Une 14e table possède pourtant une colonne AUTO\_INCREMENT : `assistantia` (vide). Elle n'apparaît pas car `information_schema` renvoie `NULL` pour son compteur : ces valeurs sont des statistiques que MySQL met en cache, elles peuvent être absentes ou avoir un léger retard. La source exacte reste `SHOW CREATE TABLE`.

Et le plus grand identifiant réellement utilisé :

```
SELECT MAX(idUtilisateur) AS max_id, COUNT(*) AS nb FROM utilisateur;
```

```
max_id | nb
999021 | 15
```

Il y a 15 utilisateurs, mais le prochain recevra le numéro 999034. C'est normal : **un numéro AUTO\_INCREMENT n'est jamais réutilisé**. Quand une ligne est supprimée, ou quand une transaction qui avait inséré une ligne est annulée (`ROLLBACK`), le numéro est « perdu ». Les très grands nombres (999012, 999021...) s'expliquent très probablement par des lignes insérées avec un identifiant choisi à la main : quand on insère une valeur explicite plus grande que le compteur, MySQL place le compteur juste après elle.

Cinq tables n'ont **pas** d'AUTO\_INCREMENT, et c'est voulu : `client`, `technicien`, `administrateur`, `particulier` et `analyseia`. Dans `entreprise`, la clé primaire n'est pas non plus en AUTO\_INCREMENT (voir le piège ci-dessous). Leur clé primaire recopie celle de la table mère (voir Q16) ; la générer automatiquement casserait la correspondance.

### Piège fréquent

Dans `entreprise`, la clé primaire est `idClient`, mais c'est `idEntreprise` qui est en AUTO\_INCREMENT. MySQL l'accepte parce qu'une colonne AUTO\_INCREMENT doit seulement être **indexée**, pas forcément clé primaire : ici elle porte un index `UNIQUE KEY idEntreprise`. Sans cet index, le `CREATE TABLE` serait refusé.

## Q18. Qu'est-ce qu'une clé étrangère ? Montrez-en une.

Une **clé étrangère** (`FOREIGN KEY`) est une colonne d'une table (la table fille) dont la valeur doit exister dans la clé d'une autre table (la table mère). Elle représente un lien : « ce véhicule appartient à tel client ». MySQL vérifie ce lien à chaque écriture.

```
CREATE TABLE vehicule (
  idVehicule INT NOT NULL AUTO_INCREMENT,
  idClient   INT NOT NULL,
  idGarage   INT DEFAULT NULL,
  ...
  CONSTRAINT fk_vehicule_client FOREIGN KEY (idClient)
    REFERENCES client (idClient)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_vehicule_garage FOREIGN KEY (idGarage)
    REFERENCES garage (idGarage)
    ON DELETE SET NULL ON UPDATE CASCADE
);
```

- `CONSTRAINT fk_vehicule_client` : donne un nom à la contrainte. C'est ce nom qui apparaît dans les messages d'erreur et qu'on utilise pour la supprimer (`ALTER TABLE vehicule DROP FOREIGN KEY fk_vehicule_client`).
- `FOREIGN KEY (idClient)` : la colonne de la table fille qui porte le lien.
- `REFERENCES client (idClient)` : la table mère et sa colonne cible (qui doit être clé primaire ou unique).
- `ON DELETE ... / ON UPDATE ...` : ce qui se passe si la ligne mère est supprimée ou si sa clé change (voir Q21).

Ma base contient 34 clés étrangères. On peut lister celles de trois tables centrales :

```
SELECT TABLE_NAME, CONSTRAINT_NAME, COLUMN_NAME,
       REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = DATABASE()
      AND REFERENCED_TABLE_NAME IS NOT NULL
      AND TABLE_NAME IN ('vehicule', 'intervention', 'paiement')
ORDER BY TABLE_NAME, CONSTRAINT_NAME;
```

| TABLE_NAME   | CONSTRAINT_NAME            | COLUMN_NAME    | REFERENCED_TABLE | REFERENCED_COLUMN |
|--------------|----------------------------|----------------|------------------|-------------------|
| intervention | fk_intervention_client     | idClient       | client           | idClient          |
| intervention | fk_intervention_garage     | idGarage       | garage           | idGarage          |
| intervention | fk_intervention_technicien | idTechnicien   | technicien       | idTechnicien      |
| intervention | fk_intervention_vehicule   | idVehicule     | vehicule         | idVehicule        |
| paiement     | fk_paiement_client         | idClient       | client           | idClient          |
| paiement     | fk_paiement_intervention   | idIntervention | intervention     | idIntervention    |
| vehicule     | fk_vehicule_client         | idClient       | client           | idClient          |
| vehicule     | fk_vehicule_garage         | idGarage       | garage           | idGarage          |

La table la plus référencée est `utilisateur` (8 clés étrangères pointent vers elle), suivie de `intervention`, `garage` et `client` (5 chacune). La table qui en possède le plus est `journalactivites` (6), car une ligne du journal peut concerner un utilisateur, un garage, un technicien, une intervention, une réparation ou une anomalie.

### À retenir

- Clé primaire = « qui suis-je » ; clé étrangère = « à qui je suis rattaché ».
- Une clé étrangère `NOT NULL` (ex. `vehicule.idClient`) rend le lien obligatoire ; une clé étrangère qui accepte `NULL` (ex. `vehicule.idGarage`) rend le lien facultatif.

### Q19. Qu'est-ce que l'intégrité référentielle ? Comment prouver qu'elle est respectée ?

L'**intégrité référentielle** est la garantie qu'aucune clé étrangère ne pointe vers une ligne qui n'existe pas. Autrement dit : pas d'intervention sur un véhicule inexistant, pas de réparation sans intervention. Une ligne qui pointe vers rien s'appelle une ligne **orpheline**.

MySQL la fait respecter de deux façons :

- **À l'insertion ou modification d'une fille** : si on essaie d'écrire `INSERT INTO vehicule (idClient, marque, modele) VALUES (123456, 'Toyota', 'Corolla')` alors que le client 123456 n'existe pas, MySQL refuse avec l'erreur 1452 « Cannot add or update a child row: a foreign key constraint fails ».
- **À la suppression d'une mère** : il applique la règle `ON DELETE` (refus, suppression en cascade ou mise à NULL, voir Q21). En cas de refus, l'erreur est la 1451 « Cannot delete or update a parent row ».

Pour prouver qu'il n'y a pas d'orphelins, on utilise une jointure externe gauche :

```
SELECT COUNT(*) AS nb_interventions_orphelines
FROM intervention i
LEFT JOIN vehicule v ON v.idVehicule = i.idVehicule
WHERE v.idVehicule IS NULL;
```

```
nb_interventions_orphelines
0
```

- `LEFT JOIN` garde toutes les interventions, même celles qui n'ont pas de véhicule correspondant ; dans ce cas les colonnes de `v` valent `NULL`.
- `WHERE v.idVehicule IS NULL` ne garde que ces cas sans correspondance, c'est-à-dire les orphelins.

Le résultat est 0, ce qui est attendu : tant que la clé étrangère existe et que `foreign_key_checks` est actif, un orphelin ne peut pas apparaître.

```
SELECT @@foreign_key_checks AS fk_checks, @@default_storage_engine AS moteur;
```

```
fk_checks | moteur
1         | InnoDB
```

#### Piège fréquent

La commande `SET foreign_key_checks = 0` désactive la vérification pour la session (on l'utilise parfois pour importer un fichier SQL dans le désordre). Pendant ce temps, on peut créer des orphelins, et MySQL ne les vérifie pas quand on remet la valeur à 1. Il faut toujours la remettre à 1 tout de suite après.

### Q20. Pourquoi avoir utilisé le moteur InnoDB ?

Un **moteur de stockage** est la partie de MySQL qui range physiquement les données. Les deux plus connus sont **InnoDB** (par défaut depuis MySQL 5.5) et **MyISAM** (ancien). La différence est décisive pour mon projet :

| Fonction                                   | InnoDB                              | MyISAM                                     |
|--------------------------------------------|-------------------------------------|--------------------------------------------|
| Clés étrangères vérifiées                  | Oui                                 | Non : la syntaxe est acceptée mais ignorée |
| Transactions ( BEGIN , COMMIT , ROLLBACK ) | Oui                                 | Non                                        |
| Verrouillage                               | Par ligne ( SELECT ... FOR UPDATE ) | Par table entière                          |
| Récupération après panne                   | Oui (journal de reprise)            | Limitée                                    |

Mon application utilise ces trois premières fonctions : les 34 clés étrangères, les transactions (inscription dans `auth/register.php`, suppression d'un client dans `admin/clients.php`, clôture de réparation) et le verrouillage de ligne `FOR UPDATE` dans `includes/payments.php` pour qu'un double clic ne crée pas deux paiements. Vérification sur la base :

```
SELECT TABLE_NAME, ENGINE
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE();
```

|                |        |                            |
|----------------|--------|----------------------------|
| TABLE_NAME     | ENGINE |                            |
| administrateur | InnoDB |                            |
| analyseia      | InnoDB |                            |
| ...            | InnoDB |                            |
| vehicule       | InnoDB | (19 tables, toutes InnoDB) |

Le moteur est choisi à la fin du `CREATE TABLE : ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;`. `utf8mb4` est l'encodage qui accepte tous les caractères (accents, émojis), et `utf8mb4_unicode_ci` la règle de comparaison (`ci` = insensible à la casse : « Toyota » = « toyota »).

### Piège fréquent

Une clé étrangère ne fonctionne que si **les deux tables** sont en InnoDB, et si les deux colonnes ont exactement le même type (ici `INT` des deux côtés). Avec MyISAM, le `CREATE TABLE` passerait sans erreur, mais aucune vérification ne serait faite.

## Que se passe-t-il quand on supprime ?

### Q21. Expliquez ON DELETE CASCADE, SET NULL, RESTRICT et ON UPDATE CASCADE.

Ces clauses disent à MySQL quoi faire avec les lignes filles quand la ligne mère est **supprimée** (`ON DELETE`) ou quand sa clé primaire **change de valeur** (`ON UPDATE`).

| Règle    | Effet sur les filles                                                                                                          | Exemple réel dans ma base                                                                      |
|----------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| CASCADE  | Les filles sont supprimées avec la mère.                                                                                      | <code>fk_reparation_intervention</code> : supprimer une intervention supprime ses réparations. |
| SET NULL | Les filles sont conservées ; leur clé étrangère passe à <code>NULL</code> . La colonne doit donc accepter <code>NULL</code> . | <code>fk_vehicule_garage</code> : supprimer un garage laisse le véhicule, sans garage.         |

| Règle     | Effet sur les filles                                                                   | Exemple réel dans ma base                                                   |
|-----------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| RESTRICT  | La suppression de la mère est refusée tant qu'il existe des filles.                    | fk_vehicule_client : impossible de supprimer un client qui a des véhicules. |
| NO ACTION | En MySQL/InnoDB, identique à RESTRICT . C'est la valeur par défaut si on n'écrit rien. | ON UPDATE de messages , notifications , technician_documents .              |

ON UPDATE CASCADE signifie : si on change la valeur de la clé primaire de la mère (par exemple UPDATE client SET idClient = 50 WHERE idClient = 6 ), MySQL recopie la nouvelle valeur dans toutes les filles ( vehicule.idClient , intervention.idClient , paiement.idClient ...). En pratique on ne modifie jamais une clé primaire, mais cette règle évite de casser les liens si cela arrive.

Répartition réelle de mes 34 clés étrangères :

```
SELECT DELETE_RULE, COUNT(*) AS nb
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = DATABASE()
GROUP BY DELETE_RULE;
```

|             |    |
|-------------|----|
| DELETE_RULE | nb |
| CASCADE     | 14 |
| SET NULL    | 15 |
| RESTRICT    | 5  |

- information\_schema est une base système, en lecture seule, qui décrit toutes les autres bases.
- REFERENTIAL\_CONSTRAINTS contient une ligne par clé étrangère, avec ses règles UPDATE\_RULE et DELETE\_RULE .
- GROUP BY DELETE\_RULE regroupe les lignes qui ont la même règle, et COUNT(\*) compte chaque groupe.

Les 5 RESTRICT protègent ce qui ne doit pas disparaître par accident : fk\_vehicule\_client , fk\_intervention\_client , fk\_intervention\_vehicule , fk\_paiement\_client et fk\_paiement\_intervention .

### À retenir

- **CASCADE** pour ce qui n'a pas de sens sans la mère (la fiche client sans son utilisateur , les réparations sans leur intervention).
- **SET NULL** pour un lien facultatif qu'on veut garder dans l'historique (le garage ou le technicien d'une intervention).
- **RESTRICT** pour ce qui a une valeur légale ou métier (véhicules, interventions, paiements).

## Q22. Que se passe-t-il si on supprime un utilisateur ?

La requête est simple :

```
DELETE FROM utilisateur WHERE idUtilisateur = 15;
```

Mais son effet dépend du rôle de l'utilisateur, car MySQL suit les clés étrangères en chaîne :

1. **Effets communs à tous les rôles** : ses messages envoyés et reçus ( fk\_messages\_expéditeur , fk\_messages\_destinataire ) et ses notifications ( fk\_notifications\_user ) sont supprimés (CASCADE).

Dans le journal, `journalactivites.idUtilisateur` passe à `NULL` (SET NULL) : la ligne d'historique reste.

2. **Si c'est un technicien** : sa ligne `technicien` est supprimée (CASCADE via `fk_technicien_utilisateur`), puis ses documents (`technician_documents`, CASCADE). Ses interventions et réparations sont conservées, mais `intervention.idTechnicien` et `reparation.idTechnicien` passent à `NULL` (SET NULL).
3. **Si c'est un compte garage** : le garage n'est pas supprimé ; `garage.idUtilisateur` passe à `NULL` (`fk_garage_utilisateur`, SET NULL). Le garage existe toujours mais n'a plus de compte de connexion.
4. **Si c'est un administrateur** : sa ligne `administrateur` est supprimée (CASCADE).
5. **Si c'est un client** : MySQL veut supprimer la ligne `client` (CASCADE), puis `particulier` ou `entreprise` (CASCADE). Mais si ce client a des véhicules, des interventions ou des paiements, les règles `RESTRICT` bloquent : **toute la suppression est annulée**, y compris celle de l'utilisateur. Voir Q23.

La syntaxe a été vérifiée sans rien supprimer :

```
EXPLAIN DELETE FROM utilisateur WHERE idUtilisateur = 999012;
```

| select_type | table       | type  | key     | rows |
|-------------|-------------|-------|---------|------|
| DELETE      | utilisateur | range | PRIMARY | 1    |

EXPLAIN montre seulement le plan : MySQL trouverait la ligne grâce à l'index `PRIMARY` et en supprimerait 1. Il ne vérifie pas les clés étrangères, car il n'exécute rien.

### Piège fréquent

Une suppression en cascade est **atomique** : soit toute la chaîne réussit, soit rien n'est supprimé. Il n'existe pas d'état où l'utilisateur serait supprimé mais pas sa fiche client.

## Q23. Et si on supprime un client qui a des véhicules ?

La clé `fk_vehicule_client` est en `ON DELETE RESTRICT`. MySQL refuse donc de supprimer la ligne `client` tant qu'un véhicule y est rattaché, avec l'erreur 1451 « Cannot delete or update a parent row: a foreign key constraint fails ». Par ricochet, `DELETE FROM utilisateur` sur ce client échoue aussi, puisque la cascade `fk_client_utilisateur` voudrait supprimer la ligne `client`.

Exemple concret sur ma base :

```
SELECT c.idClient, COUNT(v.idVehicule) AS nb_vehicules
FROM client c
LEFT JOIN vehicule v ON v.idClient = c.idClient
GROUP BY c.idClient;
```

| idClient | nb_vehicules |
|----------|--------------|
| 6        | 1            |
| 11       | 1            |
| 14       | 1            |
| 999012   | 5            |
| 999021   | 1            |

Tous les clients ont au moins un véhicule, donc aucun ne peut être supprimé directement. Le client 999012 a 5 véhicules et 3 interventions ( `SELECT COUNT(*) FROM intervention WHERE idClient = 999012` renvoie 3).

**Pourquoi ce choix ?** Un véhicule a un historique d'entretien (interventions, réparations, paiements). Le supprimer automatiquement avec son propriétaire effacerait des traces qui peuvent avoir une valeur pour le garage ou en cas de litige. RESTRICT oblige à une décision volontaire.

**Comment l'application fait-elle quand même ?** Dans `admin/clients.php`, l'administrateur supprime explicitement les filles dans le bon ordre, dans une **transaction** :

```
START TRANSACTION;
DELETE FROM paiement      WHERE idClient = ?;
DELETE FROM intervention WHERE idClient = ?; -- cascade : réparations, journal
DELETE FROM vehicule      WHERE idClient = ?; -- cascade : anomalies
DELETE FROM utilisateur   WHERE idUtilisateur = ?; -- cascade : client,
particulier/entreprise, messages, notifications
COMMIT;
```

- `START TRANSACTION` (en PHP `$conn->beginTransaction()`) ouvre un bloc « tout ou rien ».
- L'ordre va des filles vers les mères : d'abord `paiement` (qui bloque `intervention`), puis `intervention` (qui bloque `vehicule`), puis `vehicule` (qui bloque `client`).
- `COMMIT` valide l'ensemble. Si une étape échoue, le PHP appelle `ROLLBACK` et rien n'est supprimé.
- Le `?` est un paramètre de requête préparée (PDO) : la valeur est envoyée séparément, ce qui empêche l'injection SQL.

## Q24. Et si on supprime un garage ?

Presque toutes les clés qui pointent vers `garage` sont en `ON DELETE SET NULL` : `fk_vehicule_garage`, `fk_intervention_garage`, `fk_technicien_garage`, `fk_journal_garage`. Une seule est en CASCADE : `fk_document_garage` (les documents justificatifs du garage, sans intérêt sans lui).

Situation réelle de l'unique garage de la base (n°3) :

```
SELECT g.idGarage,
       (SELECT COUNT(*) FROM vehicule v      WHERE v.idGarage = g.idGarage) AS nb_veh,
       (SELECT COUNT(*) FROM intervention i  WHERE i.idGarage = g.idGarage) AS nb_int,
       (SELECT COUNT(*) FROM technicien t   WHERE t.idGarage = g.idGarage) AS nb_tech
FROM garage g;
```

| idGarage | nb_veh | nb_int | nb_tech |
|----------|--------|--------|---------|
| 3        | 0      | 5      | 2       |

Si on exécutait `DELETE FROM garage WHERE idGarage = 3` (syntaxe vérifiée par `EXPLAIN`, non exécutée) :

- la ligne du garage et ses documents ( `documentgarage` ) disparaissent ;
- les 5 interventions **restent**, avec `idGarage = NULL` : l'historique d'entretien des véhicules est préservé ;
- les 2 techniciens **restent**, avec `idGarage = NULL` : ils ne perdent pas leur compte ;
- les 7 lignes du journal qui citent ce garage ( `SELECT COUNT(*) FROM journalactivites WHERE idGarage = 3` renvoie 7) restent, avec `idGarage = NULL`.

(La table `documentgarage` est vide aujourd'hui : la cascade n'aurait rien à supprimer.)

Le SET NULL n'est possible que parce que ces colonnes sont déclarées `DEFAULT NULL` (elles acceptent NULL). Sur une colonne `NOT NULL`, MySQL refuserait de créer une telle contrainte.

### Piège fréquent

Le SET NULL peut créer une incohérence métier que MySQL ne voit pas. Les 2 techniciens du garage 3 ont `typeTechnicien = 'GARAGE'`. Après suppression du garage, ils seraient « techniciens de garage » sans garage. Aucune contrainte ne lie `typeTechnicien` et `idGarage` : c'est une faiblesse à reconnaître (voir Q30).

## Q25. Et si on supprime une intervention ?

L'intervention est le cœur de la base : 5 clés étrangères pointent vers elle, avec trois comportements différents.

| Table fille      | Contrainte                 | Règle    | Effet                                       |
|------------------|----------------------------|----------|---------------------------------------------|
| reparation       | fk_reparation_intervention | CASCADE  | réparations supprimées                      |
| journalactivites | fk_journal_intervention    | CASCADE  | lignes du journal supprimées                |
| anomalie         | fk_anomalie_intervention   | SET NULL | anomalie conservée, détachée                |
| analyseia        | fk_analyse_intervention    | SET NULL | analyse conservée, détachée                 |
| paiement         | fk_paiement_intervention   | RESTRICT | suppression refusée s'il existe un paiement |

On peut compter, pour chaque intervention, ce qui serait touché :

```
SELECT i.idIntervention,
       (SELECT COUNT(*) FROM journalactivites j WHERE j.idIntervention = i.idIntervention) AS
nb_journal,
       (SELECT COUNT(*) FROM reparation r           WHERE r.idIntervention = i.idIntervention) AS
nb_rep,
       (SELECT COUNT(*) FROM paiement p           WHERE p.idIntervention = i.idIntervention) AS
nb_paie,
       (SELECT COUNT(*) FROM anomalie a           WHERE a.idIntervention = i.idIntervention) AS
nb_anom
FROM intervention i
ORDER BY i.idIntervention;
```

| idIntervention | nb_journal | nb_rep | nb_paie | nb_anom |
|----------------|------------|--------|---------|---------|
| 4              | 0          | 0      | 0       | 0       |
| 5              | 0          | 0      | 0       | 0       |
| 916            | 0          | 0      | 0       | 0       |
| 917            | 0          | 0      | 0       | 0       |
| 918            | 0          | 1      | 0       | 0       |
| 923            | 0          | 0      | 0       | 0       |
| 924            | 0          | 0      | 0       | 0       |
| 925            | 3          | 0      | 0       | 0       |
| 926            | 2          | 1      | 0       | 1       |
| 932            | 3          | 0      | 0       | 0       |

Scénario `DELETE FROM intervention WHERE idIntervention = 926` (non exécuté) : aucun paiement, donc pas de blocage. MySQL supprimerait l'intervention, sa réparation et les 2 lignes du journal ; l'anomalie

resterait avec `idIntervention = NULL`. La cascade continue à l'étage suivant : la suppression de la réparation (n°915) mettrait à `NULL` les colonnes `idReparation` de `journalactivites` et `analyseia` qui la citeraient (ici aucune ligne ne la cite).

Si un paiement existait pour cette intervention, `RESTRICT` refuserait **toute** l'opération : ni l'intervention, ni la réparation, ni le journal ne seraient supprimés. C'est voulu : un paiement est une trace financière qui ne doit pas disparaître par une cascade.

### Piège fréquent

Le jury peut demander : « le journal d'activité ne devrait-il pas survivre ? ». C'est une bonne remarque. Avec `fk_journal_intervention` en `CASCADE`, supprimer une intervention efface aussi sa trace dans le journal. Pour un vrai journal d'audit, `SET NULL` serait plus adapté (comme c'est déjà le cas pour `fk_journal_reparation` et `fk_journal_anomalie`).

## Les autres contraintes

### Q26. À quoi sert `UNIQUE` ? Où l'avez-vous utilisé ?

Une contrainte `UNIQUE` interdit que deux lignes aient la même valeur dans une colonne. Contrairement à la clé primaire, une table peut en avoir plusieurs, et la colonne peut accepter `NULL`. MySQL la met en œuvre avec un index unique.

```
SELECT TABLE_NAME, INDEX_NAME, GROUP_CONCAT(COLUMN_NAME) AS colonnes
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = DATABASE()
  AND INDEX_NAME <> 'PRIMARY' AND NON_UNIQUE = 0
GROUP BY TABLE_NAME, INDEX_NAME;
```

| TABLE_NAME  | INDEX_NAME      | colonnes        |
|-------------|-----------------|-----------------|
| entreprise  | idEntreprise    | idEntreprise    |
| garage      | idUtilisateur   | idUtilisateur   |
| utilisateur | email           | email           |
| vehicule    | immatriculation | immatriculation |
| vehicule    | numeroChassis   | numeroChassis   |

- `NON_UNIQUE = 0` signifie « index unique ».
- `<> 'PRIMARY'` (« différent de ») exclut les clés primaires, déjà uniques par nature.
- `GROUP_CONCAT` colle les noms de colonnes d'un même index (utile pour un index sur plusieurs colonnes).

Rôle de chacune :

- `utilisateur.email` : deux comptes ne peuvent pas avoir le même email, qui sert d'identifiant de connexion. Un doublon provoque l'erreur 1062 « Duplicate entry ».
- `vehicule.immatriculation` et `vehicule.numeroChassis` : une plaque et un numéro de châssis désignent un seul véhicule réel.
- `garage.idUtilisateur` : un compte utilisateur gère au plus un garage. Cette contrainte transforme la clé étrangère en relation **1 à 1**.
- `entreprise.idEntreprise` : nécessaire à l'`AUTO_INCREMENT` (voir Q17).

Vérification sur les données :

```
SELECT COUNT(*) AS total, COUNT(immatriculation) AS avec_immat,
       COUNT(DISTINCT immatriculation) AS distinctes, COUNT(numeroChassis) AS avec_chassis
FROM vehicule;
```

| total | avec_immat | distinctes | avec_chassis |
|-------|------------|------------|--------------|
| 9     | 9          | 9          | 0            |

COUNT(colonne) compte les valeurs non NULL ; COUNT(DISTINCT colonne) compte les valeurs différentes. 9 immatriculations, 9 distinctes : aucun doublon. Aucun véhicule n'a de numéro de châssis renseigné.

### Piège fréquent

« Si numeroChassis est UNIQUE, comment 9 véhicules peuvent-ils tous l'avoir vide ? » Parce que NULL signifie « valeur inconnue » et que deux inconnues ne sont pas considérées comme égales : une colonne UNIQUE accepte autant de NULL qu'on veut. Seules les valeurs renseignées doivent être distinctes.

## Q27. Que signifient NOT NULL et DEFAULT ? Donnez des exemples.

NOT NULL rend une colonne **obligatoire** : un INSERT ou un UPDATE qui y mettrait NULL est refusé. DEFAULT donne la valeur utilisée **quand l'INSERT ne cite pas la colonne**. Les deux se combinent souvent : la colonne est obligatoire, mais on n'a pas besoin de la fournir.

```
SELECT COLUMN_NAME, IS_NULLABLE, COLUMN_DEFAULT, EXTRA
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'paiement'
ORDER BY ORDINAL_POSITION;
```

| COLUMN_NAME    | IS_NULLABLE | COLUMN_DEFAULT | EXTRA          |
|----------------|-------------|----------------|----------------|
| idPaieiment    | NO          | NULL           | auto_increment |
| idClient       | NO          | NULL           |                |
| idIntervention | NO          | NULL           |                |
| montant        | NO          | NULL           |                |
| datePaieiment  | YES         | NULL           |                |
| typePaieiment  | YES         | NULL           |                |
| statut         | NO          | EN_ATTENTE     |                |

Lecture : un paiement doit obligatoirement avoir un client, une intervention et un montant ( IS\_NULLABLE = NO sans défaut : il faut les fournir). La date et le type peuvent rester vides tant que le paiement n'est pas confirmé. Le statut est obligatoire mais vaut 'EN\_ATTENTE' si on ne le donne pas.

Autres valeurs par défaut utiles dans ma base :

- dateCreation DATETIME NOT NULL DEFAULT CURRENT\_TIMESTAMP (utilisateur, vehicule) : MySQL inscrit lui-même la date et l'heure de l'insertion.
- statut ENUM('PLANIFIEE', 'EN\_COURS', 'TERMINEE', 'ANNULEE') NOT NULL DEFAULT 'PLANIFIEE' (intervention) : ENUM limite la colonne à une liste fixe de valeurs ; toute nouvelle intervention démarre « planifiée ».
- kilometrage INT NOT NULL DEFAULT '0' (vehicule), themePreference ... DEFAULT 'light' (utilisateur), niveau ... DEFAULT 'MOYEN' (anomalie).

Exemple : `INSERT INTO vehicule (idClient, marque, modele) VALUES (?, 'Toyota', 'Corolla')` crée un véhicule avec `etat = 'BON'`, `kilometrage = 0` et `dateCreation = maintenant`, sans les avoir écrits.

### Piège fréquent

`NOT NULL` n'empêche pas une chaîne vide `' '` ni un nombre négatif. Pour cela il faut une contrainte `CHECK` (gérée depuis MySQL 8.0.16). Ma base n'en a aucune (`SELECT COUNT(*) FROM information_schema.CHECK_CONSTRAINTS WHERE CONSTRAINT_SCHEMA = DATABASE()` renvoie 0) ; ces contrôles sont faits en PHP. On pourrait ajouter par exemple :

```
ALTER TABLE paiement
ADD CONSTRAINT chk_paiement_montant CHECK (montant > 0);
```

### Q28. Qu'est-ce qu'un index ? Quels index votre base possède-t-elle ?

Un **index** est une structure de recherche rapide, comme l'index alphabétique à la fin d'un livre. Sans index, pour trouver une ligne, MySQL lit toute la table (on dit « parcours complet »). Avec un index (un arbre trié, de type `BTREE`), il va directement à la bonne ligne. Contrepartie : un index occupe de la place et ralentit un peu les écritures, car il doit être mis à jour.

```
SELECT CASE WHEN INDEX_NAME = 'PRIMARY' THEN 'PRIMARY'
           WHEN NON_UNIQUE = 0 THEN 'UNIQUE'
           ELSE 'INDEX simple' END AS sorte,
COUNT(DISTINCT TABLE_NAME, INDEX_NAME) AS nb
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = DATABASE()
GROUP BY sorte;
```

|              |    |
|--------------|----|
| sorte        | nb |
| INDEX simple | 27 |
| PRIMARY      | 19 |
| UNIQUE       | 5  |

Soit 51 index. Les 27 index simples sont tous des index sur des colonnes de clé étrangère, nommés comme la contrainte (`fk_vehicule_client`, `fk_intervention_vehicule`...). InnoDB **exige** un index sur chaque colonne de clé étrangère, pour vérifier rapidement les filles lors d'une suppression ; s'il n'existe pas, il le crée automatiquement. Les 7 autres clés étrangères (34 – 27) réutilisent un index existant : la clé primaire pour `client`, `technicien`, `administrateur`, `particulier`, `entreprise`, `analyseia`, et l'index unique pour `garage.idUtilisateur`.

Index d'une table précise :

```
SHOW INDEX FROM vehicule;
```

| Key_name           | Column_name     | Non_unique | Index_type |
|--------------------|-----------------|------------|------------|
| PRIMARY            | idVehicule      | 0          | BTREE      |
| numeroChassis      | numeroChassis   | 0          | BTREE      |
| immatriculation    | immatriculation | 0          | BTREE      |
| fk_vehicule_client | idClient        | 1          | BTREE      |
| fk_vehicule_garage | idGarage        | 1          | BTREE      |

Pour voir si une requête utilise un index, on la préfixe par `EXPLAIN` :

```
EXPLAIN SELECT * FROM vehicule WHERE couleur = 'Rouge';
```

| table    | type | possible_keys | key  | rows |
|----------|------|---------------|------|------|
| vehicule | ALL  | NULL          | NULL | 9    |

type = ALL et key = NULL : aucun index sur couleur, MySQL lit les 9 lignes. Avec WHERE immatriculation = ... (testé avec une plaque existante, non affichée ici), MySQL utilise l'index unique et accède directement à la ligne : type = const, key = immatriculation, rows = 1. Sur 9 véhicules la différence est invisible ; sur des centaines de milliers, elle est énorme.

Pour créer un index (non exécuté) : `CREATE INDEX idx_intervention_statut ON intervention (statut);`

## Q29. Comment afficher les contraintes d'une table ?

Il y a trois méthodes, de la plus simple à la plus complète.

### 1. SHOW CREATE TABLE

```
SHOW CREATE TABLE paiement;
```

```
CREATE TABLE `paiement` (  
  `idPaielement` int NOT NULL AUTO_INCREMENT,  
  `idClient` int NOT NULL,  
  `idIntervention` int NOT NULL,  
  `montant` decimal(10,2) NOT NULL,  
  `datePaielement` datetime DEFAULT NULL,  
  `typePaielement` varchar(50) ... DEFAULT NULL,  
  `statut` enum('EN_ATTENTE','PAYE','ECHOUE','ANNULE') ... NOT NULL DEFAULT 'EN_ATTENTE',  
  PRIMARY KEY (`idPaielement`),  
  KEY `fk_paiement_client` (`idClient`),  
  KEY `fk_paiement_intervention` (`idIntervention`),  
  CONSTRAINT `fk_paiement_client` FOREIGN KEY (`idClient`) REFERENCES `client` (`idClient`) ON  
DELETE RESTRICT ON UPDATE CASCADE,  
  CONSTRAINT `fk_paiement_intervention` FOREIGN KEY (`idIntervention`) REFERENCES `intervention`  
(`idIntervention`) ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

MySQL renvoie la commande exacte qui recréerait la table : toutes les colonnes, clés, index, contraintes et le moteur. C'est la méthode la plus rapide pour répondre au jury. ( `DESCRIBE paiement` donne une vue plus courte, sans les clés étrangères.)

### 2. information\_schema.TABLE\_CONSTRAINTS : la liste des contraintes et leur type

```
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE  
FROM information_schema.TABLE_CONSTRAINTS  
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'vehicule';
```

| CONSTRAINT_NAME    | CONSTRAINT_TYPE |
|--------------------|-----------------|
| immatriculation    | UNIQUE          |
| numeroChassis      | UNIQUE          |
| PRIMARY            | PRIMARY KEY     |
| fk_vehicule_client | FOREIGN KEY     |
| fk_vehicule_garage | FOREIGN KEY     |

### 3. KEY\_COLUMN\_USAGE et REFERENTIAL\_CONSTRAINTS : le détail des clés étrangères

KEY\_COLUMN\_USAGE dit quelle colonne pointe vers quelle table (exemple complet en Q18).

REFERENTIAL\_CONSTRAINTS donne les règles ON DELETE / ON UPDATE :

```
SELECT CONSTRAINT_NAME, TABLE_NAME, REFERENCED_TABLE_NAME, UPDATE_RULE, DELETE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = DATABASE() AND DELETE_RULE = 'RESTRICT';
```

| CONSTRAINT_NAME          | TABLE_NAME   | REFERENCED_TABLE | UPDATE_RULE | DELETE_RULE |
|--------------------------|--------------|------------------|-------------|-------------|
| fk_intervention_client   | intervention | client           | CASCADE     | RESTRICT    |
| fk_intervention_vehicule | intervention | vehicule         | CASCADE     | RESTRICT    |
| fk_paiement_client       | paiement     | client           | CASCADE     | RESTRICT    |
| fk_paiement_intervention | paiement     | intervention     | CASCADE     | RESTRICT    |
| fk_vehicule_client       | vehicule     | client           | CASCADE     | RESTRICT    |

- DATABASE() est une fonction qui renvoie le nom de la base courante (charles) : on évite ainsi de l'écrire en dur.
- Dans KEY\_COLUMN\_USAGE, la condition REFERENCED\_TABLE\_NAME IS NOT NULL garde seulement les clés étrangères (les clés primaires et uniques y figurent aussi, mais sans table référencée).

#### Q30. Votre schéma a-t-il des faiblesses ?

Oui, et il vaut mieux les citer soi-même avec la correction possible. Celles qui sont réellement présentes dans la base :

1. **Le paiement d'une réparation n'est pas lié à la réparation dans la base actuelle.** La table paiement n'a que idClient et idIntervention. Or le code (includes/payments.php) paie une réparation et attend des colonnes idReparation, referenceExterne et referenceCampay, ajoutées par le script scripts/migrate\_structure.php. Ce script n'a pas été appliqué sur cette base : la fonction paymentsReady() renvoie donc faux et le paiement en ligne reste inactif. De plus, même après migration, ce script ajoute idReparation avec un simple index (idx\_paiement\_reparation) mais **sans clé étrangère** : MySQL ne garantirait pas que la réparation existe. Correction (non exécutée) :

```
ALTER TABLE paiement
ADD CONSTRAINT fk_paiement_reparation FOREIGN KEY (idReparation)
REFERENCES reparation (idReparation) ON DELETE RESTRICT ON UPDATE CASCADE;
```

2. **Cohérence entre intervention.idClient et le propriétaire du véhicule.** Une intervention stocke à la fois idClient et idVehicule, deux clés étrangères indépendantes. Rien n'empêche, en SQL, une intervention dont le client n'est pas le propriétaire du véhicule. Aujourd'hui les données sont cohérentes :

```
SELECT COUNT(*) AS nb_incoherences
FROM intervention i
JOIN vehicule v ON v.idVehicule = i.idVehicule
WHERE v.idClient <> i.idClient;
```

```
nb_incoherences
0
```

mais c'est le PHP qui le garantit, pas la base. (C'est une redondance : on pourrait retrouver le client par le véhicule.)

3. **typeTechnicien et idGarage ne sont pas liés.** Un technicien 'GARAGE' devrait avoir un garage, un 'INTERNE' non. Aucune contrainte ne l'impose, et le SET NULL de fk\_technicien\_garage peut produire un technicien « GARAGE » sans garage (Q24). Les données actuelles sont cohérentes (6 internes sans garage, 2 techniciens de garage avec garage). Correction possible : CHECK (typeTechnicien = 'INTERNE' OR idGarage IS NOT NULL) . Attention : depuis MySQL 8.0.16, une contrainte CHECK ne peut pas porter sur une colonne utilisée par une clé étrangère qui a une action référentielle CASCADE, SET NULL ou SET DEFAULT, que ce soit en ON DELETE ou en ON UPDATE . Comme fk\_technicien\_garage est en ON DELETE SET NULL ON UPDATE CASCADE, il faudrait d'abord la passer en RESTRICT pour les deux (non testé ici, aucune modification n'ayant été faite).
4. **Aucune contrainte CHECK** : montants, coûts, kilométrage ou durée négatifs ne sont pas refusés par la base (Q27).
5. **Le journal d'activité est effacé avec l'intervention** (fk\_journal\_intervention en CASCADE), alors qu'un journal devrait conserver l'historique (Q25).
6. **Petites incohérences de style** : les trois tables ajoutées plus tard (messages, notifications, technician\_documents) n'ont pas de ON UPDATE CASCADE (règle NO ACTION par défaut) et suivent un autre nommage (id, user\_id, noms en anglais).
7. **Le rôle « exactement un par utilisateur » n'est pas imposé par la base.** Rien n'empêche techniquement un même idUtilisateur d'être à la fois dans client et technicien, ni un client d'être à la fois particulier et entreprise. Les vérifications par NOT EXISTS renvoient 0 cas aujourd'hui, mais c'est la transaction d'inscription en PHP qui l'assure.

### À retenir

- Toutes les relations principales sont protégées par 34 clés étrangères InnoDB, avec des règles choisies selon la valeur des données (CASCADE, SET NULL, RESTRICT).
- Les règles qui portent sur plusieurs colonnes à la fois (client ↔ véhicule, type ↔ garage) sont vérifiées en PHP, pas en SQL.
- La correction la plus urgente : appliquer la migration du paiement et lier paiement.idReparation à reparation par une clé étrangère.

## 3. Les commandes SQL essentielles

Ce chapitre reprend les commandes SQL que le jury peut vous demander d'écrire ou d'expliquer au tableau. Chaque commande est décomposée mot-clé par mot-clé, puis appliquée à votre base `charles`. Les requêtes `SELECT` ont été exécutées sur la vraie base et les résultats affichés sont ceux obtenus. Les commandes qui modifient les données (`INSERT`, `UPDATE`, `DELETE`) ont seulement été vérifiées avec `EXPLAIN`, qui contrôle la syntaxe sans rien modifier. Les commandes qui modifient la structure (`CREATE`, `ALTER`, `DROP`, `GRANT`) sont écrites et expliquées, mais n'ont pas été exécutées.

### 3.1 Les cinq familles de commandes

#### Q31. Quelles sont les grandes familles de commandes SQL, et à quoi sert chacune ?

SQL (Structured Query Language, « langage de requêtes structuré ») est le langage qui permet de parler à un SGBD relationnel comme MySQL. Ses commandes se rangent en cinq familles, selon ce qu'elles touchent : la **structure** de la base, les **données**, la **lecture**, les **droits** ou les **transactions**.

| Famille    | Signification                                                       | Rôle                                                                                         | Commandes                                                                                                           | Exemple dans votre projet                                                                                                                    |
|------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DDL</b> | Data Definition Language (langage de définition des données)        | Crée, modifie ou supprime la <b>structure</b> : bases, tables, colonnes, contraintes, index. | <code>CREATE</code> ,<br><code>ALTER</code> , <code>DROP</code> ,<br><code>TRUNCATE</code> ,<br><code>RENAME</code> | Le script <code>scripts/migrate_anomalie_intervention.php</code> fait un <code>ALTER TABLE anomalie ADD COLUMN idIntervention</code> .       |
| <b>DML</b> | Data Manipulation Language (langage de manipulation des données)    | Ajoute, modifie ou supprime des <b>lignes</b> dans les tables existantes.                    | <code>INSERT</code> ,<br><code>UPDATE</code> ,<br><code>DELETE</code>                                               | <code>client/vehicles.php</code> fait un <code>INSERT INTO vehicule</code> quand un client ajoute un véhicule.                               |
| <b>DQL</b> | Data Query Language (langage d'interrogation)                       | <b>Lit</b> les données sans les modifier.                                                    | <code>SELECT</code>                                                                                                 | Tous les tableaux de bord (compter les anomalies, lister les interventions...).                                                              |
| <b>DCL</b> | Data Control Language (langage de contrôle des accès)               | Donne ou retire des <b>droits</b> aux comptes MySQL.                                         | <code>GRANT</code> ,<br><code>REVOKE</code>                                                                         | Créer un compte MySQL dédié à l'application au lieu de <code>root</code> .                                                                   |
| <b>TCL</b> | Transaction Control Language (langage de contrôle des transactions) | Regroupe plusieurs modifications en un bloc « tout ou rien ».                                | <code>START TRANSACTION</code> ,<br><code>COMMIT</code> ,<br><code>ROLLBACK</code> ,<br><code>SAVEPOINT</code>      | <code>technicien/reparations.php</code> enregistre la réparation, clôture l'intervention et résout les anomalies dans une seule transaction. |

Certains manuels rangent `SELECT` dans le DML, car il « manipule » les données en lecture. Les deux classements se rencontrent ; l'essentiel est de savoir que `SELECT` ne modifie rien.

### À retenir

- DDL = la structure (le « moule »), DML = le contenu (les lignes), DQL = la lecture.
- DCL = qui a le droit de faire quoi, TCL = valider ou annuler un groupe de modifications.
- Dans MySQL, une commande DDL valide automatiquement la transaction en cours (« commit implicite ») : on ne peut pas annuler un `DROP TABLE` avec `ROLLBACK`.

## 3.2 DDL : créer et modifier la structure

### Q32. Comment créer votre base de données ? Expliquez chaque mot de la commande.

```
CREATE DATABASE IF NOT EXISTS charles
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE charles;
```

- `CREATE DATABASE` : crée une nouvelle base, c'est-à-dire un conteneur vide qui recevra les tables.
- `IF NOT EXISTS` : n'essaie de créer la base que si elle n'existe pas déjà. Sans cette option, MySQL renvoie une erreur si `charles` existe.
- `charles` : le nom de votre base (celui utilisé dans `config/database.php`).
- `CHARACTER SET utf8mb4` : le jeu de caractères, c'est-à-dire la façon dont les lettres sont stockées. `utf8mb4` accepte tous les caractères Unicode, y compris les accents (« Contrôle général ») et les émojis. L'ancien `utf8` de MySQL ne gère pas certains caractères sur 4 octets.
- `COLLATE utf8mb4_unicode_ci` : la collation, c'est-à-dire la règle de comparaison et de tri du texte. Le suffixe `ci` signifie « case insensitive » (insensible à la casse) : « toyota » et « Toyota » sont considérés égaux dans un `WHERE`.
- `USE charles` : sélectionne la base sur laquelle les commandes suivantes s'appliqueront.

Vos 19 tables sont toutes en `utf8mb4 / utf8mb4_unicode_ci` et utilisent le moteur InnoDB, ce qui est cohérent avec cette création.

### Q33. Écrivez la commande `CREATE TABLE` de la table `vehicule` (version simplifiée) et expliquez chaque élément.

Voici une version simplifiée mais fidèle de votre vraie table `vehicule` :

```
CREATE TABLE vehicule (
  idVehicule      INT          NOT NULL AUTO_INCREMENT,
  idClient         INT          NOT NULL,
  idGarage         INT          NULL,
  marque          VARCHAR(100) NOT NULL,
  modele          VARCHAR(100) NOT NULL,
  immatriculation VARCHAR(50)  NULL,
  etat            VARCHAR(50)  DEFAULT 'BON',
  kilometrage     INT          NOT NULL DEFAULT 0,
  annee           INT          NULL,
  dateCreation     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (idVehicule),
  UNIQUE KEY immatriculation (immatriculation),
  CONSTRAINT fk_vehicule_client FOREIGN KEY (idClient)
    REFERENCES client (idClient)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_vehicule_garage FOREIGN KEY (idGarage)
    REFERENCES garage (idGarage)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

## Les mots-clés, un par un

- `CREATE TABLE vehicule ( ... )` : crée la table `vehicule` ; entre parenthèses, on liste les colonnes puis les contraintes, séparées par des virgules.
- `INT` : nombre entier. `VARCHAR(100)` : texte de longueur variable, 100 caractères au maximum. `DATETIME` : date et heure (2026-09-29 11:01:49).
- `NOT NULL` : la colonne est obligatoire, elle ne peut pas rester vide. `NULL` : la colonne est facultative (par exemple `idGarage`, car un véhicule n'est pas forcément rattaché à un garage).
- `AUTO_INCREMENT` : MySQL attribue lui-même le numéro, en ajoutant 1 au dernier utilisé. Vous n'avez jamais à fournir `idVehicule` lors d'un ajout.
- `DEFAULT 'BON'`, `DEFAULT 0`, `DEFAULT CURRENT_TIMESTAMP` : valeur mise automatiquement si l'`INSERT` ne donne pas la colonne. `CURRENT_TIMESTAMP` est la date et l'heure du moment de l'insertion.
- `PRIMARY KEY (idVehicule)` : la clé primaire identifie chaque ligne de façon unique ; elle est forcément non nulle et sans doublon.
- `UNIQUE KEY immatriculation (immatriculation)` : interdit deux véhicules avec la même plaque. Contrairement à la clé primaire, une colonne `UNIQUE` peut contenir plusieurs `NULL`.
- `CONSTRAINT fk_vehicule_client` : donne un nom à la contrainte, ce qui permet de la retrouver ou de la supprimer plus tard.
- `FOREIGN KEY (idClient) REFERENCES client (idClient)` : clé étrangère. La valeur de `idClient` doit exister dans la table `client` : on ne peut pas enregistrer un véhicule pour un client inexistant.
- `ON DELETE RESTRICT` : interdit de supprimer un client qui possède encore des véhicules.
- `ON DELETE SET NULL` (sur `idGarage`) : si le garage est supprimé, le véhicule reste, mais son `idGarage` devient `NULL`.
- `ON UPDATE CASCADE` : si l'identifiant du client ou du garage change, la modification est répercutée automatiquement dans `vehicule`.
- `ENGINE=InnoDB` : le moteur de stockage. InnoDB est celui qui gère les clés étrangères et les transactions.

Votre vraie table contient en plus `dateAcquisition` (DATE), `numeroChassis` (VARCHAR(100), lui aussi UNIQUE) et `couleur` (VARCHAR(50)), ainsi que les index `fk_vehicule_client` et `fk_vehicule_garage`.

MySQL crée ces index automatiquement pour chaque clé étrangère s'ils n'existent pas.

### Piège fréquent

L'ordre de création compte : on ne peut pas créer `vehicule` avant `client` et `garage`, puisque ses clés étrangères y font référence. L'ordre logique est `utilisateur` → `client` → `garage` → `vehicule` → `intervention` → `anomalie`.

## Q34. Comment modifier une table existante sans la recréer ? (ALTER TABLE ADD, MODIFY, DROP)

`ALTER TABLE` modifie la structure d'une table qui contient déjà des données, sans les perdre. Votre projet l'a utilisé dans ses scripts de migration.

### Ajouter une colonne : ADD COLUMN

Exemple réel, tiré de `scripts/migrate_anomalie_intervention.php` :

```
ALTER TABLE anomalie
  ADD COLUMN idIntervention INT NULL AFTER idVehicule;

ALTER TABLE anomalie
  ADD CONSTRAINT fk_anomalie_intervention
  FOREIGN KEY (idIntervention) REFERENCES intervention (idIntervention)
  ON DELETE SET NULL ON UPDATE CASCADE;
```

- `ALTER TABLE anomalie` : on modifie la table `anomalie`.
- `ADD COLUMN idIntervention INT NULL` : ajoute une colonne entière facultative. Elle doit être `NULL`, car les anomalies déjà présentes n'ont pas de valeur pour elle.
- `AFTER idVehicule` : place la colonne juste après `idVehicule` (mot-clé propre à MySQL ; `FIRST` la mettrait en premier).
- `ADD CONSTRAINT ... FOREIGN KEY` : ajoute ensuite la clé étrangère vers `intervention`.

### Changer le type d'une colonne : MODIFY COLUMN

Exemple réel, tiré de `scripts/migrate_admin_suspension_v2.php`, qui a ajouté la valeur `SUSPENDU` :

```
ALTER TABLE garage
  MODIFY COLUMN statutGarage
  ENUM('EN_ATTENTE', 'VALIDE', 'REJETE', 'SUSPENDU') NOT NULL DEFAULT 'EN_ATTENTE';
```

`MODIFY` redéfinit entièrement la colonne : il faut réécrire son type, `NOT NULL` et `DEFAULT`. Si on oublie `NOT NULL`, la colonne devient facultative.

## Autres variantes

```
-- Renommer une colonne et changer son type en même temps
ALTER TABLE vehicule CHANGE COLUMN couleur couleurCarrosserie VARCHAR(50) NULL;

-- Renommer seulement (MySQL 8)
ALTER TABLE vehicule RENAME COLUMN couleur TO couleurCarrosserie;

-- Supprimer une colonne (et toutes ses données)
ALTER TABLE vehicule DROP COLUMN couleur;

-- Ajouter une règle de contrôle (vérifiée par MySQL depuis la version 8.0.16)
ALTER TABLE vehicule ADD CONSTRAINT chk_km CHECK (kilometrage >= 0);
```

### Piège fréquent

MODIFY garde le nom de la colonne et change sa définition ; CHANGE permet aussi de la renommer, et demande donc l'ancien et le nouveau nom. Si on réduit un ENUM en retirant une valeur déjà utilisée dans des lignes, MySQL refuse la modification en mode strict.

## Q35. Quelle est la différence entre DROP, TRUNCATE et DELETE ?

Les trois « suppriment », mais pas la même chose ni de la même manière.

```
DROP TABLE notifications;           -- la table disparaît (structure + données)
TRUNCATE TABLE notifications;       -- la table reste, vide, compteur remis à 1
DELETE FROM notifications WHERE lu = 'oui'; -- supprime seulement les lignes choisies
```

| Critère                  | DELETE                                                            | TRUNCATE                                     | DROP                                         |
|--------------------------|-------------------------------------------------------------------|----------------------------------------------|----------------------------------------------|
| Famille                  | DML                                                               | DDL                                          | DDL                                          |
| Ce qui est supprimé      | Les lignes qui vérifient le WHERE (toutes sans WHERE)             | Toutes les lignes                            | Les lignes <b>et</b> la table elle-même      |
| La table existe encore ? | Oui                                                               | Oui, vide                                    | Non                                          |
| Accepte un WHERE ?       | Oui                                                               | Non                                          | Non                                          |
| Compteur AUTO_INCREMENT  | Conservé (le prochain id continue)                                | Remis au départ                              | Disparaît avec la table                      |
| Annulable par ROLLBACK   | Oui, dans une transaction                                         | Non (commit implicite)                       | Non (commit implicite)                       |
| Clés étrangères          | Applique les règles CASCADE / SET NULL / RESTRICT ligne par ligne | Refusé si une autre table référence celle-ci | Refusé si une autre table référence celle-ci |
| Vitesse                  | Plus lent (ligne par ligne)                                       | Très rapide                                  | Très rapide                                  |

Exemple sur votre base : TRUNCATE TABLE vehicule serait refusé, car intervention et anomalie ont des clés étrangères vers vehicule. TRUNCATE TABLE notifications fonctionnerait, car aucune table ne référence notifications.

## À retenir

- DELETE = trier et retirer certaines lignes ; TRUNCATE = vider ; DROP = détruire.
- Seul DELETE peut être annulé par ROLLBACK.

## 3.3 DML : ajouter, modifier, supprimer des lignes

### Q36. Comment ajoute-t-on un véhicule dans la base ? Expliquez la commande INSERT INTO.

```
INSERT INTO vehicule (idClient, marque, modele, immatriculation, annee, couleur, kilometrage)
VALUES (6, 'Toyota', 'Hilux', 'CE-123-AB', 2019, 'Blanc', 85000);
```

- INSERT INTO vehicule : on ajoute une ligne dans la table vehicule.
- (idClient, marque, ...) : la liste des colonnes que l'on remplit, dans l'ordre choisi.
- VALUES (6, 'Toyota', ...) : les valeurs, dans le même ordre que les colonnes. Le texte et les dates s'écrivent entre apostrophes ; les nombres s'écrivent sans.

Les colonnes absentes de la liste reçoivent leur valeur par défaut : idVehicule est calculé par AUTO\_INCREMENT, etat vaut 'BON', dateCreation vaut la date du jour, et idGarage, numeroChassis et dateAcquisition valent NULL.

Vérification de la syntaxe avec EXPLAIN (aucune ligne n'est ajoutée) :

| id | select_type | table    | type |
|----|-------------|----------|------|
| 1  | INSERT      | vehicule | ALL  |

MySQL a compris la commande comme un INSERT dans vehicule : la syntaxe est correcte.

### Dans le code PHP

Votre fichier client/vehicles.php écrit la même commande avec des ? à la place des valeurs :

```
INSERT INTO vehicule (idClient, marque, modele, immatriculation, annee, couleur, kilometrage)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

C'est une **requête préparée** : PDO envoie d'abord la structure de la requête, puis les valeurs séparément avec execute([...]). Une saisie malveillante ne peut donc pas être interprétée comme du SQL : c'est la protection contre l'injection SQL. Le idClient vient de la session (\$\_SESSION['user\_id']), pas du formulaire.

### Les erreurs possibles

- Plaque déjà utilisée : erreur 1062 « Duplicate entry », à cause de la contrainte UNIQUE sur immatriculation. Votre code vérifie d'ailleurs avant l'insertion avec un SELECT idVehicule FROM vehicule WHERE immatriculation = ?.
- Client inexistant : erreur 1452 « Cannot add or update a child row: a foreign key constraint fails », à cause de fk\_vehicule\_client.
- Marque absente : erreur, car marque est NOT NULL sans valeur par défaut.

## Piège fréquent

`EXPLAIN` vérifie la syntaxe mais ne contrôle ni les clés étrangères ni l'unicité : ces contrôles n'ont lieu qu'à l'exécution réelle. Après l'insertion, PHP récupère le numéro attribué avec `$conn->lastInsertId()`, comme le fait `technicien/reparations.php` pour la réparation.

### Q37. Comment passer une anomalie au statut TRAITEE ? Pourquoi le WHERE est-il indispensable ?

```
UPDATE anomalie
SET statut = 'TRAITEE',
    dateResolution = NOW()
WHERE idAnomalie = 915;
```

- `UPDATE anomalie` : on modifie des lignes existantes de la table `anomalie`.
- `SET statut = 'TRAITEE', dateResolution = NOW()` : les nouvelles valeurs. On peut modifier plusieurs colonnes en les séparant par des virgules. `NOW()` renvoie la date et l'heure actuelles.
- `WHERE idAnomalie = 915` : la condition qui désigne la ou les lignes à modifier. Ici, une seule ligne, repérée par sa clé primaire.

Avant la modification, on peut regarder l'état actuel de la ligne (requête exécutée) :

```
SELECT idAnomalie, statut, idIntervention FROM anomalie WHERE idAnomalie = 915;
```

| idAnomalie | statut   | idIntervention |
|------------|----------|----------------|
| 915        | EN_COURS | NULL           |

Vérification de l'`UPDATE` avec `EXPLAIN` (rien n'est modifié) :

| select_type | table    | type  | key     | rows |
|-------------|----------|-------|---------|------|
| UPDATE      | anomalie | range | PRIMARY | 1    |

MySQL utilise la clé primaire et prévoit de toucher **1 ligne**.

### La version réelle de votre application

Dans `technicien/reparations.php` et `garage/reparations.php`, lorsqu'une réparation est enregistrée, toutes les anomalies encore ouvertes de l'intervention sont résolues :

```
UPDATE anomalie
SET statut = 'TRAITEE', dateResolution = NOW()
WHERE idIntervention = ?
    AND statut IN ('NOUVELLE', 'EN_COURS');
```

Le `WHERE` combine deux conditions : la bonne intervention **et** un statut encore ouvert. Ainsi, une anomalie `IGNOREE` ou déjà `TRAITEE` n'est pas modifiée, et sa date de résolution d'origine est conservée.

## Piège fréquent

Sans WHERE, UPDATE anomalie SET statut = 'TRAITEE' modifie **toutes** les lignes de la table : les 4 anomalies de votre base passeraient à TRAITEE. La bonne habitude est d'écrire d'abord un SELECT avec le même WHERE pour voir les lignes concernées, puis de le transformer en UPDATE. Autre piège : la colonne statut est un ENUM('NOUVELLE', 'EN\_COURS', 'TRAITEE', 'IGNOREE') ; écrire 'TERMINEE' (valeur de l'ENUM de intervention) provoque une erreur en mode strict.

## Q38. Comment supprimer une ligne avec DELETE, et que se passe-t-il pour les tables liées ?

```
DELETE FROM anomalie
WHERE idAnomalie = 916;
```

- DELETE FROM anomalie : on supprime des lignes de la table anomalie.
- WHERE idAnomalie = 916 : seule la ligne numéro 916 est supprimée. Comme pour UPDATE, sans WHERE toute la table est vidée.

Vérification avec EXPLAIN (rien n'est supprimé) :

| select_type | table    | type  | key     | rows |
|-------------|----------|-------|---------|------|
| DELETE      | anomalie | range | PRIMARY | 1    |

Un autre exemple, pour faire le ménage dans les notifications déjà lues et anciennes :

```
DELETE FROM notifications
WHERE lu = 'oui' AND date_creation < '2026-01-01';
```

## L'effet des clés étrangères

Quand on supprime une ligne, MySQL applique les règles ON DELETE des tables qui y font référence :

- Supprimer l'anomalie 916 : les lignes de journalactivites et analyseia qui la citaient gardent leur historique, mais leur idAnomalie passe à NULL (ON DELETE SET NULL).
- Supprimer un véhicule : ses anomalies sont supprimées automatiquement (fk\_anomalie\_vehicule est en ON DELETE CASCADE). En revanche, si le véhicule a des interventions, la suppression est **refusée**, car fk\_intervention\_vehicule est en ON DELETE RESTRICT.
- Supprimer un utilisateur : MySQL essaie de supprimer en cascade sa ligne client, technicien ou administrateur, ainsi que ses messages et notifications. Mais si c'est un client qui possède des véhicules, des interventions ou des paiements, toute la suppression est **refusée** : fk\_vehicule\_client, fk\_intervention\_client et fk\_paiement\_client sont en ON DELETE RESTRICT. Une cascade s'arrête dès qu'elle rencontre un RESTRICT, et rien n'est supprimé.

## À retenir

- DELETE supprime des lignes, jamais la table.
- Toujours un WHERE, de préférence sur la clé primaire.
- CASCADE supprime aussi les lignes liées, SET NULL les garde en coupant le lien, RESTRICT bloque la suppression.

### 3.4 DQL : lire les données avec SELECT

#### Q39. Quelle est la structure d'une requête SELECT, et comment filtre-t-on avec WHERE ?

Une requête `SELECT` s'écrit toujours dans cet ordre (les parties entre crochets sont facultatives) :

```
SELECT  colonnes          -- quoi afficher
FROM    table             -- où chercher
[WHERE  condition]        -- quelles lignes garder
[GROUP BY colonnes]       -- regrouper
[HAVING condition]        -- filtrer les groupes
[ORDER BY colonnes]       -- trier
[LIMIT  n]               -- limiter le nombre de lignes
```

Exemple : les véhicules qui ont roulé plus de 50 000 km.

```
SELECT idVehicule, marque, modele, kilometrage
FROM vehicule
WHERE kilometrage > 50000;
```

- `SELECT idVehicule, marque, modele, kilometrage` : les colonnes à afficher. `SELECT *` afficherait toutes les colonnes.
- `FROM vehicule` : la table lue.
- `WHERE kilometrage > 50000` : seules les lignes pour lesquelles la condition est vraie sont gardées.

Résultat réel :

| idVehicule | marque  | modele | kilometrage |
|------------|---------|--------|-------------|
| 918        | Citroen | Jumper | 65000       |
| 919        | Iveco   | Daily  | 142000      |

Les opérateurs de comparaison utilisables dans un `WHERE` sont : `=` (égal), `<>` ou `!=` (différent), `<`, `>`, `<=`, `>=`.

#### Piège fréquent

L'ordre d'écriture n'est pas l'ordre d'exécution. MySQL lit d'abord `FROM`, puis `WHERE`, puis `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY` et enfin `LIMIT`. C'est pourquoi un alias créé dans le `SELECT` ne peut pas être utilisé dans le `WHERE`, mais peut l'être dans l'`ORDER BY`.

#### Q40. Comment combiner plusieurs conditions avec AND et OR ?

- `AND` (et) : la ligne est gardée si **toutes** les conditions sont vraies.
- `OR` (ou) : la ligne est gardée si **au moins une** condition est vraie.
- `NOT` : inverse une condition.

Votre table `anomalie` contient actuellement 4 lignes :

| idAnomalie | niveau   | statut   |
|------------|----------|----------|
| 914        | CRITIQUE | NOUVELLE |
| 915        | MOYEN    | EN_COURS |
| 916        | MOYEN    | TRAITEE  |
| 919        | CRITIQUE | TRAITEE  |

Anomalies critiques **et** non traitées :

```
SELECT idAnomalie, niveau, statut
FROM anomalie
WHERE niveau = 'CRITIQUE' AND statut <> 'TRAITEE';
```

| idAnomalie | niveau   | statut   |
|------------|----------|----------|
| 914        | CRITIQUE | NOUVELLE |

Anomalies critiques **ou** nouvelles :

```
SELECT idAnomalie, niveau, statut
FROM anomalie
WHERE niveau = 'CRITIQUE' OR statut = 'NOUVELLE';
```

| idAnomalie | niveau   | statut   |
|------------|----------|----------|
| 914        | CRITIQUE | NOUVELLE |
| 919        | CRITIQUE | TRAITEE  |

### Piège fréquent

`AND` est prioritaire sur `OR`, comme la multiplication sur l'addition. Sans parenthèses, la requête ne fait pas ce qu'on croit :

```
-- Sans parenthèses : lu comme CRITIQUE OU (MOYEN ET NOUVELLE)
SELECT idAnomalie, niveau, statut FROM anomalie
WHERE niveau = 'CRITIQUE' OR niveau = 'MOYEN' AND statut = 'NOUVELLE';
```

| idAnomalie | niveau   | statut   |
|------------|----------|----------|
| 914        | CRITIQUE | NOUVELLE |
| 919        | CRITIQUE | TRAITEE  |

<-- traitée, pourtant affichée

```
-- Avec parenthèses : (CRITIQUE OU MOYEN) ET NOUVELLE
SELECT idAnomalie, niveau, statut FROM anomalie
WHERE (niveau = 'CRITIQUE' OR niveau = 'MOYEN') AND statut = 'NOUVELLE';
```

| idAnomalie | niveau   | statut   |
|------------|----------|----------|
| 914        | CRITIQUE | NOUVELLE |

Votre code met correctement des parenthèses dans les recherches, par exemple dans

`admin/vehicules.php` : `(v.marque LIKE ? OR v.modele LIKE ? OR ...)`, combiné ensuite par `AND` avec les autres filtres.

## Q41. Comment trier les résultats et n'en afficher qu'une partie ? (ORDER BY, LIMIT)

Les trois véhicules qui ont le plus roulé :

```
SELECT marque, modele, kilometrage
FROM vehicule
ORDER BY kilometrage DESC
LIMIT 3;
```

- `ORDER BY kilometrage` : trie les lignes selon le kilométrage.

- `DESC` : ordre décroissant (du plus grand au plus petit). `ASC`, l'ordre croissant, est la valeur par défaut si on n'écrit rien.
- `LIMIT 3` : ne garde que les 3 premières lignes du résultat trié.

Résultat réel :

| marque  | modele | kilometrage |
|---------|--------|-------------|
| Iveco   | Daily  | 142000      |
| Citroen | Jumper | 65000       |
| Renault | Master | 38000       |

On peut trier sur plusieurs colonnes : `ORDER BY kilometrage DESC, marque ASC` trie d'abord par kilométrage, puis, en cas d'égalité (le Renault Master et la Peugeot 308 ont tous deux 38 000 km), par marque.

Pour la pagination, on ajoute `OFFSET` : `LIMIT 10 OFFSET 20` saute les 20 premières lignes et affiche les 10 suivantes (c'est la page 3 si chaque page contient 10 lignes). MySQL accepte aussi l'écriture courte `LIMIT 20, 10`.

### Piège fréquent

Sans `ORDER BY`, MySQL ne garantit aucun ordre : `LIMIT 3` seul renvoie « trois lignes », pas forcément les trois premières ni les plus importantes. Avec une égalité, comme les deux véhicules à 38 000 km, l'ordre entre eux n'est pas garanti non plus : il faut un second critère de tri pour un résultat stable.

## Q42. À quoi sert `DISTINCT` ?

`DISTINCT` supprime les doublons dans le résultat : deux lignes identiques ne sont affichées qu'une fois.

```
SELECT DISTINCT marque
FROM vehicule
ORDER BY marque;
```

|                |
|----------------|
| marque         |
| Cadillac       |
| Cadillac e     |
| Citroen        |
| Iveco          |
| Mercedes       |
| Peugeot        |
| Renault        |
| Toyota Corolla |

La table contient 9 véhicules mais seulement 8 marques différentes, car il y a deux Peugeot (Boxer et 308). On peut le vérifier en comptant :

```
SELECT COUNT(*) AS total, COUNT(DISTINCT marque) AS marques_différentes
FROM vehicule;
```

| total | marques_différentes |
|-------|---------------------|
| 9     | 8                   |

Votre code l'utilise dans `garage/interventions.php` : `SELECT DISTINCT v.idVehicule ... FROM vehicule v JOIN intervention i ...`. Un véhicule qui a eu plusieurs interventions apparaîtrait plusieurs fois après la jointure ; `DISTINCT` le ramène à une seule ligne pour la liste déroulante.

### Piège fréquent

`DISTINCT` s'applique à la **ligne entière**, pas seulement à la première colonne. `SELECT DISTINCT marque, modele` renverrait deux lignes Peugeot, car les couples (Peugeot, Boxer) et (Peugeot, 308) sont différents. Remarquez aussi « Cadillac » et « Cadillac e » : ce sont deux valeurs différentes pour MySQL. C'est un défaut de saisie, et le jury peut vous le faire remarquer.

## Q43. Comment rechercher un texte partiel ? (LIKE, % et \_)

`LIKE` compare un texte à un **motif** qui contient des caractères spéciaux (« jokers ») :

- `%` remplace **n'importe quelle suite de caractères**, y compris aucune.
- `_` remplace **exactement un** caractère.

| Motif | Signification                                  |
|-------|------------------------------------------------|
| 'T%'  | commence par T                                 |
| '%o'  | se termine par o                               |
| '%o%' | contient un o n'importe où                     |
| 'DM%' | immatriculation commençant par DM              |
| '30_' | « 30 » suivi d'un seul caractère (308, 301...) |

```
SELECT marque, modele FROM vehicule WHERE marque LIKE 'T%';
```

|        |  |         |
|--------|--|---------|
| marque |  | modele  |
| Toyota |  | Corolla |
|        |  | berline |

```
SELECT marque, modele FROM vehicule WHERE modele LIKE '%o%';
```

|         |  |        |
|---------|--|--------|
| marque  |  | modele |
| Peugeot |  | Boxer  |

```
SELECT COUNT(*) AS nb FROM vehicule WHERE immatriculation LIKE 'DM%';
```

|    |
|----|
| nb |
| 5  |

Grâce à la collation `utf8mb4_unicode_ci`, `LIKE 't%'` trouve aussi « Toyota » : la casse (majuscules/minuscules) est ignorée.

Votre barre de recherche (`client/vehicles.php`) utilise `(marque LIKE ? OR modele LIKE ? OR immatriculation LIKE ?)` ; le PHP entoure le texte saisi de `%` pour chercher « contient ».

### Piège fréquent

WHERE marque = 'T%' cherche le texte exact « T% » et ne trouve rien : les jokers ne fonctionnent qu'avec LIKE. Par ailleurs, un motif qui commence par % empêche MySQL d'utiliser un index : sur une très grande table, la recherche est plus lente.

### Q44. À quoi sert IN ?

IN (liste) teste si une valeur fait partie d'une liste. C'est une écriture courte pour plusieurs OR sur la même colonne : statut IN ('PLANIFIEE', 'EN\_COURS') équivaut à statut = 'PLANIFIEE' OR statut = 'EN\_COURS'.

```
SELECT idIntervention, type, statut
FROM intervention
WHERE statut IN ('PLANIFIEE', 'EN_COURS');
```

| idIntervention | type                             | statut    |
|----------------|----------------------------------|-----------|
| 916            | Panne / dysfonctionnement        | PLANIFIEE |
| 917            | Entretien courant                | PLANIFIEE |
| 923            | Remplacement plaquettes de frein | PLANIFIEE |
| 924            | Contrôle général                 | PLANIFIEE |
| 925            | Diagnostic électronique          | EN_COURS  |
| 932            | Diagnostic général               | PLANIFIEE |

Ce sont les 6 interventions « actives » sur 10. Autre exemple : les véhicules immobilisés.

```
SELECT idVehicule, etat FROM vehicule WHERE etat IN ('EN_PANNE', 'EN_ENTRETIEN');
```

| idVehicule | etat         |
|------------|--------------|
| 918        | EN_ENTRETIEN |
| 919        | EN_PANNE     |

NOT IN fait l'inverse : WHERE statut NOT IN ('TERMINEE', 'ANNULEE'). On peut aussi mettre une sous-requête dans les parenthèses : WHERE idVehicule IN (SELECT idVehicule FROM anomalie WHERE niveau = 'CRITIQUE').

### Piège fréquent

Si la liste d'un NOT IN contient un NULL (par exemple une sous-requête qui renvoie des idIntervention vides), la condition n'est jamais vraie et le résultat est vide. Il faut alors ajouter WHERE ... IS NOT NULL dans la sous-requête, ou utiliser NOT EXISTS.

### Q45. À quoi sert BETWEEN, et quel est le piège avec les dates ?

BETWEEN a AND b garde les valeurs comprises entre a et b, **bornes incluses**. C'est équivalent à >= a AND <= b.

```
SELECT marque, modele, kilometrage
FROM vehicule
WHERE kilometrage BETWEEN 20000 AND 70000
ORDER BY kilometrage;
```

| marque  | modele | kilometrage |
|---------|--------|-------------|
| Peugeot | Boxer  | 21000       |
| Renault | Master | 38000       |
| Peugeot | 308    | 38000       |
| Citroen | Jumper | 65000       |

### Piège fréquent

Avec une colonne `DATETIME` comme `dateIntervention`, la date `'2026-09-29'` veut dire `2026-09-29 00:00:00`, c'est-à-dire minuit au tout début de la journée. Une intervention du 29 septembre à 11 h 01 est donc **exclue** par `BETWEEN '2026-09-01' AND '2026-09-29'`.

```
SELECT COUNT(*) AS nb FROM intervention
WHERE dateIntervention BETWEEN '2026-09-01' AND '2026-09-29';
```

```
nb
7
```

```
SELECT COUNT(*) AS nb FROM intervention
WHERE dateIntervention >= '2026-09-01' AND dateIntervention < '2026-09-30';
```

```
nb
8
```

La deuxième écriture (« à partir du 1er inclus, avant le 30 exclu ») compte bien l'intervention du 29 à 11 h 01. C'est la forme recommandée pour filtrer des périodes sur une colonne `DATETIME`.

### Q46. Comment trouver les valeurs manquantes ? Pourquoi ne peut-on pas écrire « = NULL » ?

`NULL` ne veut pas dire zéro ni texte vide : il signifie « valeur inconnue » ou « absente ». Comme la valeur est inconnue, MySQL ne peut pas dire si elle est égale à autre chose : `idGarage = NULL` ne donne ni vrai ni faux, mais « inconnu », et la ligne n'est jamais gardée. Il faut utiliser les opérateurs dédiés `IS NULL` et `IS NOT NULL`.

```
SELECT COUNT(*) AS nb FROM vehicule WHERE idGarage IS NULL;
```

```
nb
9
```

```
SELECT COUNT(*) AS nb FROM vehicule WHERE idGarage = NULL; -- mauvaise écriture
```

```
nb
0
```

Les 9 véhicules ont bien un `idGarage` vide, mais `= NULL` en trouve 0 : c'est la preuve que cette écriture ne fonctionne pas.

Autre exemple utile : les anomalies qui ne sont pas encore résolues n'ont pas de date de résolution.

```
SELECT COUNT(*) AS nb FROM anomalie WHERE dateResolution IS NULL;
```

```
nb
2
```

On retrouve les anomalies 914 (NOUVELLE) et 915 (EN\_COURS). Les deux anomalies TRAITÉE ont reçu leur date grâce au `dateResolution = NOW()` de l'UPDATE vu en Q37.

Pour remplacer un NULL à l'affichage, on utilise `COALESCE(annee, 'inconnue')` ou `IFNULL(annee, 'inconnue')` : la fonction renvoie la première valeur non nulle.

### Piège fréquent

Les fonctions d'agrégat ignorent les NULL : `COUNT(*)` compte toutes les lignes, mais `COUNT(annee)` ne compte que celles où `annee` est renseignée. Dans votre table, 3 véhicules sur 9 ont une `annee` à NULL, donc `COUNT(annee)` donnerait 6.

## Q47. Qu'est-ce qu'un alias (AS) et à quoi sert-il ?

AS donne un **nom temporaire** à une colonne ou à une table, uniquement pendant la requête. Rien n'est renommé dans la base.

### Alias de colonne

```
SELECT marque AS Marque,
       modele AS Modele,
       kilometrage / 1000 AS milliers_de_km
FROM vehicule
ORDER BY milliers_de_km DESC
LIMIT 3;
```

| Marque  | Modele | milliers_de_km |
|---------|--------|----------------|
| Iveco   | Daily  | 142.0000       |
| Citroen | Jumper | 65.0000        |
| Renault | Master | 38.0000        |

L'alias donne un nom lisible à une colonne calculée (`kilometrage / 1000`). En PHP, c'est ce nom qui devient la clé du tableau : `$row['milliers_de_km']`. Votre code s'en sert souvent, par exemple `SELECT v.idVehicule AS id` dans `garage/interventions.php`, ou `COUNT(DISTINCT v.idVehicule) AS vehicules_count` dans `admin/clients.php`.

### Alias de table

```
SELECT v.marque, a.niveau, a.statut
FROM anomalie AS a
JOIN vehicule AS v ON v.idVehicule = a.idVehicule;
```

|         |          |          |
|---------|----------|----------|
| marque  | niveau   | statut   |
| Iveco   | CRITIQUE | NOUVELLE |
| Citroen | MOYEN    | EN_COURS |
| Renault | MOYEN    | TRAITEE  |
| Peugeot | CRITIQUE | TRAITEE  |

Ici, `a` désigne `anomalie` et `v` désigne `vehicule`. C'est indispensable quand deux tables ont une colonne de même nom : `anomalie` et `vehicule` ont toutes deux `idVehicule`, et il faut préciser `a.idVehicule` ou `v.idVehicule`, sinon MySQL renvoie l'erreur « Column 'idVehicule' in field list is ambiguous ».

### À retenir

- Le mot `AS` est facultatif : `FROM anomalie a` est équivalent à `FROM anomalie AS a`.
- Un alias de colonne est utilisable dans `ORDER BY` (et `HAVING` dans MySQL), mais pas dans `WHERE`, qui est exécuté avant le `SELECT`.

## 3.5 TCL et DCL : transactions et droits

**Q48. Que sont une transaction (COMMIT, ROLLBACK) et les droits (GRANT, REVOKE) ? Donnez un exemple tiré de votre projet.**

### Les transactions (TCL)

Une **transaction** est un groupe de modifications traité comme un seul bloc : soit tout est enregistré, soit rien ne l'est. Quand un technicien clôture une réparation ( `technicien/reparations.php` ), trois choses doivent arriver ensemble : la réparation est ajoutée, l'intervention passe à `TERMINEE` et ses anomalies passent à `TRAITEE`. Si la deuxième étape échouait, on aurait une réparation enregistrée pour une intervention toujours « en cours ».

```
START TRANSACTION;

INSERT INTO reparation (idIntervention, idTechnicien, titre, description, cout, statut)
VALUES (925, 999020, 'Remplacement capteur', 'Capteur ABS remplacé', 120.00, 'TERMINEE');

UPDATE intervention SET statut = 'TERMINEE'
WHERE idIntervention = 925 AND idTechnicien = 999020;

UPDATE anomalie SET statut = 'TRAITEE', dateResolution = NOW()
WHERE idIntervention = 925 AND statut IN ('NOUVELLE', 'EN_COURS');

COMMIT;      -- tout est validé définitivement
-- ou ROLLBACK; -- tout est annulé, comme si rien ne s'était passé
```

- `START TRANSACTION` (ou `BEGIN`) : ouvre le bloc. Les modifications suivantes restent provisoires.
- `COMMIT` : valide et rend visibles toutes les modifications du bloc.
- `ROLLBACK` : annule toutes les modifications faites depuis `START TRANSACTION`.
- `SAVEPOINT nom` / `ROLLBACK TO nom` : point de retour intermédiaire, pour n'annuler qu'une partie.

En PHP, votre code utilise les méthodes équivalentes de PDO : `$conn->beginTransaction()`, puis `$conn->commit()`, et `$conn->rollback()` dans le bloc `catch` en cas d'erreur. On retrouve ce schéma dans

auth/register.php (création de l'utilisateur et de son profil client), admin/clients.php, admin/garages.php et admin/techniciens.php.

Sur votre serveur, la vérification donne :

```
SELECT @@autocommit AS ac, @@transaction_isolation AS iso;
```

```
ac | iso
1 | REPEATABLE-READ
```

autocommit = 1 signifie que, hors transaction, chaque commande est validée immédiatement.

REPEATABLE-READ est le niveau d'isolation par défaut d'InnoDB : pendant une transaction, une même lecture renvoie toujours le même résultat, même si une autre connexion modifie les données entre-temps.

Dans l'exemple, l'intervention 925 (actuellement EN\_COURS) et le technicien 999020 qui lui est affecté existent bien ; le titre, la description et le coût sont inventés. La colonne description doit obligatoirement figurer dans l'INSERT : elle est NOT NULL sans valeur par défaut, et votre serveur est en mode strict (STRICT\_TRANS\_TABLES), donc son absence provoquerait l'erreur 1364 « Field 'description' doesn't have a default value ». Le AND idTechnicien = ? reprend la requête réelle de technicien/reparations.php : un technicien ne peut clôturer que ses propres interventions. Ces trois commandes ont été vérifiées avec EXPLAIN uniquement, sans être exécutées.

## Les droits (DCL)

Le jury peut remarquer que l'application se connecte avec le compte root, qui a tous les droits (la requête SELECT CURRENT\_USER() renvoie root@localhost). En production, on crée un compte limité à ce dont l'application a besoin :

```
CREATE USER 'smartauto_app'@'localhost' IDENTIFIED BY 'MotDePasseSolide!2026';

GRANT SELECT, INSERT, UPDATE, DELETE
ON charles.*
TO 'smartauto_app'@'localhost';

REVOKE DELETE ON charles.* FROM 'smartauto_app'@'localhost';
```

- CREATE USER 'smartauto\_app'@'localhost' : crée un compte MySQL qui ne peut se connecter que depuis la machine locale.
- IDENTIFIED BY '...' : son mot de passe.
- GRANT SELECT, INSERT, UPDATE, DELETE : les droits accordés, ici uniquement les commandes DML et DQL. Le compte ne peut donc ni DROP ni ALTER.
- ON charles.\* : sur toutes les tables (\*) de la base charles.
- TO ... : le compte qui reçoit les droits.
- REVOKE DELETE ON charles.\* FROM ... : retire un droit déjà accordé. Après cette commande, le compte ne peut plus supprimer de lignes. **Attention** : on retire un droit au même niveau que celui où il a été donné ; un droit accordé sur charles.\* ne peut pas être retiré pour une seule table (MySQL répond « There is no such grant defined »).

### À retenir

- Transaction = « tout ou rien » : `COMMIT` valide, `ROLLBACK` annule. Elle ne fonctionne qu'avec le moteur InnoDB, celui de vos 19 tables.
- Les commandes DDL ( `CREATE`, `ALTER`, `DROP`, `TRUNCATE` ) valident automatiquement la transaction : elles ne s'annulent pas.
- Principe du moindre privilège : l'application ne devrait avoir que les droits dont elle a besoin, pas ceux de `root`.

## 4. Requêtes avancées : jointures, regroupements, sous-requêtes

Ce chapitre couvre les requêtes que le jury demande le plus souvent d'écrire au tableau : combiner plusieurs tables (jointures), compter et additionner (agrégats, `GROUP BY`), filtrer des groupes (`HAVING`), imbriquer une requête dans une autre (sous-requêtes), et manipuler des dates. Chaque requête de ce chapitre a été **réellement exécutée en lecture seule** sur la base `charles` le 02/10/2026 ; les résultats affichés sont ceux obtenus ce jour-là (ils changeront quand de nouvelles données seront saisies). Les noms, e-mails et téléphones ne sont jamais affichés : on montre des identifiants, des marques de véhicules ou des comptes.

Rappel des volumes au moment des tests : 9 véhicules, 10 interventions, 2 réparations, 4 anomalies, 1 garage enregistré, 8 techniciens, 5 clients, 15 utilisateurs.

### 4.1 Les jointures

#### Q50. Qu'est-ce qu'une jointure et pourquoi en avez-vous besoin dans votre base ?

Une **jointure** est une opération qui assemble, dans un même résultat, des lignes venant de deux tables (ou plus) en se servant d'une colonne commune. Dans votre base, les informations sont volontairement réparties dans plusieurs tables pour éviter les répétitions (c'est la normalisation) : la table `intervention` ne contient pas la marque du véhicule, elle contient seulement `idVehicule`, qui est une **clé étrangère** vers `vehicule.idVehicule` (contrainte `fk_intervention_vehicule`). Pour afficher « intervention n°925 sur une Peugeot 308 », il faut donc relier les deux tables : c'est le rôle de la jointure.

Les mots-clés :

- `FROM intervention i` : la table de départ. La lettre `i` est un **alias**, un surnom court qui évite de réécrire le nom complet de la table.
- `INNER JOIN vehicule v` : la deuxième table à associer, surnommée `v`.
- `ON v.idVehicule = i.idVehicule` : la **condition de jointure**. Elle dit « associe une intervention au véhicule dont l'identifiant est égal à celui noté dans l'intervention ». Presque toujours, on écrit ici « clé primaire = clé étrangère ».
- `i.type`, `v.marque` : on préfixe chaque colonne par l'alias pour dire de quelle table elle vient (obligatoire quand deux tables ont une colonne de même nom, par exemple `idVehicule` ou `statut`).

```
SELECT i.idIntervention, i.statut, v.marque, v.modele
FROM intervention i
INNER JOIN vehicule v ON v.idVehicule = i.idVehicule
WHERE i.statut = 'EN_COURS';
```

| idIntervention | statut   | marque  | modele |
|----------------|----------|---------|--------|
| 925            | EN_COURS | Peugeot | 308    |

(1 ligne)

Sans la clause `ON`, MySQL associerait chaque intervention à chaque véhicule ( $10 \times 9 = 90$  lignes) : c'est un **produit cartésien**, presque toujours une erreur.

### Q51. Quelle est la différence entre INNER JOIN et LEFT JOIN ? Montrez-la sur vos données.

**INNER JOIN** (ou simplement **JOIN**) ne garde que les lignes qui ont une correspondance **des deux côtés**. **LEFT JOIN** (ou **LEFT OUTER JOIN**) garde **toutes** les lignes de la table de gauche (celle écrite après **FROM**), même si elles n'ont aucune correspondance à droite ; dans ce cas, les colonnes de droite sont remplies avec **NULL** (valeur « absente »).

Exemple : on relie les 9 véhicules à leurs interventions.

```
SELECT COUNT(*) AS lignes_inner
FROM vehicule v
INNER JOIN intervention i ON i.idVehicule = v.idVehicule;

SELECT COUNT(*) AS lignes_left
FROM vehicule v
LEFT JOIN intervention i ON i.idVehicule = v.idVehicule;
```

```
lignes_inner = 10
lignes_left  = 14
```

Pourquoi 14 ? Voici le résultat détaillé du **LEFT JOIN** (colonnes non sensibles) :

```
SELECT v.idVehicule, v.marque, i.idIntervention, i.statut
FROM vehicule v
LEFT JOIN intervention i ON i.idVehicule = v.idVehicule
ORDER BY v.idVehicule, i.idIntervention;
```

| idVehicule | marque         | idIntervention | statut                             |
|------------|----------------|----------------|------------------------------------|
| 3          | Toyota Corolla | 4              | TERMINEE                           |
| 3          | Toyota Corolla | 5              | TERMINEE                           |
| 3          | Toyota Corolla | 932            | PLANIFIEE                          |
| 4          | Cadillac       | NULL           | NULL <-- disparaît avec INNER JOIN |
| 5          | Cadillac e     | NULL           | NULL <-- disparaît avec INNER JOIN |
| 916        | Renault        | 918            | TERMINEE                           |
| 917        | Peugeot        | NULL           | NULL <-- disparaît avec INNER JOIN |
| 918        | Citroen        | 917            | PLANIFIEE                          |
| 919        | Iveco          | 916            | PLANIFIEE                          |
| 920        | Mercedes       | NULL           | NULL <-- disparaît avec INNER JOIN |
| 922        | Peugeot        | 923            | PLANIFIEE                          |
| 922        | Peugeot        | 924            | PLANIFIEE                          |
| 922        | Peugeot        | 925            | EN_COURS                           |
| 922        | Peugeot        | 926            | TERMINEE                           |

(14 lignes)

Les 10 lignes qui ont une intervention sont communes aux deux requêtes. Les 4 véhicules sans intervention (4, 5, 917, 920) n'apparaissent qu'avec le **LEFT JOIN**, avec **NULL** à droite :  $10 + 4 = 14$ .

| Type       | Lignes gardées                                            | Sur vos données | Usage dans le projet                                                                                                    |
|------------|-----------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| INNER JOIN | Seulement les couples qui correspondent                   | 10              | Top techniciens ( <code>admin/statistics.php</code> ) : seuls les techniciens ayant au moins une intervention           |
| LEFT JOIN  | Toutes les lignes de gauche + correspondances éventuelles | 14              | Charge des garages ( <code>admin/dashboard.php</code> ) : un garage sans intervention doit quand même apparaître avec 0 |

Deuxième exemple réel : la table `technicien` compte 8 lignes. Avec `INNER JOIN intervention` regroupé par technicien, la requête « top techniciens » de `admin/statistics.php` ne renvoie que **4** techniciens (ceux qui ont au moins une intervention : 2, 2, 2 et 1 interventions). Avec un `LEFT JOIN`, on obtient les **8** techniciens, dont 4 avec 0 intervention.

### À retenir

- `INNER JOIN` = intersection : une ligne sans partenaire disparaît.
- `LEFT JOIN` = tout le côté gauche est conservé ; ce qui manque à droite vaut `NULL`.
- Il existe aussi `RIGHT JOIN` (symétrique, on garde tout le côté droit) ; on l'utilise rarement car on peut toujours inverser l'ordre des tables et écrire un `LEFT JOIN`. MySQL ne propose pas `FULL OUTER JOIN`.

## Q52. Comment trouver les véhicules qui n'ont jamais eu aucune intervention ?

C'est une question classique de « recherche d'absence ». Il y a trois façons de l'écrire ; les trois donnent le même résultat sur votre base.

### Méthode 1 : LEFT JOIN + IS NULL

```
SELECT v.idVehicule, v.marque, v.modele
FROM vehicule v
LEFT JOIN intervention i ON i.idVehicule = v.idVehicule
WHERE i.idIntervention IS NULL;
```

On garde tous les véhicules (`LEFT JOIN`), puis `WHERE i.idIntervention IS NULL` ne retient que les lignes où aucune intervention n'a été trouvée. On teste une colonne de droite qui ne peut jamais être `NULL` quand une correspondance existe : ici la clé primaire `idIntervention`. Attention : on écrit `IS NULL` et non `= NULL` (voir le piège ci-dessous).

### Méthode 2 : NOT EXISTS

```
SELECT v.idVehicule, v.marque, v.modele
FROM vehicule v
WHERE NOT EXISTS (
    SELECT 1 FROM intervention i
    WHERE i.idVehicule = v.idVehicule
);
```

Pour chaque véhicule, MySQL vérifie s'il existe au moins une intervention qui le concerne ; `NOT EXISTS` garde le véhicule s'il n'en existe aucune. C'est la forme la plus lisible et la plus sûre.

### Méthode 3 : NOT IN

```
SELECT v.idVehicule, v.marque, v.modele
FROM vehicule v
WHERE v.idVehicule NOT IN (SELECT i.idVehicule FROM intervention i);
```

| idVehicule | marque     | modele   |
|------------|------------|----------|
| 4          | Cadillac   | escalade |
| 5          | Cadillac e | escalade |
| 917        | Peugeot    | Boxer    |
| 920        | Mercedes   | Sprinter |

(4 lignes, identiques pour les 3 méthodes)

### Piège fréquent

`NOT IN` ne peut plus être vrai pour **aucune** ligne si la sous-requête renvoie un seul `NULL` : la comparaison « x différent de `NULL` » ne vaut ni vrai ni faux mais « inconnu » (`NULL`), et `WHERE` ne garde que les lignes où la condition est vraie. Le résultat est alors vide. Ici c'est sans danger car `intervention.idVehicule` est déclaré `NOT NULL`. Mais si vous écriviez « techniciens qui n'ont aucune intervention » avec `NOT IN (SELECT idTechnicien FROM intervention)`, la colonne `idTechnicien` peut être `NULL` (3 interventions n'ont pas de technicien) et la requête renvoie 0 ligne. Test réel : `NOT IN` donne 0 technicien, alors que `NOT EXISTS (SELECT 1 FROM intervention i WHERE i.idTechnicien = t.idTechnicien)` en trouve correctement 4. Préférez `NOT EXISTS`. De même, `WHERE colonne = NULL` ne trouve jamais rien : on écrit toujours `IS NULL / IS NOT NULL`.

### Q53. Écrivez une requête qui affiche une intervention avec son véhicule, son client, son garage et son technicien (4 à 5 tables).

On enchaîne plusieurs jointures : chaque `JOIN ... ON ...` ajoute une table au résultat précédent. Le choix entre `INNER` et `LEFT` se fait **colonne par colonne en regardant le schéma** :

- `intervention.idVehicule` et `intervention.idClient` sont `NOT NULL` : toute intervention a un véhicule et un client, donc `INNER JOIN` ne fait rien disparaître.
- `intervention.idGarage` et `intervention.idTechnicien` sont `DEFAULT NULL` : une demande peut ne pas encore être affectée. Il faut un `LEFT JOIN`, sinon ces interventions disparaissent.

```
SELECT i.idIntervention, i.type, i.statut, i.priorite,
       DATE_FORMAT(i.dateIntervention, '%d/%m/%Y') AS date_fr,
       v.marque, v.modele,
       uc.nom AS client_nom,      -- (non affiché ici : donnée personnelle)
       g.idGarage,
       g.nomGarage,              -- (non affiché ici)
       ut.nom AS technicien_nom  -- (non affiché ici)
FROM intervention i
INNER JOIN vehicule v      ON v.idVehicule      = i.idVehicule
INNER JOIN utilisateur uc  ON uc.idUtilisateur = i.idClient
LEFT JOIN garage g         ON g.idGarage        = i.idGarage
LEFT JOIN utilisateur ut   ON ut.idUtilisateur = i.idTechnicien
WHERE i.statut = 'EN_COURS';
```

| idIntervention | type                    | statut   | priorite | date_fr    | marque  | modele | idGarage |
|----------------|-------------------------|----------|----------|------------|---------|--------|----------|
| 925            | Diagnostic électronique | EN_COURS | HAUTE    | 23/09/2026 | Peugeot | 308    | 3        |

(1 ligne ; colonnes `client_nom`, `nomGarage` et `technicien_nom` masquées)

Remarquez que la table `utilisateur` est jointe **deux fois** avec deux alias différents : `uc` pour le client et `ut` pour le technicien. C'est possible car le client (`idClient`) et le technicien (`idTechnicien`) sont tous

deux des utilisateurs (héritage : `client.idClient` et `technicien.idTechnicien` sont des clés étrangères vers `utilisateur.idUtilisateur`).

Démonstration de l'importance du `LEFT JOIN`, sur toutes les interventions (sans le `WHERE`) :

Avec `INNER JOIN` garage + `INNER JOIN` technicien : 3 lignes  
Avec `LEFT JOIN` garage + `LEFT JOIN` technicien : 10 lignes (toutes les interventions)

Votre code utilise exactement ce schéma dans `admin/dashboard.php`, bloc « demandes en attente d'affectation » :

```
SELECT i.idIntervention AS id, i.type, i.dateIntervention, i.statut,
       v.marque, v.modele, v.immatriculation,
       uc.prenom AS client_prenom, uc.nom AS client_nom
FROM intervention i
JOIN vehicule v      ON v.idVehicule = i.idVehicule
JOIN utilisateur uc  ON uc.idUtilisateur = i.idClient
WHERE i.idTechnicien IS NULL AND (i.idGarage IS NULL OR i.statut = 'ANNULEE')
ORDER BY i.dateIntervention ASC
LIMIT 5;
```

Ligne par ligne : on prend les interventions ( `FROM` ), on ajoute le véhicule et le client (deux `JOIN` sûrs car colonnes `NOT NULL`), on ne garde que celles sans technicien et soit sans garage, soit refusées ( `WHERE` ; les parenthèses sont indispensables car `AND` est évalué avant `OR` ), on trie de la plus ancienne à la plus récente ( `ORDER BY ... ASC` ) et on n'affiche que les 5 premières ( `LIMIT 5` ). Le compteur associé ( `SELECT COUNT(*) ...` avec le même `WHERE` ) renvoie aujourd'hui **1**.

## 4.2 Fonctions d'agrégat et regroupements

### Q54. Que font `COUNT`, `SUM`, `AVG`, `MIN` et `MAX` ? Quelle différence entre `COUNT(*)` et `COUNT(colonne)` ?

Une **fonction d'agrégat** prend un ensemble de lignes et le résume en une seule valeur :

| Fonction                                      | Rôle                                                      | Ignore les NULL ?                                       |
|-----------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------|
| <code>COUNT(*)</code>                         | Nombre de lignes                                          | Non, compte toutes les lignes                           |
| <code>COUNT(col)</code>                       | Nombre de valeurs non nulles de <code>col</code>          | Oui                                                     |
| <code>COUNT(DISTINCT col)</code>              | Nombre de valeurs différentes                             | Oui                                                     |
| <code>SUM(col)</code>                         | Somme                                                     | Oui ; renvoie <code>NULL</code> s'il n'y a aucune ligne |
| <code>AVG(col)</code>                         | Moyenne (somme / nombre de valeurs non nulles)            | Oui                                                     |
| <code>MIN(col)</code> / <code>MAX(col)</code> | Plus petite / plus grande valeur (nombres, dates, textes) | Oui                                                     |

Différence `COUNT(*)` / `COUNT(colonne)` sur votre table `intervention` :

```
SELECT COUNT(*)          AS nb_interventions,
       COUNT(idTechnicien) AS avec_technicien,
       COUNT(idGarage)    AS avec_garage
FROM intervention;
```

|                  |  |                 |  |             |
|------------------|--|-----------------|--|-------------|
| nb_interventions |  | avec_technicien |  | avec_garage |
| 10               |  | 7               |  | 5           |

Il y a 10 interventions ; 3 n'ont pas de technicien et 5 n'ont pas de garage (valeurs `NULL`), donc `COUNT(colonne)` les ignore.

Requête réelle de `admin/statistics.php` sur les réparations :

```
SELECT COUNT(*) AS total_reparations,
       SUM(cout) AS total_cout_reparations,
       AVG(cout) AS cout_moyen_reparations
FROM reparation;
```

|                   |  |                        |  |                        |
|-------------------|--|------------------------|--|------------------------|
| total_reparations |  | total_cout_reparations |  | cout_moyen_reparations |
| 2                 |  | 265.00                 |  | 132.500000             |

Avec `MIN`, `MAX` et un regroupement par statut :

```
SELECT statut, COUNT(*) AS nb, SUM(cout) AS total,
       MIN(cout) AS mini, MAX(cout) AS maxi,
       AVG(dureeIntervention) AS duree_moy
FROM reparation
GROUP BY statut;
```

|          |  |    |  |        |  |        |  |        |  |           |
|----------|--|----|--|--------|--|--------|--|--------|--|-----------|
| statut   |  | nb |  | total  |  | mini   |  | maxi   |  | duree_moy |
| TERMINEE |  | 2  |  | 265.00 |  | 120.00 |  | 145.00 |  | 0.750000  |

Sur les dates : `SELECT MIN(dateIntervention), MAX(dateIntervention) FROM intervention;` donne la première (2025-09-25 20:13:00) et la dernière (2026-09-29 11:01:49) intervention enregistrées.

### Q55. Combien d'interventions y a-t-il par statut ? Expliquez `GROUP BY`.

`GROUP BY colonne` range les lignes en **paquets** qui ont la même valeur dans cette colonne, puis applique la fonction d'agrégat à chaque paquet séparément. On obtient une ligne de résultat par paquet.

```
SELECT statut, COUNT(*) AS nombre
FROM intervention
GROUP BY statut
ORDER BY nombre DESC;
```

|           |  |        |
|-----------|--|--------|
| statut    |  | nombre |
| PLANIFIEE |  | 5      |
| TERMINEE  |  | 4      |
| EN_COURS  |  | 1      |

(3 lignes)

- `SELECT statut, COUNT(*)` : on affiche la valeur du groupe et le nombre de lignes du groupe.
- `GROUP BY statut` : un groupe par valeur de l' `ENUM` (`PLANIFIEE`, `EN_COURS`, `TERMINEE`, `ANNULEE`).
- `ORDER BY nombre DESC` : tri décroissant sur l'alias calculé.

Remarque : le statut `ANNULEE` n'apparaît pas, car aucune intervention n'est annulée aujourd'hui. `GROUP BY` ne crée jamais de groupe vide. C'est pour cela que `admin/dashboard.php` complète en PHP les jours sans intervention à 0 pour la courbe des 7 derniers jours.

On peut regrouper sur plusieurs colonnes : `GROUP BY statut, niveau` sur `anomalie` donne une ligne par combinaison (NOUVELLE/CRITIQUE : 1, EN\_COURS/MOYEN : 1, TRAITEE/MOYEN : 1, TRAITEE/CRITIQUE : 1).

### Piège fréquent

Votre serveur MySQL 8 fonctionne avec le mode `ONLY_FULL_GROUP_BY` (vérifié avec `SELECT @@sql_mode`). Toute colonne du `SELECT` doit alors être soit dans le `GROUP BY`, soit dans une fonction d'agrégat. La requête `SELECT statut, type, COUNT(*) FROM intervention GROUP BY statut` est refusée avec l'erreur 1055 (« Expression #2 of SELECT list is not in GROUP BY clause... ») : MySQL ne sait pas quel `type` afficher pour un groupe qui en contient plusieurs. Exception : si on regroupe par la clé primaire (ex. `GROUP BY u.idUtilisateur` dans `statistics.php`), on peut afficher `u.nom`, car le nom dépend fonctionnellement de la clé.

## Q56. Quel garage a le plus d'interventions ?

```
SELECT g.idGarage, g.nomGarage, COUNT(i.idIntervention) AS nb_interventions
FROM garage g
LEFT JOIN intervention i ON i.idGarage = g.idGarage
GROUP BY g.idGarage, g.nomGarage
ORDER BY nb_interventions DESC
LIMIT 1;
```

| idGarage | nomGarage | nb_interventions |
|----------|-----------|------------------|
| 3        | (masqué)  | 5                |

- `LEFT JOIN` : on garde tous les garages, même ceux sans intervention (ils auraient 0).
- `COUNT(i.idIntervention)` : on compte les interventions réellement trouvées, pas les lignes.
- `GROUP BY g.idGarage, g.nomGarage` : une ligne par garage. On met l'identifiant pour ne pas fusionner deux garages qui porteraient le même nom.
- `ORDER BY ... DESC LIMIT 1` : on trie du plus grand au plus petit et on ne garde que le premier.

Aujourd'hui la base ne contient qu'un garage (n°3, statut `VALIDE`) ; il a 5 interventions, dont 4 actives (`PLANIFIEE` ou `EN_COURS`).

### Piège fréquent

Avec un `LEFT JOIN`, `COUNT(*)` donnerait **1** à un garage sans intervention, car la ligne « garage + NULL » existe bel et bien. `COUNT(i.idIntervention)` donne correctement 0, puisque `COUNT(colonne)` ignore les `NULL`. Autre limite : si deux garages sont à égalité, `LIMIT 1` n'en affiche qu'un seul, choisi arbitrairement.

### Q57. Quelle est la différence entre WHERE et HAVING ?

WHERE filtre les **lignes avant** le regroupement ; HAVING filtre les **groupes après** le regroupement. Conséquence : on ne peut pas mettre une fonction d'agrégat ( COUNT , SUM ...) dans un WHERE , car au moment où WHERE s'applique, les groupes n'existent pas encore.

Question : « quels véhicules ont au moins 2 interventions non annulées ? »

```
SELECT idVehicule, COUNT(*) AS nb
FROM intervention
WHERE statut <> 'ANNULEE'      -- 1) filtre sur chaque ligne
GROUP BY idVehicule           -- 2) un groupe par véhicule
HAVING COUNT(*) >= 2;         -- 3) filtre sur chaque groupe
```

```
idVehicule | nb
3           | 3
922        | 4
(2 lignes)
```

Si l'on met la condition sur le nombre dans le WHERE , MySQL refuse (test réel) :

```
SELECT idVehicule, COUNT(*) FROM intervention
WHERE COUNT(*) > 1
GROUP BY idVehicule;
```

ERREUR 1111 : Invalid use of group function

|                    | WHERE                                | HAVING                                    |
|--------------------|--------------------------------------|-------------------------------------------|
| Moment             | Avant GROUP BY                       | Après GROUP BY                            |
| Filtre             | Des lignes                           | Des groupes                               |
| Agrégats autorisés | Non                                  | Oui ( HAVING COUNT(*) >= 2 )              |
| Alias du SELECT    | Non (erreur 1054 « Unknown column ») | Oui en MySQL ( HAVING nb >= 2 fonctionne) |

Le commentaire de votre fichier includes/activity\_log.php le rappelle : « un alias calculé n'est pas utilisable dans WHERE ». C'est pour cela que le filtre par rôle y est réécrit avec des EXISTS .

**Ordre logique d'exécution** d'une requête, à connaître par cœur : FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT . Cet ordre explique toutes les règles ci-dessus : WHERE passe avant le calcul des alias et des groupes, ORDER BY passe après et peut donc utiliser les alias.

### Q58. Avec un LEFT JOIN, est-ce la même chose de mettre une condition dans ON ou dans WHERE ?

Non, et c'est une question piège très fréquente. Une condition dans ON sert à **choisir les partenaires** à droite, sans supprimer de lignes à gauche. Une condition dans WHERE s'applique **après** la jointure et supprime les lignes où elle est fausse, y compris celles complétées par NULL . Le LEFT JOIN redevient alors, en pratique, un INNER JOIN .

```
-- Condition dans ON : tous les véhicules, avec 0 si aucune intervention terminée
SELECT v.idVehicule, COUNT(i.idIntervention) AS terminees
FROM vehicule v
LEFT JOIN intervention i ON i.idVehicule = v.idVehicule AND i.statut = 'TERMINEE'
GROUP BY v.idVehicule;
```

9 lignes : véhicule 3 → 2, 916 → 1, 922 → 1, et les 6 autres → 0

```
-- Condition dans WHERE : les véhicules à 0 disparaissent
SELECT v.idVehicule, COUNT(i.idIntervention) AS terminees
FROM vehicule v
LEFT JOIN intervention i ON i.idVehicule = v.idVehicule
WHERE i.statut = 'TERMINEE'
GROUP BY v.idVehicule;
```

3 lignes : véhicule 3 → 2, 916 → 1, 922 → 1

Pour les lignes sans intervention, `i.statut` vaut `NULL` ; `NULL = 'TERMINEE'` n'est pas vrai, donc le `WHERE` les élimine.

### Q59. Quel est le chiffre d'affaires des réparations terminées ? Et par garage ?

Le montant d'une réparation est stocké dans `reparation.cout`, de type `DECIMAL(10,2)` : un nombre exact à 2 décimales, adapté à l'argent (contrairement à `FLOAT`, qui fait des erreurs d'arrondi).

```
SELECT COALESCE(SUM(cout), 0) AS chiffre_affaires
FROM reparation
WHERE statut = 'TERMINEE';
```

```
chiffre_affaires
265.00
```

- `WHERE statut = 'TERMINEE'` : on ne garde que les réparations clôturées.
- `SUM(cout)` : on additionne leurs coûts (145,00 + 120,00).
- `COALESCE(x, 0)` : renvoie le premier argument non nul. Si aucune réparation n'est terminée, `SUM` renvoie `NULL` ; `COALESCE` le remplace par 0 pour afficher « 0 » plutôt que rien.

Par garage, il faut remonter de la réparation vers l'intervention, qui porte `idGarage` (3 tables) :

```
SELECT g.idGarage,
       COUNT(DISTINCT i.idIntervention) AS nb_interventions,
       COALESCE(SUM(r.cout), 0) AS ca
FROM garage g
LEFT JOIN intervention i ON i.idGarage = g.idGarage
LEFT JOIN reparation r   ON r.idIntervention = i.idIntervention
                        AND r.statut = 'TERMINEE'
GROUP BY g.idGarage
ORDER BY ca DESC;
```

```
idGarage | nb_interventions | ca
3         | 5                 | 120.00
```

COUNT(DISTINCT ...) évite de compter deux fois une intervention qui aurait plusieurs réparations (la relation intervention → réparation est de type 1 à plusieurs). Le garage 3 n'a que 120,00 car l'autre réparation terminée (145,00) concerne l'intervention 918, qui n'a pas de garage ( idGarage à NULL ).

### Piège fréquent

Le jury peut demander pourquoi vous n'utilisez pas la table paiement . Elle existe (colonne montant , statut PAYE ...) mais elle est vide aujourd'hui (0 ligne) : le « chiffre d'affaires » calculé ici est le **montant facturable** des réparations, pas l'argent réellement encaissé. Pour l'encaissé, il faudrait  
`SELECT SUM(montant) FROM paiement WHERE statut = 'PAYE' .`

## 4.3 CASE WHEN et compteurs du tableau de bord

### Q60. À quoi sert CASE WHEN ? Expliquez la requête de statistiques des interventions de admin/statistics.php.

CASE est le « si... alors... sinon » du SQL. Il calcule une valeur différente selon une condition, ligne par ligne.

```
CASE
  WHEN condition1 THEN valeur1
  WHEN condition2 THEN valeur2
  ELSE valeur_par_defaut
END
```

MySQL teste les WHEN dans l'ordre et s'arrête au premier qui est vrai. Sans ELSE , la valeur par défaut est NULL .

#### Usage 1 : compter plusieurs catégories en une seule requête

```
SELECT
  COUNT(*) AS total_interventions,
  SUM(CASE WHEN statut = 'PLANIFIEE' THEN 1 ELSE 0 END) AS interventions_planifiees,
  SUM(CASE WHEN statut = 'EN_COURS' THEN 1 ELSE 0 END) AS interventions_en_cours,
  SUM(CASE WHEN statut = 'TERMINEE' THEN 1 ELSE 0 END) AS interventions_terminees
FROM intervention;
```

| total_interventions | interventions_planifiees | interventions_en_cours | interventions_terminees |
|---------------------|--------------------------|------------------------|-------------------------|
| 10                  | 5                        | 1                      | 4                       |

Explication : pour chaque intervention, le CASE vaut 1 si le statut correspond, 0 sinon ; SUM additionne ces 1, ce qui revient à compter. L'avantage par rapport à 3 requêtes COUNT(\*) ... WHERE statut = ... est qu'on lit la table **une seule fois**. admin/dashboard.php applique la même technique aux clients (typeClient : 4 PARTICULIER , 1 ENTREPRISE ) et à la charge des garages.

#### Usage 2 : COUNT(CASE ...) sans ELSE

Dans le « top techniciens » de statistics.php : COUNT(CASE WHEN i.statut = 'TERMINEE' THEN 1 END) . Sans ELSE , le CASE vaut NULL pour les autres statuts, et COUNT ignore les NULL : on compte donc uniquement les terminées. Résultat réel (noms masqués) : 2/2, 1/2, 1/2 et 0/1 interventions terminées.

### Usage 3 : transformer une valeur en score

```
SELECT type, COUNT(*) AS count,
       AVG(CASE WHEN niveau = 'CRITIQUE' THEN 3
                WHEN niveau = 'MOYEN'      THEN 2
                ELSE 1 END) AS severity_avg
FROM anomalie
GROUP BY type
ORDER BY count DESC;
```

| type         | count | severity_avg |
|--------------|-------|--------------|
| Freinage     | 2     | 2.5000       |
| Moteur       | 1     | 3.0000       |
| Pneumatiques | 1     | 2.0000       |

Le niveau (ENUM texte) est converti en nombre (3, 2, 1) pour pouvoir en faire une moyenne de gravité par type d'anomalie.

### Usage 4 : afficher un libellé

```
SELECT idIntervention, statut,
       CASE WHEN statut = 'TERMINEE' THEN 'Clôturée'
            WHEN statut IN ('PLANIFIEE', 'EN_COURS') THEN 'Active'
            ELSE 'Annulée' END AS etat
FROM intervention
WHERE idIntervention IN (4, 923, 925);
```

| idIntervention | statut    | etat     |
|----------------|-----------|----------|
| 4              | TERMINEE  | Clôturée |
| 923            | PLANIFIEE | Active   |
| 925            | EN_COURS  | Active   |

## 4.4 Sous-requêtes

### Q61. Qu'est-ce qu'une sous-requête ? Donnez des exemples dans WHERE, avec IN et dans le SELECT.

Une **sous-requête** est un `SELECT` écrit entre parenthèses à l'intérieur d'une autre requête. MySQL calcule son résultat et l'utilise dans la requête principale.

#### a) Sous-requête qui renvoie une seule valeur (scalaire), dans WHERE

« Quelles réparations coûtent plus que la moyenne ? »

```
SELECT idReparation, cout
FROM reparation
WHERE cout > (SELECT AVG(cout) FROM reparation);
```

| idReparation | cout   |
|--------------|--------|
| 913          | 145.00 |

(la moyenne vaut 132.50)

On ne peut pas écrire `WHERE cout > AVG(cout)` (agrégat interdit dans `WHERE`, voir Q57) : la sous-requête calcule d'abord la moyenne, qui est ensuite comparée à chaque ligne.

## b) IN avec une liste de valeurs : le compteur d'anomalies actives

Requête réelle de `admin/dashboard.php` :

```
SELECT COUNT(*) FROM anomalie WHERE statut IN ('NOUVELLE', 'EN_COURS');
```

2

`IN (a, b)` est un raccourci pour `statut = 'NOUVELLE' OR statut = 'EN_COURS'`. Une anomalie est dite « active » tant qu'elle n'est ni `TRAITEE` ni `IGNOREE`. On peut compléter : `SUM(niveau = 'CRITIQUE')` donne 1 anomalie critique parmi ces 2 (en MySQL, une comparaison vaut 1 si vraie, 0 si fausse).

## c) IN avec une sous-requête

« Quels véhicules ont au moins une anomalie critique ? »

```
SELECT v.idVehicule, v.marque
FROM vehicule v
WHERE v.idVehicule IN (SELECT a.idVehicule FROM anomalie a WHERE a.niveau = 'CRITIQUE');
```

| idVehicule | marque  |
|------------|---------|
| 919        | Iveco   |
| 922        | Peugeot |

## d) Sous-requêtes dans le SELECT : les totaux de statistics.php

```
SELECT
  (SELECT COUNT(*) FROM utilisateur) AS total_users,
  (SELECT COUNT(*) FROM client) AS clients,
  (SELECT COUNT(*) FROM technicien) AS techniciens,
  (SELECT COUNT(*) FROM administrateur) AS admins;
```

| total_users | clients | techniciens | admins |
|-------------|---------|-------------|--------|
| 15          | 5       | 8           | 1      |

Quatre petites requêtes indépendantes sont regroupées en une seule ligne de résultat : un seul aller-retour entre PHP et MySQL.

## e) Sous-requête corrélée

Une sous-requête est **corrélée** quand elle utilise une colonne de la requête extérieure ; elle est alors réévaluée pour chaque ligne.

```
SELECT v.idVehicule, v.marque,
  (SELECT COUNT(*) FROM intervention i WHERE i.idVehicule = v.idVehicule) AS
  nb_interventions
FROM vehicule v
ORDER BY nb_interventions DESC;
```

|               |                    |               |                |             |
|---------------|--------------------|---------------|----------------|-------------|
| 922 Peugeot 4 | 3 Toyota Corolla 3 | 916 Renault 1 | 918 Citroen 1  | 919 Iveco 1 |
| 4 Cadillac 0  | 5 Cadillac e 0     | 917 Peugeot 0 | 920 Mercedes 0 | (9 lignes)  |

## Q62. Que font EXISTS et NOT EXISTS ? Où les utilisez-vous ?

EXISTS (sous-requête) est vrai si la sous-requête renvoie **au moins une ligne**, faux sinon. Il ne regarde pas le contenu des colonnes, d'où l'habitude d'écrire `SELECT 1`. MySQL peut s'arrêter dès la première ligne trouvée. NOT EXISTS est l'inverse : vrai si la sous-requête ne renvoie rien.

Exemple réel : le filtre « véhicules avec anomalie active » de `client/vehicules.php` :

```
SELECT COUNT(*) AS vehicules_avec_anomalie_active
FROM vehicule v
WHERE EXISTS (
    SELECT 1 FROM anomalie a
    WHERE a.idVehicule = v.idVehicule
    AND a.statut IN ('NOUVELLE', 'EN_COURS')
);
```

```
vehicules_avec_anomalie_active
2
```

Le filtre opposé (« véhicules sans anomalie active ») est la même sous-requête précédée de NOT EXISTS.

Autre usage réel, pour la **sécurité** : dans `garage/vehicule.php` et `garage/anomalies.php`, un garage ne voit un véhicule que s'il existe une intervention de ce véhicule chez lui :

```
... WHERE v.idVehicule = ?
    AND EXISTS (SELECT 1 FROM intervention i
                WHERE i.idVehicule = v.idVehicule AND i.idGarage = ?)
```

Les `?` sont des paramètres de requête préparée (`$conn->prepare(...)` puis `execute([...])`) : la valeur est envoyée séparément du texte SQL, ce qui empêche l'injection SQL.

Exemple métier : « interventions terminées pour lesquelles aucune fiche de réparation n'a été saisie ».

```
SELECT COUNT(*) AS interventions_terminees_sans_reparation
FROM intervention i
WHERE i.statut = 'TERMINEE'
    AND NOT EXISTS (SELECT 1 FROM reparation r WHERE r.idIntervention = i.idIntervention);
```

```
2
```

### À retenir

- IN compare une valeur à une liste ; EXISTS teste seulement « y a-t-il au moins une ligne ? ».
- Pour « ceux qui n'ont pas de ... », préférez NOT EXISTS à NOT IN (problème des NULL, Q52).
- EXISTS ne multiplie pas les lignes, contrairement à une jointure : un véhicule avec 2 anomalies actives n'est compté qu'une fois.

## 4.5 UNION et UNION ALL

### Q63. Quelle est la différence entre UNION et UNION ALL ?

Les deux **empilent** verticalement les résultats de plusieurs `SELECT` (alors qu'une jointure les met côte à côte). Les `SELECT` doivent avoir le même nombre de colonnes, de types compatibles ; les noms des colonnes du résultat sont ceux du premier `SELECT`.

- `UNION` supprime les doublons (MySQL doit trier ou comparer toutes les lignes : c'est plus coûteux).
- `UNION ALL` garde tout, doublons compris, sans travail supplémentaire : c'est plus rapide.

Exemple : la liste des véhicules concernés par une intervention ou par une anomalie.

```
SELECT idVehicule FROM intervention
UNION
SELECT idVehicule FROM anomalie;
```

idVehicule : 3, 916, 918, 919, 922 (5 lignes, chaque véhicule une seule fois)

Même requête avec `UNION ALL` : 14 lignes (10 interventions + 4 anomalies, doublons gardés)

Exemple utile avec `UNION ALL` : un tableau récapitulatif des rôles.

```
SELECT 'client' AS role, COUNT(*) AS nb FROM client
UNION ALL SELECT 'technicien', COUNT(*) FROM technicien
UNION ALL SELECT 'garage', COUNT(*) FROM garage WHERE idUtilisateur IS NOT NULL
UNION ALL SELECT 'admin', COUNT(*) FROM administrateur;
```

| role       | nb |
|------------|----|
| client     | 5  |
| technicien | 8  |
| garage     | 1  |
| admin      | 1  |

### Q64. Expliquez ligne par ligne la requête de `getUserRole()` dans `config/roles.php`. Pourquoi `UNION ALL` ?

Dans votre modèle, un utilisateur n'a pas de colonne « rôle ». Le rôle est déduit de la table de spécialisation dans laquelle il apparaît (héritage : `administrateur`, `client`, `technicien` partagent la clé `idUtilisateur` ; un compte garage est relié par `garage.idUtilisateur`).

```
SELECT 'admin' AS role FROM administrateur WHERE idAdministrateur = ?
UNION ALL
SELECT 'client' FROM client WHERE idClient = ?
UNION ALL
SELECT 'technicien' FROM technicien WHERE idTechnicien = ?
UNION ALL
SELECT 'garage' FROM garage WHERE idUtilisateur = ?
LIMIT 1
```

1. `SELECT 'admin' AS role FROM administrateur WHERE idAdministrateur = ?` : si l'utilisateur est dans la table `administrateur`, cette partie renvoie une ligne contenant le texte constant `'admin'` ; sinon, elle ne renvoie rien. `AS role` nomme la colonne du résultat global.

2. Les trois `SELECT` suivants font le même test dans `client`, `technicien` et `garage` ; la recherche est rapide car elle porte sur une clé primaire ou une clé `UNIQUE`.
3. `UNION ALL` empile les résultats : on obtient 0 ou 1 ligne par table.
4. `LIMIT 1` : on garde la première ligne trouvée.
5. Les quatre `?` reçoivent le même identifiant via `$stmt->execute([$id, $id, $id, $id])` ; `fetchColumn()` lit la valeur ou renvoie `false` si aucun rôle n'est trouvé (la fonction retourne alors `null`).

Test réel avec l'identifiant de l'administrateur (sans afficher l'identifiant) :

```
role
admin
```

**Pourquoi `UNION ALL` et pas `UNION` ?** Les quatre branches renvoient des textes différents ( `'admin'`, `'client'` ...), il ne peut donc jamais y avoir de doublon à supprimer. `UNION` ferait un travail de dédoublonnage inutile ; `UNION ALL` donne le même résultat, plus vite. De plus, une seule requête remplace quatre allers-retours PHP/MySQL.

J'ai vérifié qu'aucun utilisateur n'a plusieurs rôles (requête de contrôle avec 4 `EXISTS` : 0 utilisateur multi-rôle) : `LIMIT 1` ne masque donc aucun cas ambigu aujourd'hui.

### Piège fréquent

« Et si un utilisateur était à la fois client et technicien ? » Réponse honnête : `LIMIT 1` sans `ORDER BY` renverrait l'une des deux lignes. En pratique MySQL lit les branches dans l'ordre, ce qui donne une priorité admin, puis client, puis technicien, puis garage, mais la norme SQL ne garantit pas cet ordre. Pour le garantir, on pourrait ajouter une colonne de priorité dans chaque branche et un `ORDER BY` `priorite`. C'est l'application (inscription) qui empêche qu'un compte ait deux rôles.

## 4.6 Fonctions de date

**Q65. Quelles fonctions de date utilisez-vous ? Expliquez `NOW()`, `CURDATE()`, `DATE_FORMAT`, `DATEDIFF` et `INTERVAL`.**

| Fonction                                          | Rôle                                            | Résultat le 02/10/2026 |
|---------------------------------------------------|-------------------------------------------------|------------------------|
| <code>NOW()</code>                                | Date et heure actuelles du serveur              | 2026-10-02 15:37:23    |
| <code>CURDATE()</code>                            | Date du jour, sans l'heure                      | 2026-10-02             |
| <code>DATE_SUB(CURDATE(), INTERVAL 6 DAY)</code>  | Retire une durée                                | 2026-09-26             |
| <code>DATE_ADD(NOW(), INTERVAL 1 MONTH)</code>    | Ajoute une durée                                | 2026-11-02 15:37:23    |
| <code>DATEDIFF('2026-10-02', '2026-09-23')</code> | Nombre de jours entre deux dates (1re moins 2e) | 9                      |
| <code>DATE_FORMAT(d, '%d/%m/%Y à %Hh%i')</code>   | Met en forme une date en texte                  | 23/09/2026 à 14h05     |

| Fonction                     | Rôle                              | Résultat le 02/10/2026                   |
|------------------------------|-----------------------------------|------------------------------------------|
| DATE(d) , YEAR(d) , MONTH(d) | Extrait la date, l'année, le mois | YEAR(NOW()) = 2026,<br>MONTH(NOW()) = 10 |

Codes de DATE\_FORMAT : %Y année sur 4 chiffres, %m mois sur 2 chiffres, %d jour, %H heure sur 2 chiffres (00 à 23), %i minutes. INTERVAL n UNITE désigne une durée (DAY, MONTH, YEAR, HOUR ...); on peut aussi écrire NOW() - INTERVAL 7 DAY.

### Exemple réel 1 : la courbe des 7 derniers jours (admin/dashboard.php)

```
SELECT DATE(dateIntervention) AS jour, COUNT(*) AS total
FROM intervention
WHERE dateIntervention >= DATE_SUB(CURDATE(), INTERVAL 6 DAY)
GROUP BY DATE(dateIntervention);
```

| jour       | total |
|------------|-------|
| 2026-09-29 | 1     |
| 2026-09-28 | 1     |

DATE(...) enlève l'heure pour regrouper toutes les interventions d'une même journée. CURDATE() moins 6 jours plus aujourd'hui donne 7 jours. Les jours sans intervention n'apparaissent pas (pas de groupe vide) : le PHP les complète avec 0.

### Exemple réel 2 : évolution mensuelle (admin/statistics.php)

```
SELECT DATE_FORMAT(dateCreation, '%Y-%m') AS month, COUNT(*) AS count
FROM utilisateur
WHERE dateCreation >= DATE_SUB(NOW(), INTERVAL 12 MONTH)
GROUP BY DATE_FORMAT(dateCreation, '%Y-%m')
ORDER BY month ASC;
```

| month   | count |
|---------|-------|
| 2026-06 | 2     |
| 2026-08 | 4     |
| 2026-09 | 5     |

Ligne par ligne : on transforme chaque date d'inscription en « année-mois » ( '2026-09' ), on ne garde que les 12 derniers mois, on regroupe par mois, puis on trie. Le format %Y-%m est choisi exprès car il se trie correctement comme du texte (ce ne serait pas le cas de %m/%Y). La même requête sur intervention.dateIntervention donne 8 interventions en 2026-09.

### Exemple métier : ancienneté des anomalies et retards

```
SELECT idAnomalie, niveau, statut,
       DATE_FORMAT(dateDetection, '%d/%m/%Y') AS detectee,
       DATEDIFF(COALESCE(dateResolution, NOW()), dateDetection) AS jours
FROM anomalie;
```

| idAnomalie | niveau   | statut   | detectee   | jours |                        |
|------------|----------|----------|------------|-------|------------------------|
| 914        | CRITIQUE | NOUVELLE | 23/09/2026 | 9     | (toujours ouverte)     |
| 915        | MOYEN    | EN_COURS | 21/09/2026 | 11    | (toujours ouverte)     |
| 916        | MOYEN    | TRAITEE  | 11/09/2026 | 2     | (résolue en 2 jours)   |
| 919        | CRITIQUE | TRAITEE  | 15/09/2026 | 0     | (résolue le jour même) |

Si l'anomalie est résolue, on mesure jusqu'à `dateResolution` ; sinon `COALESCE` remplace le `NULL` par `NOW()` et on obtient l'âge actuel.

```
-- Interventions planifiées dont la date est dépassée de plus de 7 jours
SELECT idIntervention, DATE_FORMAT(dateIntervention, '%d/%m/%Y') AS le,
       DATEDIFF(NOW(), dateIntervention) AS jours_ecoules
FROM intervention
WHERE statut = 'PLANIFIEE' AND dateIntervention < NOW() - INTERVAL 7 DAY;
```

```
923 | 24/09/2026 | 8
916 | 24/09/2026 | 8
932 | 25/09/2026 | 7
```

### Piège fréquent

Pour une durée en heures (ex. entre `reparation.dateDebut` et `dateFin`), `DATEDIFF` ne convient pas : il ne compte que des jours entiers. On utilise `TIMESTAMPDIFF(HOUR, dateDebut, dateFin)`. Sur votre base, les 2 réparations ont `dateDebut` et `dateFin` à `NULL`, le résultat est donc `NULL` : toute opération avec `NULL` donne `NULL`. La durée est en fait stockée dans `dureeIntervention` (moyenne 0,75).

## 4.7 Synthèse et vues

**Q66. Expliquez la requête « charge des garages » du tableau de bord administrateur : elle combine presque tout.**

```
SELECT g.nomGarage,
       SUM(CASE WHEN i.statut IN ('PLANIFIEE', 'EN_COURS') THEN 1 ELSE 0 END) AS actives
FROM garage g
LEFT JOIN intervention i ON i.idGarage = g.idGarage
WHERE g.statutGarage = 'VALIDE'
GROUP BY g.idGarage, g.nomGarage
ORDER BY actives DESC
LIMIT 6;
```

Dans l'ordre d'exécution :

1. `FROM garage g LEFT JOIN intervention i ON ...` : chaque garage est associé à ses interventions ; un garage sans intervention est gardé avec `NULL`.
2. `WHERE g.statutGarage = 'VALIDE'` : on enlève les garages en attente, rejetés ou suspendus. La condition porte sur la table de gauche, elle ne casse donc pas le `LEFT JOIN` (contrairement à Q58).
3. `GROUP BY g.idGarage, g.nomGarage` : un groupe par garage.
4. `SUM(CASE ... THEN 1 ELSE 0 END)` : on compte les interventions actives du groupe. Pour un garage sans intervention, `i.statut` est `NULL`, le `CASE` donne 0 et le garage affiche bien 0 (un `COUNT(*)` aurait affiché 1).
5. `ORDER BY actives DESC LIMIT 6` : les 6 garages les plus chargés.

```
nomGarage | actives
(masqué)  | 4
```

## Q67. Qu'est-ce qu'une vue ? Pourriez-vous en créer une pour votre projet ?

Une **vue** (VIEW) est une requête `SELECT` enregistrée dans la base sous un nom, et qu'on interroge ensuite comme une table. Elle ne stocke pas de données (c'est une « table virtuelle ») : à chaque lecture, MySQL réexécute la requête, et le résultat est donc toujours à jour. Votre base n'en contient aucune aujourd'hui (vérifié dans `information_schema.VIEWS` : 0 vue).

Exemple qu'on pourrait ajouter (écrit et expliqué ici, **non exécuté** sur la base ; seule la partie `SELECT` a été testée, elle renvoie les 10 interventions) :

```
CREATE VIEW v_intervention_detail AS
SELECT i.idIntervention, i.type, i.statut, i.priorite, i.dateIntervention,
       v.idVehicule, v.marque, v.modele, v.immatriculation,
       i.idClient, g.idGarage, g.nomGarage, i.idTechnicien
FROM intervention i
INNER JOIN vehicule v ON v.idVehicule = i.idVehicule
LEFT JOIN garage g ON g.idGarage = i.idGarage;
```

- `CREATE VIEW nom AS` : crée la vue et lui donne un nom (préfixe `v_` par convention, pour la distinguer d'une table).
- La suite est un `SELECT` normal : ici la jointure de Q53, qu'on n'aura plus à réécrire.

Utilisation :

```
SELECT statut, COUNT(*) FROM v_intervention_detail GROUP BY statut;
SELECT * FROM v_intervention_detail WHERE idGarage = 3;
```

Pour la modifier : `CREATE OR REPLACE VIEW v_intervention_detail AS ...` ; pour la supprimer : `DROP VIEW v_intervention_detail`; (cela supprime seulement la définition, jamais les données des tables).

### À retenir

- Intérêts d'une vue : simplifier les requêtes répétées, cacher la complexité des jointures, et **sécuriser** en ne montrant que certaines colonnes (la vue ci-dessus n'expose ni `motDePasse` ni `email`) ; on peut donner à un compte MySQL le droit de lire la vue sans lui donner accès aux tables.
- Une vue ne rend pas une requête plus rapide : elle est réexécutée à chaque lecture.
- Une vue sans agrégat, sans `DISTINCT`, sans `GROUP BY` ni `UNION` peut accepter des `UPDATE` (avec une jointure, MySQL l'accepte seulement si l'`UPDATE` ne modifie qu'une des tables) ; une vue avec agrégats, `DISTINCT`, `GROUP BY` ou `UNION` est en lecture seule.

### À retenir pour tout le chapitre

- Ordre d'exécution : FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT .
- Clé étrangère NOT NULL : INNER JOIN sans risque. Clé étrangère qui peut être NULL ( idGarage , idTechnicien ) : LEFT JOIN .
- Avec LEFT JOIN , comptez avec COUNT(colonne\_de\_droite) , pas COUNT(\*) .
- NULL se teste avec IS NULL , se remplace avec COALESCE , et il est ignoré par les agrégats sauf COUNT(\*) .
- SUM(CASE WHEN ... THEN 1 ELSE 0 END) compte plusieurs catégories en une seule lecture de la table.
- UNION dédouble, UNION ALL non (et va plus vite) : getUserRole utilise UNION ALL car ses branches ne peuvent pas produire de doublons.

## 5. Transactions, sécurité et performances

Ce chapitre couvre les questions que le jury pose souvent après le schéma : « que se passe-t-il si une opération échoue au milieu ? », « votre base est-elle protégée contre les attaques ? », « que ferez-vous quand elle grossira ? ». Toutes les requêtes `SELECT` et `EXPLAIN` ci-dessous ont été exécutées sur la vraie base `charles` (MySQL 8.0.41). Les commandes qui modifient la base (`INSERT`, `UPDATE`, `CREATE USER`, `GRANT` ...) sont seulement écrites et expliquées : elles n'ont pas été lancées.

### 5.1 Les transactions

#### Q70. Que signifie ACID ? Votre base respecte-t-elle ces propriétés ?

ACID désigne les quatre garanties qu'un moteur de base de données doit offrir pour qu'une **transaction** soit fiable. Une transaction est un groupe d'opérations SQL traité comme un seul bloc.

| Lettre | Propriété  | Ce que cela veut dire                                                                                                                                                                 | Exemple dans SmartAutoTrack                                                                                                                              |
|--------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| A      | Atomicité  | Tout ou rien : soit toutes les opérations du bloc sont enregistrées, soit aucune.                                                                                                     | À la clôture d'une réparation, on n'aura jamais une ligne dans <code>reparation</code> avec une <code>intervention</code> restée <code>EN_COURS</code> . |
| C      | Cohérence  | La base passe d'un état valide à un autre état valide : les contraintes (clés étrangères, <code>NOT NULL</code> , <code>UNIQUE</code> , <code>ENUM</code> ) sont toujours respectées. | Impossible d'insérer une réparation dont <code>idIntervention</code> n'existe pas, grâce à la contrainte <code>fk_reparation_intervention</code> .       |
| I      | Isolation  | Deux transactions qui s'exécutent en même temps ne se gênent pas : chacune voit des données stables.                                                                                  | Deux clics simultanés sur « Payer » ne créent pas deux paiements (verrou <code>FOR UPDATE</code> , question Q74).                                        |
| D      | Durabilité | Une fois validée ( <code>COMMIT</code> ), la modification survit à une panne ou à une coupure de courant.                                                                             | InnoDB écrit d'abord dans un journal (le redo log) sur le disque avant de confirmer.                                                                     |

Ces garanties sont fournies par le **moteur de stockage InnoDB**. Il faut donc vérifier que toutes les tables l'utilisent. La requête suivante interroge `information_schema`, une base « catalogue » que MySQL tient à jour et qui décrit toutes les tables :

```
SELECT ENGINE, COUNT(*) AS nb
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE()
GROUP BY ENGINE;
```

- `information_schema.TABLES` : la liste de toutes les tables du serveur.
- `WHERE TABLE_SCHEMA = DATABASE()` : on garde seulement les tables de la base courante (`DATABASE()` renvoie son nom, ici `charles`).
- `GROUP BY ENGINE` avec `COUNT(*)` : on compte les tables par moteur.

|        |    |
|--------|----|
| ENGINE | nb |
| InnoDB | 19 |

Les 19 tables sont en InnoDB. La réponse est donc oui : les transactions et les clés étrangères fonctionnent réellement. (L'ancien moteur MyISAM, lui, ignore les transactions et les clés étrangères.)

### Q71. Qu'est-ce qu'une transaction et quelles commandes SQL l'utilisent ?

Par défaut, MySQL est en mode **autocommit** : chaque requête est validée immédiatement et seule. On le vérifie :

```
SELECT @@autocommit AS autocommit, @@transaction_isolation AS isolation;
```

```
autocommit    isolation
1             REPEATABLE-READ
```

`@@` lit une variable du serveur. `1` signifie que l'autocommit est actif. Pour regrouper plusieurs requêtes, on ouvre une transaction avec trois commandes :

| Commande                                                   | Rôle                                                                                                       | Équivalent PHP (PDO)                       |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>START TRANSACTION</code><br>(ou <code>BEGIN</code> ) | Ouvre la transaction : l'autocommit est suspendu, les modifications suivantes restent provisoires.         | <code>\$conn-&gt;beginTransaction()</code> |
| <code>COMMIT</code>                                        | Valide définitivement toutes les modifications depuis le <code>START TRANSACTION</code> .                  | <code>\$conn-&gt;commit()</code>           |
| <code>ROLLBACK</code>                                      | Annule toutes les modifications depuis le <code>START TRANSACTION</code> , comme si rien ne s'était passé. | <code>\$conn-&gt;rollBack()</code>         |

Pendant la transaction, les autres utilisateurs ne voient pas encore les modifications provisoires. Ils les voient seulement après le `COMMIT`.

Dans le projet, des transactions sont utilisées dans `technicien/reparations.php`, `garage/reparations.php`, `includes/payments.php`, `auth/register.php` (création d'un compte dans utilisateur puis dans la table du rôle), `admin/clients.php`, `admin/garages.php`, `admin/techniciens.php` et `garage/techniciens.php`.

### Q72. Montrez une vraie transaction de votre projet et expliquez-la.

Quand un technicien clôture une réparation (`technicien/reparations.php`), cinq écritures doivent réussir ensemble. Voici l'équivalent en SQL pur de ce que fait le code PHP (les `?` sont remplacés par des valeurs d'exemple) :

```

START TRANSACTION;

-- 1. Créer la fiche de réparation
INSERT INTO reparation (idIntervention, idTechnicien, titre, description, diagnostic,
                      travauxEffectues, piecesUtilisees, dureeIntervention, cout,
                      recommandations, statut)
VALUES (12, 7, 'Vidange', 'Vidange moteur', 'Huile usée', 'Remplacement huile et filtre',
      'Filtre à huile', 1.5, 25000, 'Revenir dans 5000 km', 'TERMINEE');

-- 2. Clôturer l'intervention (seulement si elle appartient à ce technicien)
UPDATE intervention SET statut = 'TERMINEE'
WHERE idIntervention = 12 AND idTechnicien = 7;

-- 3. Marquer comme traitées les anomalies liées à cette intervention
UPDATE anomalie SET statut = 'TRAITEE', dateResolution = NOW()
WHERE idIntervention = 12 AND statut IN ('NOUVELLE', 'EN_COURS');

-- 4. Écrire une ligne dans journalactivites (fonction technicien_log)
-- 5. Prévenir le client
INSERT INTO notifications (user_id, type, titre, message)
VALUES (25, 'rapport', 'Rapport de réparation disponible',
      'Le rapport de réparation pour votre véhicule est disponible.');
```

```

COMMIT;
```

Explication mot par mot des commandes utilisées :

- `INSERT INTO table (colonnes) VALUES (valeurs)` : ajoute une nouvelle ligne ; chaque valeur va dans la colonne qui est à la même position. Les colonnes absentes ( `idReparation`, `dateReparation` ) prennent leur valeur par défaut ( `AUTO_INCREMENT` et `CURRENT_TIMESTAMP` ).
- `UPDATE table SET colonne = valeur WHERE condition` : modifie les lignes qui vérifient la condition. Sans `WHERE`, **toutes** les lignes seraient modifiées.
- `statut IN ('NOUVELLE', 'EN_COURS')` : vrai si le statut vaut l'une des valeurs de la liste ; on ne touche donc pas aux anomalies déjà `TRAITEE` ou `IGNOREE`.
- `NOW()` : la date et l'heure actuelles du serveur.

Le code PHP réel est le suivant (extrait) :

```

$conn->beginTransaction();
$stmt = $conn->prepare("INSERT INTO reparation (idIntervention, idTechnicien, titre, ...)
VALUES (?, ?, ?, ..., 'TERMINEE')");
$stmt->execute([$interventionId, $selfId, $titre, ...]);
$reparationId = (int)$conn->lastInsertId();
$conn->prepare("UPDATE intervention SET statut = 'TERMINEE' WHERE idIntervention = ? AND
idTechnicien = ?")
->execute([$interventionId, $selfId]);
$conn->prepare("UPDATE anomalie SET statut = 'TRAITEE', dateResolution = NOW() WHERE
idIntervention = ? AND statut IN ('NOUVELLE', 'EN_COURS')")
->execute([$interventionId]);
technicien_log($conn, 'Réparation renseignée et intervention clôturée', [...]);
$conn->prepare("INSERT INTO notifications (user_id, type, titre, message) VALUES (?,
'rapport', ...)");
->execute([$iv['idClient'], '...']);
$conn->commit();
```

`lastInsertId()` récupère le numéro `idReparation` que MySQL vient de générer, pour le noter dans le journal. La syntaxe de ces requêtes a été vérifiée avec `EXPLAIN` (qui analyse sans rien modifier) : MySQL accepte l'`UPDATE intervention` en utilisant la clé primaire, et l'`UPDATE anomalie` en utilisant l'index `fk_anomalie_intervention` :

```
EXPLAIN UPDATE anomalie SET statut = 'TRAITEE', dateResolution = NOW()
WHERE idIntervention = 1 AND statut IN ('NOUVELLE', 'EN_COURS');
```

| select_type | table    | type  | key                      | rows | Extra       |
|-------------|----------|-------|--------------------------|------|-------------|
| UPDATE      | anomalie | range | fk_anomalie_intervention | 1    | Using where |

**Pourquoi une transaction ici ?** Sans elle, si le serveur plantait après l'étape 1, on aurait une réparation `TERMINEE` attachée à une intervention encore `EN_COURS` : le technicien pourrait en saisir une deuxième, et le client ne serait jamais prévenu. Avec la transaction, c'est tout ou rien (atomicité).

### Q73. Que se passe-t-il si une des requêtes échoue au milieu de la transaction ?

La connexion PDO est configurée avec `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` (fichier `config/database.php`) : toute erreur SQL lance une **exception** PHP, ce qui interrompt le code avant le `commit()`. Les modifications déjà faites restent provisoires et sont annulées par un `ROLLBACK`.

Démonstration en SQL pur (exemple à faire sur une base de test, jamais sur la vraie) :

```
START TRANSACTION;
UPDATE intervention SET statut = 'TERMINEE' WHERE idIntervention = 12;
-- on se rend compte d'une erreur
ROLLBACK;
-- l'intervention 12 a retrouvé son ancien statut
```

Dans `includes/payments.php`, le schéma est explicite et complet :

```
$conn->beginTransaction();
try {
    ... // lectures et INSERT
    $conn->commit();
} catch (Throwable $e) {
    $conn->rollBack(); // on annule tout
    throw $e;        // et on signale l'erreur
}
```

Dans `technicien/reparations.php`, il n'y a pas de bloc `try/catch` autour de la transaction. En cas d'erreur, l'exception arrête le script ; à la fin du script, PDO annule de lui-même une transaction restée ouverte, et MySQL fait de même quand une connexion se ferme sans `COMMIT`. Le résultat est le même (rien n'est enregistré), mais il est plus propre d'écrire le `rollBack()` explicitement, comme dans `payments.php` ou `admin/garages.php` (qui teste `$conn->inTransaction()` avant d'annuler).

## Piège fréquent

« Si le serveur web plante après le `COMMIT`, la réparation est-elle perdue ? » Non : après `COMMIT`, la durabilité (le D de ACID) garantit que les données sont sur le disque. Et s'il plante avant le `COMMIT`, la connexion est coupée et MySQL annule la transaction inachevée. Si c'est le serveur MySQL lui-même qui plante, InnoDB annule les transactions inachevées au redémarrage, grâce à son journal.

### Q74. À quoi sert `SELECT ... FOR UPDATE` dans votre code de paiement ?

Dans `includes/payments.php`, avant de créer un paiement Mobile Money, le code relit la réparation **en la verrouillant** :

```
SELECT r.idReparation, r.titre, r.cout, i.idIntervention, i.idClient
FROM reparation r
JOIN intervention i ON i.idIntervention = r.idIntervention
WHERE r.idReparation = ? AND i.idClient = ?
      AND r.statut = 'TERMINEE' AND r.cout > 0
FOR UPDATE;
```

- `reparation r` : `r` est un alias, un surnom court pour la table.
- `JOIN intervention i ON ...` : relie chaque réparation à son intervention pour connaître le client.
- `i.idClient = ?` : vérifie que la réparation appartient bien au client connecté (il ne peut pas payer celle d'un autre).
- `FOR UPDATE` : pose un **verrou exclusif de ligne** sur les lignes lues. Tant que la transaction n'est pas terminée (`COMMIT` ou `ROLLBACK`), une autre transaction qui voudrait verrouiller ou modifier ces mêmes lignes doit **attendre**.

**Le problème évité.** Le client clique deux fois très vite sur « Payer ». Sans verrou, les deux requêtes lisent en même temps « aucun paiement en cours », puis insèrent chacune un paiement : le client serait débité deux fois. Avec `FOR UPDATE`, la deuxième requête attend que la première ait fait son `INSERT` et son `COMMIT` ; quand elle reprend, elle voit le paiement `EN_ATTENTE` et refuse (« Un paiement est déjà en cours »).

Le verrou porte sur une seule ligne de `reparation` et une seule ligne de `intervention`, pas sur les tables entières : les autres clients peuvent payer leurs propres réparations en parallèle. C'est possible parce que la recherche passe par la clé primaire, comme le montre `EXPLAIN` exécuté sur une réparation terminée réelle :

| table | type  | key     | rows |
|-------|-------|---------|------|
| r     | const | PRIMARY | 1    |
| i     | const | PRIMARY | 1    |

`const` signifie que MySQL trouve directement une ligne unique par la clé primaire ; il ne verrouille donc que cette ligne dans chaque table.

### Piège fréquent

« Pouvez-vous nous montrer un paiement ? » Attention : sur la base actuelle, la table `paiement` n'a que 7 colonnes (`idPaiement`, `idClient`, `idIntervention`, `montant`, `datePaiement`, `typePaiement`, `statut`) et elle est vide. Les colonnes `idReparation`, `referenceExterne`, `referenceCampay` et `telephone`, utilisées par le code, n'existent pas encore : la migration `scripts/migrate_structure.php` n'a pas été appliquée. La fonction `paymentsReady()` renvoie donc `false`, et l'`INSERT INTO paiement` de `payments.php` échouerait (erreur 1054 « Unknown column 'idReparation' »). Il faut lancer la migration avant toute démonstration, ou présenter le paiement comme une fonctionnalité prévue.

L'appel réseau vers l'opérateur de paiement (CamPay) est fait **après** le `COMMIT`, volontairement : garder un verrou pendant un appel réseau lent bloquerait les autres.

### Q75. Qu'est-ce que le niveau d'isolation ? Lequel utilisez-vous ?

Le niveau d'isolation règle ce qu'une transaction voit des modifications des autres transactions en cours. Il existe quatre niveaux, du moins strict au plus strict : `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`. La requête de Q71 a montré que la base utilise `REPEATABLE-READ`, le niveau par défaut d'InnoDB.

Avec `REPEATABLE READ`, une transaction ne voit jamais de données non validées par une autre (pas de « lecture sale »), et si elle relit la même ligne, elle obtient la même valeur. Ce niveau suffit pour l'application ; pour les cas sensibles comme le paiement, on ajoute un verrou explicite ( `FOR UPDATE` ).

### À retenir

- Transaction = `START TRANSACTION ... COMMIT` (valider) ou `ROLLBACK` (annuler).
- ACID : Atomicité, Cohérence, Isolation, Durabilité ; fournies par InnoDB (19 tables sur 19).
- Clôture de réparation : 1 `INSERT` reparation + 1 `UPDATE` intervention + 1 `UPDATE` anomalie + 1 log + 1 notification, tout ou rien.
- `FOR UPDATE` verrouille la ligne lue jusqu'à la fin de la transaction : empêche le double paiement.

## 5.2 Sécurité

### Q76. Qu'est-ce qu'une injection SQL ? Montrez une attaque.

Une injection SQL consiste à taper, dans un champ de formulaire, du texte qui sera interprété comme du **code SQL**. Elle est possible quand le programme construit la requête en collant (concaténant) la saisie de l'utilisateur dans le texte SQL.

Code vulnérable (n'existe pas dans le projet, c'est le contre-exemple) :

```
$sql = "SELECT * FROM utilisateur WHERE email = '" . $_POST['email'] . "'";  
$conn->query($sql);
```

Si l'attaquant tape comme email : `' OR '1'='1'`, la requête envoyée devient :

```
SELECT * FROM utilisateur WHERE email = '' OR '1'='1';
```

La condition `'1'='1'` est toujours vraie, et `OR` rend toute la condition vraie : la requête renvoie **tous** les utilisateurs. Test réel (avec `COUNT(*)` pour ne pas afficher de données personnelles) :

```
SELECT COUNT(*) AS nb FROM utilisateur WHERE email = '' OR '1'='1';
```

```
nb
15
```

Les 15 comptes de la table sont renvoyés, alors qu'aucun n'a un email vide. Une variante utilise `--` (début d'un commentaire SQL) pour neutraliser la fin de la requête : `x' OR 1=1 --` donne aussi 15. Dans une page de connexion mal écrite, cela permettrait d'entrer sans mot de passe ; avec `'; DROP TABLE ...` sur certains systèmes, de détruire des données.

### Q77. Comment votre projet se protège-t-il de l'injection SQL ?

Toutes les requêtes qui contiennent une saisie utilisateur passent par des **requêtes préparées** PDO, en deux temps :

1. `prepare()` : on envoie au serveur le texte SQL avec des `?` à la place des valeurs. MySQL analyse et compile la requête à ce moment-là.
2. `execute([...])` : on envoie les valeurs séparément. MySQL les traite uniquement comme des **données**, jamais comme du code.

Exemple réel, utilisé à la connexion ( `config/roles.php` , fonction `findUserByEmail` ) :

```
$stmt = $conn->prepare('SELECT * FROM utilisateur WHERE email = ?');
$stmt->execute([$email]);
$row = $stmt->fetch(PDO::FETCH_ASSOC);
```

Si l'attaquant tape `' OR '1'='1'`, MySQL cherche un utilisateur dont l'email est littéralement ce texte. Cela revient à la requête suivante, où les apostrophes sont « échappées » (doublées) et font partie de la valeur :

```
SELECT COUNT(*) AS nb FROM utilisateur WHERE email = ''' OR ''1''='1';
```

```
nb
0
```

Aucun résultat : l'attaque échoue.

Dans `config/database.php`, l'option `PDO::ATTR_EMULATE_PREPARES => false` est importante. Par défaut, PDO peut « simuler » les requêtes préparées en remplaçant lui-même les `?` dans le texte avant l'envoi. Avec `false`, ce sont de **vraies** requêtes préparées côté serveur MySQL : la requête et les données voyagent séparément. Le paramètre `charset=utf8mb4` dans la chaîne de connexion évite aussi certaines attaques liées aux encodages.

### Piège fréquent

« Un `?` peut-il remplacer un nom de table ou de colonne ? » Non, seulement une valeur. Quand le code doit construire une liste variable, il génère des `?` : dans `payments.php`, `IN ($in)` est fabriqué avec `array_fill(0, count($repairIds), '?')`, donc on obtient `IN (?, ?, ?)` et les valeurs passent toujours par `execute()`.

### Q78. Comment sont stockés les mots de passe ? Pourrait-on les lire dans la base ?

Les mots de passe ne sont **jamais stockés en clair**. La colonne `utilisateur.motDePasse` (`VARCHAR(255)` `NOT NULL`) contient une **empreinte** (un « hachage ») calculée par PHP :

```
function hashPassword($password) {
    return password_hash($password, PASSWORD_DEFAULT); // à l'inscription
}
function verifyPassword($password, $hash) {
    return password_verify($password, $hash);           // à la connexion
}
```

Vérification réelle sur la base, sans afficher les empreintes elles-mêmes :

```
SELECT LEFT(motDePasse, 7) AS debut_hash, LENGTH(motDePasse) AS longueur, COUNT(*) AS nb
FROM utilisateur
GROUP BY LEFT(motDePasse, 7), LENGTH(motDePasse);
```

- `LEFT(texte, 7)` : les 7 premiers caractères ; `LENGTH()` : la longueur.
- `GROUP BY` sur ces deux valeurs : on regroupe les comptes qui ont le même format.

| debut_hash | longueur | nb |
|------------|----------|----|
| \$2y\$10\$ | 60       | 15 |

Les 15 comptes ont une empreinte **bcrypt** : `$2y$` désigne l'algorithme, `10` le « coût » (2 puissance 10, soit 1024 tours de calcul, ce qui rend chaque essai lent pour un pirate), et la longueur est de 60 caractères. Un hachage est à sens unique : on ne peut pas retrouver le mot de passe à partir de l'empreinte. `password_hash` ajoute aussi un **sel** aléatoire : deux utilisateurs avec le même mot de passe ont des empreintes différentes. À la connexion, `password_verify` recalcule l'empreinte du mot de passe tapé et la compare.

La colonne fait 255 caractères (et pas 60) pour accepter un futur algorithme plus long (par exemple Argon2) si `PASSWORD_DEFAULT` change.

### Piège fréquent

« Pourquoi pas MD5 ou SHA1 ? » Ils sont conçus pour être très rapides : un pirate peut tester des milliards de mots de passe par seconde. bcrypt est volontairement lent et salé. Et « Un administrateur peut-il voir le mot de passe d'un client ? » Non, il ne voit que l'empreinte ; il peut seulement en définir un nouveau (script `scripts/set_password.php`).

### Q79. Avec quel utilisateur MySQL l'application se connecte-t-elle ? Quels droits devrait-il avoir ?

En local, l'application se connecte avec le compte `root` (valeur par défaut dans `config/database.php`, que l'on peut remplacer par les variables d'environnement `HCH_DB_USER` / `HCH_DB_PASS` ou le fichier `config/local.php`). Vérification :

```
SELECT CURRENT_USER() AS utilisateur_connecte;
```

```
utilisateur_connecte  
root@localhost
```

`SHOW GRANTS;` (liste des droits du compte courant) confirme que `root` a **tous** les droits sur **toutes** les bases (`ON *.*`), y compris `DROP`, `CREATE USER` et `FILE`, avec `WITH GRANT OPTION` (il peut donner ses droits à d'autres). C'est acceptable sur un poste de développement, mais **pas en production** : si une faille existait dans le code, l'attaquant aurait tout le serveur.

On applique le **principe du moindre privilège** : chaque compte reçoit seulement les droits dont il a besoin. Commandes à exécuter en production (écrites ici, non exécutées) :

```
-- 1. Créer un compte dédié, utilisable seulement depuis le serveur web  
CREATE USER 'smartauto_app'@'localhost' IDENTIFIED BY 'UnMotDePasseLongEtSolide!2026';  
  
-- 2. Donner uniquement la lecture et l'écriture des données, sur la seule base charles  
GRANT SELECT, INSERT, UPDATE, DELETE ON charles.* TO 'smartauto_app'@'localhost';  
  
-- 3. Vérifier  
SHOW GRANTS FOR 'smartauto_app'@'localhost';  
  
-- Retirer un droit si besoin  
REVOKE DELETE ON charles.* FROM 'smartauto_app'@'localhost';
```

- `CREATE USER 'nom'@'hôte'` : crée un compte ; `'localhost'` signifie qu'il ne peut se connecter que depuis la machine elle-même.
- `IDENTIFIED BY` : son mot de passe (MySQL le stocke haché).
- `GRANT droits ON base.table TO compte` : accorde des droits ; `charles.*` = toutes les tables de la base `charles`, et rien d'autre.
- `REVOKE ... FROM` : retire un droit.

Ce compte ne peut ni supprimer une table (`DROP`), ni modifier la structure (`ALTER`), ni créer d'autres comptes. Les migrations (`scripts/migrate_*.php`) seraient lancées avec un autre compte, administrateur, utilisé seulement à ce moment-là. On peut aussi créer un compte en lecture seule (`GRANT SELECT`) pour les statistiques.

### À retenir

- Injection SQL : la saisie devient du code quand on concatène ; parade : `prepare()` + `execute()` avec des `?`, et `EMULATE_PREPARES = false`.
- Mots de passe : `password_hash` (`bcrypt`, `$2y$10$`, 60 caractères, salé), vérifiés par `password_verify`.
- `root` en local seulement ; en production, un compte dédié avec `GRANT SELECT, INSERT, UPDATE, DELETE ON charles.*`.

## 5.3 Performances

### Q80. Qu'est-ce qu'un index et comment savoir si MySQL l'utilise ?

Un index est une structure triée (un arbre B-Tree) que MySQL tient à jour à côté d'une table, comme l'index à la fin d'un livre : au lieu de lire toutes les pages, on va directement à la bonne. Sans index, MySQL doit lire toutes les lignes de la table (on parle de « parcours complet », full table scan).

La commande `EXPLAIN`, placée devant un `SELECT`, n'exécute pas la requête : elle montre **comment** MySQL compte la faire. Les colonnes à regarder :

- `type` : la méthode d'accès. Du meilleur au pire : `const` (une seule ligne via une clé unique), `ref` (via un index non unique), `range` (une plage de l'index), `ALL` (toute la table est lue).
- `key` : l'index réellement choisi (`NULL` = aucun).
- `rows` : l'estimation du nombre de lignes lues.

**Exemple 1**, recherche par immatriculation (colonne `UNIQUE`) :

```
EXPLAIN SELECT idVehicule, marque FROM vehicule WHERE immatriculation = '...';
```

| table    | type  | possible_keys   | key             | rows |
|----------|-------|-----------------|-----------------|------|
| vehicule | const | immatriculation | immatriculation | 1    |

**Exemple 2**, véhicules d'un client (clé étrangère, donc indexée) :

```
EXPLAIN SELECT idVehicule, marque FROM vehicule WHERE idClient = 5;
```

| table    | type | key                | rows |
|----------|------|--------------------|------|
| vehicule | ref  | fk_vehicule_client | 1    |

**Exemple 3**, recherche par marque (aucun index) :

```
EXPLAIN SELECT idVehicule FROM vehicule WHERE marque = 'Toyota';
```

| table    | type | possible_keys | key  | rows | Extra       |
|----------|------|---------------|------|------|-------------|
| vehicule | ALL  | NULL          | NULL | 9    | Using where |

Ici `type = ALL` : MySQL lit les 9 véhicules un par un. Avec 9 lignes c'est instantané ; avec 900 000, ce serait lent.

Une jointure profite aussi des index : pour les réparations d'un client, MySQL utilise `fk_intervention_client` puis `fk_reparation_intervention` :

```
EXPLAIN SELECT r.idReparation, r.cout
FROM reparation r JOIN intervention i ON i.idIntervention = r.idIntervention
WHERE i.idClient = 5;
```

| table | type | key                        | ref                      |
|-------|------|----------------------------|--------------------------|
| i     | ref  | fk_intervention_client     | const                    |
| r     | ref  | fk_reparation_intervention | charles.i.idIntervention |

## Q81. Quels index existent déjà dans votre base, et quand faut-il en ajouter ?

MySQL crée automatiquement un index pour chaque `PRIMARY KEY`, chaque `UNIQUE` et chaque clé étrangère (c'est pourquoi on voit des `KEY fk_...` dans les `CREATE TABLE`). Liste réelle pour `vehicule` et `utilisateur` :

```
SELECT TABLE_NAME, INDEX_NAME, NON_UNIQUE, COLUMN_NAME
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME IN ('vehicule', 'utilisateur')
ORDER BY TABLE_NAME, INDEX_NAME;
```

| TABLE_NAME  | INDEX_NAME         | NON_UNIQUE | COLUMN_NAME     |
|-------------|--------------------|------------|-----------------|
| utilisateur | email              | 0          | email           |
| utilisateur | PRIMARY            | 0          | idUtilisateur   |
| vehicule    | fk_vehicule_client | 1          | idClient        |
| vehicule    | fk_vehicule_garage | 1          | idGarage        |
| vehicule    | immatriculation    | 0          | immatriculation |
| vehicule    | numeroChassis      | 0          | numeroChassis   |
| vehicule    | PRIMARY            | 0          | idVehicule      |

`NON_UNIQUE = 0` signifie que l'index interdit les doublons.

On ajoute un index sur une colonne **souvent utilisée** dans un `WHERE`, un `JOIN` ou un `ORDER BY`, quand `EXPLAIN` montre `type = ALL` sur une grosse table. Exemple : les tableaux de bord filtrent beaucoup les interventions par statut, et `EXPLAIN` montre aujourd'hui un parcours complet :

```
EXPLAIN SELECT idIntervention FROM intervention WHERE statut = 'EN_COURS';
```

| table        | type | key  | rows | filtered | Extra       |
|--------------|------|------|------|----------|-------------|
| intervention | ALL  | NULL | 10   | 25.00    | Using where |

Si la table grossissait, on créerait un index composé (non exécuté ici) :

```
CREATE INDEX idx_intervention_tech_statut ON intervention (idTechnicien, statut);
-- suppression si inutile :
DROP INDEX idx_intervention_tech_statut ON intervention;
```

Un index composé sur `(idTechnicien, statut)` sert la requête réelle du technicien `WHERE i.idTechnicien = ? AND i.statut = 'EN_COURS'`. L'ordre compte : il sert aussi une recherche sur `idTechnicien` seul, mais pas sur `statut` seul.

### Piège fréquent

« Pourquoi ne pas indexer toutes les colonnes ? » Chaque index prend de la place et doit être mis à jour à chaque `INSERT`, `UPDATE` ou `DELETE` : trop d'index ralentissent les écritures. Un index est aussi peu utile sur une colonne avec très peu de valeurs différentes (un `ENUM` à 4 valeurs seul, par exemple) et sur une petite table : avec 10 lignes, MySQL préfère tout lire.

## 5.4 Sauvegarde et restauration

### Q82. Comment sauvegardez-vous et restaurez-vous la base ?

L'outil standard est `mysqldump`, un programme en ligne de commande livré avec MySQL. Il produit un fichier `.sql` qui contient les `CREATE TABLE` et les `INSERT` nécessaires pour tout recréer. Ces commandes se tapent dans un terminal (invite de commandes Windows), pas dans MySQL. Pour la sauvegarde :

```
mysqldump -u root -p --single-transaction --routines --triggers charles >
sauvegarde_charles_2026-10-02.sql
```

Pour la restauration dans une base vide :

```
mysql -u root -p -e "CREATE DATABASE charles_restaurée CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci"
mysql -u root -p charles_restaurée < sauvegarde_charles_2026-10-02.sql
```

- `-u root` : le compte utilisé ; `-p` : demande le mot de passe.
- `--single-transaction` : fait la sauvegarde dans une transaction `REPEATABLE READ` ; grâce à InnoDB, on obtient une photo cohérente de toutes les tables sans bloquer l'application.
- `--routines` : inclut aussi les procédures et fonctions stockées s'il y en a ; `--triggers` inclut les déclencheurs (c'est déjà le comportement par défaut, on l'écrit pour être explicite).
- `> fichier.sql` : écrit le résultat dans un fichier ; `< fichier.sql` : lit un fichier et l'envoie à `mysql`, qui exécute toutes ses commandes.

Sans terminal, phpMyAdmin propose les mêmes opérations (onglets « Exporter » et « Importer »).

Bonnes pratiques : sauvegarder automatiquement chaque jour, garder plusieurs versions, stocker une copie sur une autre machine, et **tester** régulièrement une restauration dans une base séparée (comme `charles_restaurée` ci-dessus) pour être sûr que la sauvegarde est utilisable.

#### Piège fréquent

« Copier le dossier de données de MySQL suffit-il ? » Pas pendant que le serveur tourne : les fichiers InnoDB peuvent être dans un état intermédiaire. `mysqldump --single-transaction` est la méthode sûre.

## 5.5 Questions pièges classiques

### Q83. Quelle différence entre une clé primaire et une contrainte UNIQUE ? Comment évitez-vous les doublons ?

|                  | PRIMARY KEY                                         | UNIQUE                                       |
|------------------|-----------------------------------------------------|----------------------------------------------|
| Nombre par table | Une seule                                           | Plusieurs possibles                          |
| Valeur NULL      | Interdite                                           | Autorisée (et plusieurs NULL sont acceptés)  |
| Rôle             | Identifier chaque ligne ; cible des clés étrangères | Interdire les doublons sur une donnée métier |

|         | PRIMARY KEY         | UNIQUE                                                              |
|---------|---------------------|---------------------------------------------------------------------|
| Exemple | vehicule.idVehicule | vehicule.immatriculation, vehicule.numeroChassis, utilisateur.email |

Les doublons sont donc empêchés **par la base elle-même**, pas seulement par le code PHP : si on tente d'inscrire deux fois le même email ou la même immatriculation, MySQL refuse avec l'erreur 1062 « Duplicate entry ». C'est plus sûr qu'une vérification en PHP, qui pourrait être contournée par deux inscriptions simultanées. On peut le vérifier en cherchant des doublons :

```
SELECT immatriculation, COUNT(*) AS nb
FROM vehicule
GROUP BY immatriculation
HAVING COUNT(*) > 1;
```

- `GROUP BY immatriculation` : un groupe par immatriculation.
- `HAVING COUNT(*) > 1` : garde seulement les groupes de plus d'une ligne. `HAVING` filtre après le regroupement, alors que `WHERE` filtre les lignes avant.

(aucune ligne)

```
SELECT COUNT(*) AS total, COUNT(immatriculation) AS avec_immat,
COUNT(DISTINCT immatriculation) AS distinctes
FROM vehicule;
```

```
total  avec_immat  distinctes
9      9           9
```

9 véhicules, 9 immatriculations, 9 valeurs distinctes : aucun doublon. `COUNT(colonne)` ignore les `NULL` et `COUNT(DISTINCT colonne)` compte les valeurs différentes.

La normalisation évite aussi la redondance : le nom et l'email d'un client ne sont écrits qu'une fois, dans `utilisateur` ; les autres tables ne stockent que son identifiant (`idClient`), relié par une clé étrangère.

## Q84. Que feriez-vous si la base devenait très grosse ?

D'abord, mesurer. Taille actuelle réelle :

```
SELECT ROUND(SUM(DATA_LENGTH + INDEX_LENGTH) / 1024 / 1024, 2) AS taille_mo
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = DATABASE();
```

```
taille_mo
0.80
```

`DATA_LENGTH` est la place des données et `INDEX_LENGTH` celle des index, en octets ; on divise deux fois par 1024 pour obtenir des mégaoctets. La plus grosse table est `journalactivites` (112 Ko), logique puisque chaque action y ajoute une ligne : c'est elle qui grossira le plus vite, avec `notifications`.

Ensuite, les mesures par ordre de priorité :

1. **Index** : repérer les requêtes lentes (journal des requêtes lentes de MySQL, `slow_query_log`), les analyser avec `EXPLAIN`, et ajouter les index manquants (Q81).

2. **Pagination** : ne jamais charger toute une table dans une page ; afficher par tranches avec `ORDER BY ... LIMIT 20 OFFSET 40`.
3. **Ne lire que les colonnes utiles** : éviter `SELECT *` sur les tables qui contiennent de longs textes (`reparation.description`, `travauxEffectues...`).
4. **Archivage** : déplacer les vieilles lignes du journal et les notifications lues dans une table d'archive, ou les supprimer après une durée définie.
5. **Matériel et configuration** : augmenter la mémoire réservée à InnoDB (`innodb_buffer_pool_size`) pour que les données fréquentes restent en mémoire.
6. Pour de très gros volumes : **réplication** (un serveur principal pour les écritures, des copies pour les lectures) et **partitionnement** des grandes tables par date.

Les choix déjà faits aident : identifiants entiers `INT AUTO_INCREMENT` (index compacts), clés étrangères indexées, types adaptés (`DECIMAL(10,2)` pour les montants, `ENUM` pour les statuts).

### Piège fréquent

« `INT` suffira-t-il ? » Un `INT` signé va jusqu'à 2 147 483 647. Le compteur `AUTO_INCREMENT` de `utilisateur` est déjà à 999 034 (des identifiants ont été réservés lors de migrations), ce qui laisse encore une très large marge. Si un jour cela ne suffisait pas, on passerait à `BIGINT` avec `ALTER TABLE`, en modifiant aussi les clés étrangères qui pointent vers la colonne.

### À retenir

- Index = accès direct ; `EXPLAIN` : `const / ref` = bien, `ALL` = toute la table lue.
- PK, UNIQUE et clés étrangères sont indexées automatiquement ; trop d'index ralentit les écritures.
- Sauvegarde : `mysqldump --single-transaction charles > fichier.sql` ; restauration : `mysql base < fichier.sql` ; tester la restauration.
- PRIMARY KEY : une seule, jamais NULL ; UNIQUE : plusieurs, NULL permis ; les doublons sont refusés par MySQL (erreur 1062).
- Base actuelle : 0,80 Mo ; si elle grossit : index, pagination, archivage du journal, puis réplication.

## 6. Aide-mémoire et entraînement

Ce dernier chapitre sert à réviser vite avant la soutenance. Il contient trois parties : d'abord une **fiche mémo** qui rappelle chaque commande vue, à quoi elle sert et un exemple court sur la base `charles` ; ensuite **10 exercices**, du plus simple au plus difficile, tous exécutés sur la vraie base ; enfin **les 10 questions que le jury posera presque sûrement**, avec une réponse courte à apprendre.

Rappel : tous les résultats affichés ci-dessous viennent d'une exécution réelle sur la base (MySQL 8.0.41). Les nombres peuvent changer si des données sont ajoutées après la rédaction de ce document. Les données personnelles (noms, emails, téléphones) ne sont jamais affichées.

### 6.1 Fiche mémo : lire des données (SELECT)

| Commande / mot-clé                     | À quoi elle sert                                                                                                     | Exemple court sur la base                                               |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>SELECT ... FROM</code>           | Lire des colonnes dans une table. <code>SELECT</code> dit quelles colonnes, <code>FROM</code> dit dans quelle table. | <code>SELECT marque, modele FROM vehicule;</code>                       |
| <code>WHERE</code>                     | Garder seulement les lignes qui respectent une condition (un filtre).                                                | <code>SELECT * FROM anomalie WHERE niveau = 'CRITIQUE';</code>          |
| <code>AND, OR, NOT</code>              | Combiner plusieurs conditions.                                                                                       | <code>... WHERE lu = 'non' AND type = 'message'</code>                  |
| <code>IN (...)</code>                  | Tester si une valeur fait partie d'une liste (raccourci de plusieurs <code>OR</code> ).                              | <code>... WHERE statut IN ('NOUVELLE', 'EN_COURS')</code>               |
| <code>LIKE</code>                      | Chercher un motif dans un texte. <code>%</code> = n'importe quelle suite de caractères.                              | <code>... WHERE email LIKE '%@gmail.com'</code>                         |
| <code>IS NULL / IS NOT NULL</code>     | Tester l'absence de valeur. On n'écrit jamais <code>= NULL</code> .                                                  | <code>... WHERE idTechnicien IS NULL</code>                             |
| <code>ORDER BY ... ASC/DESC</code>     | Trier le résultat, croissant ( <code>ASC</code> , par défaut) ou décroissant ( <code>DESC</code> ).                  | <code>... ORDER BY kilometrage DESC</code>                              |
| <code>LIMIT n</code>                   | Ne garder que les <code>n</code> premières lignes du résultat.                                                       | <code>... ORDER BY kilometrage DESC LIMIT 3</code>                      |
| <code>DISTINCT</code>                  | Supprimer les doublons dans le résultat.                                                                             | <code>SELECT DISTINCT marque FROM vehicule;</code>                      |
| <code>AS</code>                        | Donner un nom (alias) à une colonne ou une table.                                                                    | <code>SELECT COUNT(*) AS nb FROM client;</code>                         |
| <code>COUNT, SUM, AVG, MIN, MAX</code> | Fonctions d'agrégat : compter, additionner, faire la moyenne, prendre le minimum ou le maximum.                      | <code>SELECT SUM(cout) FROM reparation;</code>                          |
| <code>GROUP BY</code>                  | Regrouper les lignes qui ont la même valeur pour calculer un agrégat par groupe.                                     | <code>SELECT statut, COUNT(*) FROM intervention GROUP BY statut;</code> |
| <code>HAVING</code>                    | Filtrer les groupes après le <code>GROUP BY</code> (alors que <code>WHERE</code> filtre les lignes avant).           | <code>... GROUP BY idClient HAVING COUNT(*) &gt;= 2</code>              |

| Commande / mot-clé                                                    | À quoi elle sert                                                                                                                   | Exemple court sur la base                                                            |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>INNER JOIN ... ON</code>                                        | Associer deux tables et ne garder que les lignes qui ont une correspondance des deux côtés.                                        | <code>intervention i JOIN reparation r ON r.idIntervention = i.idIntervention</code> |
| <code>LEFT JOIN ... ON</code>                                         | Garder toutes les lignes de la table de gauche, même sans correspondance (les colonnes de droite valent alors <code>NULL</code> ). | <code>vehicule v LEFT JOIN intervention i ON i.idVehicule = v.idVehicule</code>      |
| Sous-requête                                                          | Une requête placée à l'intérieur d'une autre, entre parenthèses.                                                                   | <code>... WHERE kilometrage &gt; (SELECT AVG(kilometrage) FROM vehicule)</code>      |
| <code>EXISTS / NOT EXISTS</code>                                      | Tester si une sous-requête renvoie au moins une ligne (ou aucune).                                                                 | <code>... WHERE NOT EXISTS (SELECT 1 FROM reparation r WHERE ...)</code>             |
| <code>CASE WHEN ... THEN ... ELSE ... END</code>                      | Calculer une valeur selon des conditions (le « si / sinon » du SQL).                                                               | <code>CASE WHEN niveau = 'CRITIQUE' THEN 'Urgent' ELSE 'Normal' END</code>           |
| <code>COALESCE(a, b)</code>                                           | Renvoyer la première valeur non nulle : sert à remplacer un <code>NULL</code> par une valeur lisible.                              | <code>COALESCE(specialite, '(non renseignée)')</code>                                |
| <code>NOW()</code> , <code>DATEDIFF</code> , <code>DATE_FORMAT</code> | Fonctions de date : date et heure actuelles, écart en jours, mise en forme d'une date.                                             | <code>DATE_FORMAT(dateIntervention, '%Y-%m')</code>                                  |

## 6.2 Fiche mémo : modifier des données (INSERT, UPDATE, DELETE)

| Commande                              | À quoi elle sert                                                                                                                                                         | Exemple court sur la base                                                                                      |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>INSERT INTO ... VALUES</code>   | Ajouter une nouvelle ligne. Les colonnes non citées prennent leur valeur par défaut ( <code>DEFAULT</code> ) ou leur numéro automatique ( <code>AUTO_INCREMENT</code> ). | <code>INSERT INTO anomalie (idVehicule, description, niveau) VALUES (1, 'Bruit au freinage', 'FAIBLE');</code> |
| <code>UPDATE ... SET ... WHERE</code> | Modifier des lignes existantes. Sans <code>WHERE</code> , <b>toutes</b> les lignes sont modifiées.                                                                       | <code>UPDATE anomalie SET statut = 'TRAITEE', dateResolution = NOW() WHERE idAnomalie = 1;</code>              |
| <code>DELETE FROM ... WHERE</code>    | Supprimer des lignes. Sans <code>WHERE</code> , la table entière est vidée.                                                                                              | <code>DELETE FROM notifications WHERE lu = 'oui' AND date_creation &lt; NOW() - INTERVAL 90 DAY;</code>        |
| <code>EXPLAIN</code>                  | Afficher comment MySQL va exécuter une requête (index utilisé, nombre de lignes lues) <b>sans l'exécuter</b> pour un <code>UPDATE / DELETE / INSERT</code> .             | <code>EXPLAIN UPDATE anomalie SET statut = 'TRAITEE' WHERE idAnomalie = 1;</code>                              |

Les trois exemples ci-dessus ont été vérifiés avec `EXPLAIN` (donc sans rien modifier). Pour l'`UPDATE`, MySQL a répondu qu'il utiliserait la clé primaire et ne lirait qu'une seule ligne :

```
EXPLAIN UPDATE anomalie SET statut = 'TRAITEE', dateResolution = NOW() WHERE idAnomalie = 1;
```

```
select_type | table      | type | key      | rows | Extra
UPDATE      | anomalie  | range | PRIMARY | 1    | Using where
```

## 6.3 Fiche mémo : structure, contraintes et sécurité

| Commande / mot-clé                    | À quoi elle sert                                                                              | Exemple dans la base                                                            |
|---------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| CREATE TABLE                          | Créer une table avec ses colonnes, leurs types et leurs contraintes.                          | CREATE TABLE vehicule (idVehicule INT NOT NULL AUTO_INCREMENT, ...)             |
| ALTER TABLE                           | Modifier une table existante (ajouter une colonne, une contrainte, un index).                 | ALTER TABLE paiement ADD COLUMN idReparation INT NULL;                          |
| DROP TABLE                            | Supprimer une table et toutes ses données, définitivement.                                    | DROP TABLE assistantia; (à ne pas faire)                                        |
| PRIMARY KEY                           | Identifiant unique et obligatoire de chaque ligne.                                            | PRIMARY KEY (idVehicule)                                                        |
| AUTO_INCREMENT                        | MySQL attribue lui-même le numéro suivant à chaque insertion.                                 | idIntervention INT NOT NULL AUTO_INCREMENT                                      |
| FOREIGN KEY ... REFERENCES            | Clé étrangère : la valeur doit exister dans la table référencée.                              | FOREIGN KEY (idVehicule) REFERENCES vehicule (idVehicule)                       |
| ON DELETE CASCADE                     | Si le parent est supprimé, les lignes enfants le sont aussi.                                  | fk_reparation_intervention                                                      |
| ON DELETE SET NULL                    | Si le parent est supprimé, la clé étrangère de l'enfant passe à NULL (la ligne reste).        | fk_intervention_technicien                                                      |
| ON DELETE RESTRICT                    | Interdit de supprimer le parent tant qu'il a des enfants.                                     | fk_vehicule_client ,<br>fk_paiement_intervention                                |
| ON UPDATE CASCADE                     | Si l'identifiant du parent change, il est mis à jour chez les enfants.                        | Présent sur presque toutes les clés étrangères.                                 |
| UNIQUE                                | Interdit deux lignes avec la même valeur dans la colonne.                                     | utilisateur.email , vehicule.immatriculation ,<br>vehicule.numeroChassis        |
| NOT NULL                              | La colonne est obligatoire.                                                                   | vehicule.marque , reparation.description                                        |
| DEFAULT                               | Valeur mise automatiquement si on n'en donne pas.                                             | statut ... DEFAULT 'PLANIFIEE' ,<br>dateDetection ... DEFAULT CURRENT_TIMESTAMP |
| ENUM(...)                             | La colonne n'accepte qu'une liste fermée de valeurs.                                          | anomalie.niveau<br>ENUM('FAIBLE', 'MOYEN', 'CRITIQUE')                          |
| DECIMAL(10,2)                         | Nombre exact à 2 décimales, adapté aux montants (pas d'erreur d'arrondi).                     | reparation.cout , paiement.montant                                              |
| CREATE INDEX / KEY                    | Index : accélère les recherches sur une colonne.                                              | KEY fk_vehicule_client (idClient)                                               |
| START TRANSACTION , COMMIT , ROLLBACK | Grouper plusieurs modifications : tout est validé ( COMMIT ) ou tout est annulé ( ROLLBACK ). | Suppression d'un client dans admin/clients.php .                                |
| SELECT ... FOR UPDATE                 | Lire une ligne en la verrouillant jusqu'à la fin de la transaction.                           | Création d'un paiement dans includes/payments.php .                             |

| Commande / mot-clé                            | À quoi elle sert                                                               | Exemple dans la base                                                    |
|-----------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| GRANT / REVOKE                                | Donner ou retirer des droits à un compte MySQL.                                | GRANT SELECT, INSERT, UPDATE, DELETE ON charles.* TO 'app'@'localhost'; |
| SHOW TABLES ,<br>DESCRIBE , SHOW CREATE TABLE | Lister les tables, voir les colonnes d'une table, voir sa définition complète. | SHOW CREATE TABLE intervention;                                         |

### À retenir

- L'ordre d'écriture d'un SELECT est toujours : SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT .
- WHERE filtre les lignes avant le regroupement, HAVING filtre les groupes après.
- UPDATE et DELETE sans WHERE touchent toute la table.
- EXPLAIN permet de vérifier une requête sans l'exécuter.

## 6.4 Dix exercices d'entraînement

Chaque exercice part d'un besoin réel de l'application, donne la requête solution, le résultat obtenu sur la base et l'explication. Essayez d'écrire la requête vous-même avant de lire la solution.

### Q90. Exercice 1 (facile) : combien de véhicules sont enregistrés dans SmartAutoTrack ?

```
SELECT COUNT(*) AS nb_vehicules
FROM vehicule;
```

```
nb_vehicules
9
```

COUNT(\*) compte toutes les lignes de la table `vehicule`. AS `nb_vehicules` donne un nom clair à la colonne du résultat. Il n'y a pas de WHERE, donc toutes les lignes sont comptées.

### Q91. Exercice 2 (facile) : quels sont les 3 véhicules qui ont le plus de kilomètres ?

```
SELECT idVehicule, marque, kilometrage
FROM vehicule
ORDER BY kilometrage DESC
LIMIT 3;
```

```
idVehicule | marque | kilometrage
919        | Iveco  | 142000
918        | Citroen| 65000
916        | Renault| 38000
```

ORDER BY `kilometrage DESC` trie du plus grand au plus petit kilométrage, puis `LIMIT 3` ne garde que les trois premières lignes. Remarque : la Peugeot 308 a aussi 38 000 km ; en cas d'égalité, MySQL ne garantit pas laquelle sort en premier. Pour un ordre stable, on ajouterait un second critère : `ORDER BY kilometrage DESC, idVehicule`.

### Q92. Exercice 3 (facile) : combien d'interventions y a-t-il dans chaque statut ?

```
SELECT statut, COUNT(*) AS nb
FROM intervention
GROUP BY statut
ORDER BY nb DESC;
```

|           |  |    |
|-----------|--|----|
| statut    |  | nb |
| PLANIFIEE |  | 5  |
| TERMINEE  |  | 4  |
| EN_COURS  |  | 1  |

`GROUP BY statut` forme un groupe par valeur de statut ; `COUNT(*)` compte les lignes de chaque groupe. Le statut `ANNULEE` n'apparaît pas car aucune intervention ne l'a : `GROUP BY` ne crée que les groupes qui existent réellement. C'est exactement ce genre de requête qui alimente les compteurs des tableaux de bord.

### Q93. Exercice 4 (moyen) : quels clients possèdent au moins deux véhicules ?

```
SELECT idClient, COUNT(*) AS nb_vehicules
FROM vehicule
GROUP BY idClient
HAVING COUNT(*) >= 2;
```

|          |  |              |
|----------|--|--------------|
| idClient |  | nb_vehicules |
| 999012   |  | 5            |

On regroupe les véhicules par propriétaire (`GROUP BY idClient`), puis `HAVING COUNT(*) >= 2` ne garde que les groupes d'au moins deux véhicules. On ne peut pas écrire `WHERE COUNT(*) >= 2` : au moment où `WHERE` s'applique, les groupes n'existent pas encore. Ce client est bien un client entreprise : sa ligne dans `client` a `typeClient = 'ENTREPRISE'` et il a une ligne dans la table `entreprise` (vérifié). Il gère donc une flotte de véhicules.

### Q94. Exercice 5 (moyen) : afficher chaque intervention qui a une réparation, avec le statut et le coût de la réparation.

```
SELECT i.idIntervention, i.type, i.statut,
       r.statut AS statut_reparation, r.cout
FROM intervention i
INNER JOIN reparation r ON r.idIntervention = i.idIntervention;
```

|                |  |                   |  |          |  |                   |  |        |
|----------------|--|-------------------|--|----------|--|-------------------|--|--------|
| idIntervention |  | type              |  | statut   |  | statut_reparation |  | cout   |
| 918            |  | Entretien courant |  | TERMINEE |  | TERMINEE          |  | 145.00 |
| 926            |  | Entretien courant |  | TERMINEE |  | TERMINEE          |  | 120.00 |

`i` et `r` sont des alias (des surnoms) des tables. `INNER JOIN ... ON` associe chaque réparation à son intervention grâce à la clé étrangère `reparation.idIntervention`. Seules les interventions qui ont une réparation apparaissent : c'est la règle de la jointure interne. On écrit `r.statut AS statut_reparation` parce que les deux tables ont une colonne `statut` et qu'il faut les distinguer.

**Q95. Exercice 6 (moyen) : pour chaque véhicule, combien d'interventions a-t-il eues, y compris ceux qui n'en ont jamais eu ?**

```
SELECT v.marque, v.modele, COUNT(i.idIntervention) AS nb_interventions
FROM vehicule v
LEFT JOIN intervention i ON i.idVehicule = v.idVehicule
GROUP BY v.idVehicule, v.marque, v.modele
ORDER BY nb_interventions DESC;
```

| marque         | modele   | nb_interventions |
|----------------|----------|------------------|
| Peugeot        | 308      | 4                |
| Toyota Corolla | berline  | 3                |
| Renault        | Master   | 1                |
| Citroen        | Jumper   | 1                |
| Iveco          | Daily    | 1                |
| Cadillac       | escalade | 0                |
| Cadillac e     | escalade | 0                |
| Peugeot        | Boxer    | 0                |
| Mercedes       | Sprinter | 0                |

LEFT JOIN garde **tous** les véhicules, même ceux sans intervention (les colonnes de intervention valent alors NULL). L'astuce importante est COUNT(i.idIntervention) et non COUNT(\*) : COUNT(colonne) ignore les NULL, ce qui donne bien 0 pour un véhicule sans intervention. Avec COUNT(\*), ces véhicules afficheraient 1 à tort. On regroupe par v.idVehicule pour que deux véhicules de même marque et modèle ne soient pas fusionnés.

**Q96. Exercice 7 (moyen) : combien d'interventions n'ont encore aucune réparation enregistrée ?**

```
SELECT COUNT(*) AS interventions_sans_reparation
FROM intervention i
WHERE NOT EXISTS (
    SELECT 1 FROM reparation r
    WHERE r.idIntervention = i.idIntervention
);
```

|                               |
|-------------------------------|
| interventions_sans_reparation |
| 8                             |

Pour chaque intervention i, la sous-requête cherche une réparation liée. NOT EXISTS est vrai quand la sous-requête ne trouve **aucune** ligne. Le SELECT 1 veut dire « peu importe ce qu'on renvoie, on veut juste savoir s'il y a une ligne ». Vérification : 10 interventions au total, 2 avec réparation (exercice 5), donc 8 sans réparation.

**Q97. Exercice 8 (difficile) : quels véhicules roulent plus que la moyenne du parc ?**

```
SELECT marque, modele, kilometrage
FROM vehicule
WHERE kilometrage > (SELECT AVG(kilometrage) FROM vehicule);
```

| marque  | modele | kilometrage |
|---------|--------|-------------|
| Renault | Master | 38000       |
| Citroen | Jumper | 65000       |
| Iveco   | Daily  | 142000      |
| Peugeot | 308    | 38000       |

La sous-requête entre parenthèses est exécutée d'abord : elle calcule la moyenne des kilométrages (environ 34 833 km sur les 9 véhicules). La requête principale garde ensuite les véhicules au-dessus de cette valeur. On ne peut pas écrire directement `WHERE kilometrage > AVG(kilometrage)` : un agrégat n'est pas permis dans un `WHERE`, d'où la sous-requête. Remarque : trois véhicules ont 0 km (kilométrage non saisi), ce qui fait baisser la moyenne.

### Q98. Exercice 9 (difficile) : lister les anomalies encore ouvertes, les plus graves d'abord, avec le nombre de jours depuis leur détection.

```
SELECT v.marque, v.modele, a.niveau, a.statut,
       DATEDIFF(NOW(), a.dateDetection) AS jours_ouverte
FROM anomalie a
JOIN vehicule v ON v.idVehicule = a.idVehicule
WHERE a.statut IN ('NOUVELLE', 'EN_COURS')
ORDER BY FIELD(a.niveau, 'CRITIQUE', 'MOYEN', 'FAIBLE');
```

| marque  | modele | niveau   | statut   | jours_ouverte |
|---------|--------|----------|----------|---------------|
| Iveco   | Daily  | CRITIQUE | NOUVELLE | 9             |
| Citroen | Jumper | MOYEN    | EN_COURS | 11            |

La jointure récupère la marque et le modèle du véhicule concerné. `WHERE a.statut IN (...)` exclut les anomalies `TRAITEE` et `IGNOREE`. `DATEDIFF(NOW(), a.dateDetection)` calcule le nombre de jours entre aujourd'hui et la détection (cette valeur change chaque jour ; celle-ci est celle du jour du test).

`FIELD(a.niveau, 'CRITIQUE', 'MOYEN', 'FAIBLE')` renvoie la position du niveau dans la liste (1, 2 ou 3) : trier sur cette position met les anomalies critiques en tête, ce qu'un tri alphabétique ne ferait pas.

### Q99. Exercice 10 (difficile) : combien d'utilisateurs a chaque rôle (administrateur, technicien, client, garage) ?

```
SELECT
  CASE
    WHEN EXISTS (SELECT 1 FROM administrateur WHERE idAdministrateur = u.idUtilisateur) THEN
      'ADMIN'
    WHEN EXISTS (SELECT 1 FROM technicien WHERE idTechnicien = u.idUtilisateur) THEN
      'TECHNICIEN'
    WHEN EXISTS (SELECT 1 FROM client WHERE idClient = u.idUtilisateur) THEN
      'CLIENT'
    WHEN EXISTS (SELECT 1 FROM garage WHERE idUtilisateur = u.idUtilisateur) THEN
      'GARAGE'
    ELSE 'AUCUN'
  END AS role,
  COUNT(*) AS nb
FROM utilisateur u
GROUP BY role;
```

| role       | nb |
|------------|----|
| CLIENT     | 5  |
| ADMIN      | 1  |
| TECHNICIEN | 8  |
| GARAGE     | 1  |

Dans la base, le rôle n'est pas une colonne : il est déduit de la table « spécialisée » dans laquelle l'utilisateur apparaît (`administrateur`, `technicien`, `client`) ou du lien `garage.idUtilisateur`. Pour chaque utilisateur, `CASE` teste ces tables l'une après l'autre avec `EXISTS` et renvoie le premier rôle trouvé. On regroupe ensuite sur ce rôle calculé. Le total ( $5 + 1 + 8 + 1 = 15$ ) correspond bien au nombre d'utilisateurs, et aucun n'est sans rôle.

### Piège fréquent

Le jury peut demander : « Pourquoi `COUNT(i.idIntervention)` et pas `COUNT(*)` dans l'exercice 6 ? ». Réponse : avec un `LEFT JOIN`, un véhicule sans intervention produit quand même une ligne, remplie de `NULL` côté intervention. `COUNT(*)` compte cette ligne (résultat 1), alors que `COUNT(i.idIntervention)` ignore les `NULL` (résultat 0, qui est la bonne réponse).

## 6.5 Les 10 questions que le jury posera presque sûrement

### Q100. Présentez votre base de données en une minute.

La base `charles` est une base MySQL 8 de 19 tables (moteur InnoDB, encodage utf8mb4) qui gère le suivi des véhicules de SmartAutoTrack. Au centre, `utilisateur` contient les comptes ; chaque compte est spécialisé en `client` (lui-même `particulier` ou `entreprise`), `technicien` ou `administrateur`, ou est lié à un `garage`. Un client possède des `vehicule` ; sur un véhicule on signale des `anomalie`, on planifie des `intervention` qui donnent lieu à des `reparation` et à des `paiement`. Autour, des tables de suivi : `journalactivites` (historique), `notifications`, `messages`, documents des garages et des techniciens, et les tables d'analyse IA.

### Q101. Pourquoi avoir choisi une base relationnelle ?

Les données de l'application sont fortement liées entre elles (un client a des véhicules, un véhicule a des interventions, une intervention a des réparations et des paiements). Une base relationnelle représente ces liens avec des clés étrangères, que MySQL vérifie lui-même : il est impossible d'enregistrer une réparation pour une intervention qui n'existe pas. Elle offre aussi les transactions (indispensables pour les paiements) et le langage SQL pour produire les statistiques des tableaux de bord avec des jointures et des `GROUP BY`.

### Q102. À quoi servent les clés étrangères dans votre base ?

Une clé étrangère garantit qu'une valeur pointe vers une ligne qui existe réellement : par exemple `fk_reparation_intervention` oblige `reparation.idIntervention` à correspondre à une intervention existante. Elle définit aussi ce qui se passe à la suppression du parent : `CASCADE` (les enfants sont supprimés, ex. les réparations d'une intervention), `SET NULL` (le lien est effacé, ex. une intervention garde

son historique si le technicien est supprimé) ou `RESTRICT` (suppression interdite, ex. un client qui a encore des véhicules). C'est ce qu'on appelle l'**intégrité référentielle**.

### Q103. Que se passe-t-il quand on supprime un client ?

Si le client a encore des véhicules, des interventions ou des paiements, un `DELETE` direct sur l'utilisateur échouerait, car `vehicule`, `intervention` et `paiement` référencent le client en `ON DELETE RESTRICT` (vérifié dans `information_schema` : ce sont les 5 seules clés `RESTRICT` de la base). Le code `admin/clients.php` supprime donc, dans une transaction, d'abord les paiements, puis les interventions, puis les véhicules, puis l'utilisateur. Le reste suit automatiquement en cascade : les réparations et les entrées du journal liées aux interventions, les anomalies liées aux véhicules, et les lignes `client`, `particulier / entreprise`, `messages` et `notifications` liées à l'utilisateur.

```
SELECT TABLE_NAME, CONSTRAINT_NAME, REFERENCED_TABLE_NAME, DELETE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = DATABASE() AND DELETE_RULE = 'RESTRICT';
```

| TABLE_NAME   | CONSTRAINT_NAME          | REFERENCED_TABLE_NAME | DELETE_RULE |
|--------------|--------------------------|-----------------------|-------------|
| intervention | fk_intervention_client   | client                | RESTRICT    |
| intervention | fk_intervention_vehicule | vehicule              | RESTRICT    |
| paiement     | fk_paiement_client       | client                | RESTRICT    |
| paiement     | fk_paiement_intervention | intervention          | RESTRICT    |
| vehicule     | fk_vehicule_client       | client                | RESTRICT    |

### Q104. Comment l'accès aux données est-il protégé ?

D'abord contre l'injection SQL : toutes les requêtes qui reçoivent une saisie utilisateur passent par des requêtes préparées PDO (`prepare()` puis `execute([...])`), donc la saisie est traitée comme une valeur et jamais comme du code SQL. Ensuite, les mots de passe ne sont jamais stockés en clair : la colonne `motDePasse` contient un hachage produit par `password_hash()` et vérifié par `password_verify()`. Enfin, les identifiants de connexion MySQL ne sont pas écrits dans le code (variables d'environnement ou `config/local.php` non versionné), chaque page vérifie le rôle de l'utilisateur en session (`requireRole()`) et les actions sensibles exigent un jeton CSRF.

### Q105. Qu'est-ce qu'une transaction et où l'utilisez-vous ?

Une transaction regroupe plusieurs modifications qui doivent réussir ensemble ou pas du tout : `START TRANSACTION` (en PHP `beginTransaction()`) l'ouvre, `COMMIT` valide tout, `ROLLBACK` annule tout en cas d'erreur. On l'utilise pour la suppression d'un client (quatre `DELETE` enchaînés), l'inscription, la validation des garages et techniciens, la clôture des réparations, et la création d'un paiement, où `SELECT ... FOR UPDATE` verrouille la réparation pour que deux clics simultanés ne créent pas deux paiements. Les transactions fonctionnent car les tables sont en InnoDB.

#### Q106. Pourquoi client, technicien et administrateur n'ont-ils pas leur propre AUTO\_INCREMENT ?

C'est la traduction d'un **héritage** : un client « est un » utilisateur. Leur clé primaire ( `idClient` , `idTechnicien` , `idAdministrateur` ) est en même temps une clé étrangère vers `utilisateur.idUtilisateur` , donc ils reprennent le numéro de l'utilisateur. Les informations communes (nom, email, mot de passe) sont stockées une seule fois dans `utilisateur` , et chaque table spécialisée ne garde que ses colonnes propres ( `typeClient` , `specialite` ...). Le même principe s'applique à `particulier` et `entreprise` sous `client` .

#### Q107. Votre base est-elle normalisée ?

Oui, elle vise la troisième forme normale : chaque information est stockée à un seul endroit et dépend uniquement de la clé de sa table. Par exemple, la marque d'un véhicule n'est pas recopiée dans chaque intervention : l'intervention ne garde que `idVehicule` , et on retrouve la marque par une jointure. Cela évite les incohérences (une marque modifiée à un endroit mais pas à l'autre). Quelques colonnes redondantes existent pour la pratique, comme `idClient` dans `intervention` alors qu'on pourrait le retrouver via le véhicule ; c'est un choix pour simplifier les requêtes, à assumer devant le jury.

#### Q108. Pourquoi utiliser des ENUM pour les statuts ?

Un `ENUM` limite la colonne à une liste fermée de valeurs, par exemple `intervention.statut` n'accepte que `PLANIFIEE` , `EN_COURS` , `TERMINEE` ou `ANNULEE` . La base refuse donc une faute de frappe comme « `TERMINE` », ce qui garantit des compteurs justes dans les tableaux de bord. Combiné à `DEFAULT` (ex. `DEFAULT 'PLANIFIEE'` ), il donne aussi un état initial automatique. L'inconvénient est qu'ajouter une valeur demande un `ALTER TABLE` .

#### Q109. À quoi servent les index et les contraintes UNIQUE dans votre base ?

Un index est comme le sommaire d'un livre : il permet à MySQL de trouver une ligne sans lire toute la table. La clé primaire est un index, et InnoDB crée un index sur chaque clé étrangère (les `KEY fk_...` ), ce qui accélère les jointures. Les contraintes `UNIQUE` sont des index qui interdisent en plus les doublons : un email ne peut servir qu'à un seul compte, et une immatriculation ou un numéro de châssis qu'à un seul véhicule. On peut le vérifier avec `EXPLAIN` , qui montre par exemple que la mise à jour d'une anomalie par son identifiant utilise la clé `PRIMARY` et ne lit qu'une ligne.

#### À retenir

- Savoir réciter la présentation de la base (Q100) sans regarder ses notes.
- Connaître les trois comportements des clés étrangères : `CASCADE` , `SET NULL` , `RESTRICT` , avec un exemple de chaque dans la base.
- Sécurité = requêtes préparées + mots de passe hachés + identifiants hors du code + contrôle du rôle.
- Transaction = tout ou rien : `beginTransaction()` , `commit()` , `rollback()` .

### Piège fréquent

Le jury peut demander « Où sont les paiements en ligne dans la base ? ». La table `paiement` existe mais est vide pour l'instant. Le code du paiement Mobile Money ( `includes/payments.php` ) attend des colonnes supplémentaires ( `idReparation` , `referenceExterne` , `referenceCampay` ) qui ne sont pas encore dans la base ; la fonction `paymentsReady()` le détecte et désactive alors le paiement en ligne. Répondez honnêtement que cette fonctionnalité nécessite une migration ( `ALTER TABLE paiement ADD COLUMN ...` ) avant d'être activée.

## 7. Questions pièges sur les mots de passe

Ces deux questions testent deux choses différentes : la première, si vous connaissez les **bonnes pratiques de sécurité** ; la seconde, si vous **maîtrisez votre code**. Elles s'appuient sur les fonctions réelles du projet :

`hashPassword()` et `verifyPassword()` dans `config/config.php`.

### Q102. Le jury vous demande de rendre les mots de passe visibles dans la base de données. Que faites-vous ?

C'est très probablement une **question piège**. La bonne réponse est d'expliquer pourquoi il ne faut pas le faire, puis de proposer ce qu'on peut montrer à la place.

#### Ce qu'il faut comprendre

L'application ne stocke pas les mots de passe, mais leur **empreinte** (hash), calculée par `password_hash()` avec l'algorithme `bcrypt`. La colonne `utilisateur.motDePasse` contient une chaîne de 60 caractères de cette forme :

```
$2y$10$Kq3m1Zr8e7tV0pX... (60 caractères)
|
|  └─ sel aléatoire (22 caractères) puis empreinte
|  └─ coût : 2^10 tours de calcul
|  └─ algorithme bcrypt
```

- **C'est à sens unique.** On calcule l'empreinte à partir du mot de passe, mais on ne peut **jamais** retrouver le mot de passe à partir de l'empreinte. Ce n'est pas un chiffrement : il n'existe aucune clé pour « déchiffrer ».
- **Un sel aléatoire** est inclus dans chaque empreinte : deux utilisateurs qui ont le même mot de passe ont des empreintes différentes.
- **À la connexion**, `password_verify()` recalcule l'empreinte du mot de passe saisi avec le même sel et la compare à celle de la base.

Rendre visibles les mots de passe **existants** est donc techniquement impossible.

#### La réponse à donner

« Non, et c'est volontaire. Les mots de passe sont hachés avec `bcrypt` via `password_hash()`. C'est une fonction à sens unique : même moi, en tant que développeur ou administrateur, je ne peux pas les lire. Si la base était volée, les mots de passe resteraient protégés. Beaucoup de gens réutilisent le même mot de passe ailleurs (email, banque) : le stocker en clair les mettrait en danger. C'est aussi une exigence de protection des données personnelles. Si un utilisateur oublie son mot de passe, on ne le lui renvoie pas : on lui permet d'en définir un nouveau. »

#### Ce que vous pouvez montrer à la place

1. **Le hash dans phpMyAdmin** : la colonne ne contient que des empreintes illisibles.

```
SELECT email, motDePasse FROM utilisateur LIMIT 3;
```

2. **Deux hachages du même mot de passe sont différents** (lancez la commande deux fois de suite) :

```
C:\xampp\php\php.exe -r "echo password_hash('Garage@2026', PASSWORD_DEFAULT), PHP_EOL;"
```

3. **La réinitialisation** : `scripts/set_password.php` définit un nouveau mot de passe, puis on se connecte avec. C'est la seule façon correcte de « récupérer » un compte.

### Piège fréquent

Dire « on peut déchiffrer le mot de passe ». C'est faux : un hash ne se déchiffre pas. Et si le jury propose un chiffrement réversible (AES) : il sert aux données qu'on doit relire (un numéro de document, par exemple), pas aux mots de passe, car celui qui obtient la clé obtient tous les mots de passe d'un coup.

### À retenir

- **Hacher ≠ chiffrer.** Un hash ne se déchiffre pas.
- `password_hash()` pour enregistrer, `password_verify()` pour vérifier.
- Un mot de passe oublié se **réinitialise**, il ne se récupère pas.

## Q103. Le jury vous demande d'enlever la fonction de hachage pour vérifier que vous maîtrisez votre code. Que faites-vous ?

Ici, le jury ne demande plus si c'est une bonne idée : il veut voir si vous savez **où se trouve le hachage** dans votre code et **ce que la modification va provoquer**. Annoncez dès le départ que vous remettrez le code en état ensuite.

### 1. Où se trouve le hachage

Tout passe par **deux fonctions centralisées** dans `config/config.php` :

```
function hashPassword($password) {
    return password_hash($password, PASSWORD_DEFAULT); // enregistrement
}

function verifyPassword($password, $hash) {
    return password_verify($password, $hash);           // connexion
}
```

| Fichier                                                                                                      | Rôle                                                                                  |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>auth/register.php</code>                                                                               | <code>hashPassword()</code> à l'inscription                                           |
| <code>auth/login.php</code>                                                                                  | <code>verifyPassword()</code> à la connexion                                          |
| <code>profile.php</code>                                                                                     | Vérifie l'ancien mot de passe, enregistre le nouveau                                  |
| <code>admin/garages.php</code> , <code>admin/techniciens.php</code> ,<br><code>garage/techniciens.php</code> | <code>hashPassword()</code> à la création de comptes                                  |
| <code>scripts/set_password.php</code>                                                                        | Appelle <code>password_hash()</code> <b>directement</b> , sans passer par la fonction |

**Point fort à dire** : « Comme j'ai centralisé le hachage dans deux fonctions, je n'ai que deux lignes à modifier pour toute l'application. »

## 2. La modification

```
function hashPassword($password) {  
    return $password; // stocké en clair  
(démonstration)  
}  
  
function verifyPassword($password, $hash) {  
    return hash_equals((string)$hash, (string)$password); // comparaison directe  
}
```

- `hash_equals` **plutôt que** `===` : la comparaison prend toujours le même temps, ce qui évite une attaque « par chronométrage ».
- **Pour être complet**, dans `scripts/set_password.php`, il faut aussi remplacer `password_hash($password, PASSWORD_DEFAULT)` par `$password`. Le signaler montre que vous connaissez votre code en entier.

## 3. Les conséquences, à annoncer avant qu'on vous les demande

1. **Les comptes existants ne peuvent plus se connecter** : leur colonne `motDePasse` contient encore les hashes bcrypt, et la nouvelle comparaison les compare au mot de passe saisi, donc elle échoue toujours. Il faut redéfinir le mot de passe du compte de démonstration.
2. **Les nouveaux mots de passe sont visibles en clair** :

```
SELECT email, motDePasse FROM utilisateur;
```

3. **Les règles de robustesse s'appliquent toujours** (8 caractères, minuscule, majuscule, chiffre, caractère spécial), car elles sont dans `includes/password_policy.php` et ne dépendent pas du hachage.

## 4. La démonstration

- 1 Modifier les deux fonctions dans `config/config.php`.
- 2 Créer un compte depuis la page d'inscription.
- 3 Montrer dans phpMyAdmin que son mot de passe apparaît en clair.
- 4 Se connecter avec ce compte : la connexion fonctionne toujours.

## 5. Remettre le code comme avant

C'est essentiel, et le dire montre votre maîtrise. Le projet est sous Git :

```
git checkout -- config/config.php scripts/set_password.php
```

Le hachage est rétabli instantanément. Concluez : « Le compte créé pendant la démo a un mot de passe en clair dans la base : je dois lui redéfinir un mot de passe, ou le supprimer, pour qu'il ne reste aucun mot de passe en clair. »

### Piège fréquent

Oublier que les comptes existants ne marcheront plus, ou oublier `scripts/set_password.php`, qui n'utilise pas la fonction centrale. Autre erreur : laisser le code modifié après la démonstration.

### À retenir

- Le hachage est centralisé : `hashPassword()` et `verifyPassword()` dans `config/config.php`.
- Annoncer les conséquences **avant** de modifier, et **restaurer** avec Git après.
- Phrase de conclusion : « Techniquement, c'est deux lignes. Mais en production je ne le ferais jamais : une seule fuite de la base exposerait les mots de passe de tous les utilisateurs. »

**Conseil :** entraînez-vous une fois avant la soutenance, sur une copie du projet ou après avoir fait un commit, pour être sûr de pouvoir revenir en arrière.